

BITÁCORAS COMPUTACIÓN GRÁFICA

2020-2021

2º curso – 2º cuadrimestre

Os seguintes apuntamentos foron recollidos en forma de bitácoras durante cada clase do curso polas seguintes persoas:

Adriana Aurora Rodríguez Oreiro, Adrián Vidal Lorenzo, Andrea Solla Alfonsín, Andrés Balea Domínguez, Belén Valle Cao, Diana Mascareñas Sande, Elena Segade Martínez, Hugo Vázquez Docampo, Inés Quintana Raña, Javier Beiro Piñón, Manuel Cid Domínguez, Martín Campos Zamora, Miguel Méndez Martínez, Nerea Freiría Alonso, Nicolás Vilela Pérez, Pablo Gil Pérez, Pablo Martínez González, Roberto de la Iglesia, Roque Fernández Iglesias, Teresa Gutiérrez Blanco



Non son apuntamentos recollidos de forma 100% formal e pode haber erros nalgúns datos ou fallos de formato. A pesar disto, pensamos que poden ser de gran axuda para os vindeiros cursos :)



Escola
Técnica
Superior
de Enxeñaría



Grado en Ingeniería Informática

2º curso – 2º cuatrimestre

Computación Gráfica

10-02-2021

Clase 1

Temas trabajados: Presentación, Tema 1

Índice

1. Presentación	3
1.1 Evaluación del curso	3
1.2 Sobre las prácticas	3
1.3 Material	3
1.4 Conocimientos previos	3
2. Tema 1: Introducción	4
2.1 Historia de la computación gráfica.....	4
2.2 Visión Humana.....	5
2.3 Frames por segundo	5
2.4 Hardware gráfico: dispositivos. Tubo de Rayos Catódicos	5
2.5 Terminales Vectoriales.....	6
2.5 Terminales Raster.....	6
2.6 Frame buffer.....	7
2.7 Terminales LED (raster).....	7
2.8 Arquitectura de los terminales raster.....	8
2.9 Proceso de rasterización	9
2.10 Conceptos generales de la computación gráfica.....	10

1. Presentación

1.1 Evaluación del curso

La parte de teoría del curso se corresponde con el 65% de la nota final y la práctica con el 20%. En las prácticas se evaluará:

- Funcionamiento.
- Aplicabilidad de la teoría.
- Claridad de código.
- Elegancia de código.

A parte de esto, se puede realizar un proyecto no obligatorio que supone un 15% de la nota. Se otorgará una puntuación de +0.25 por ampliación de la práctica modelado y + 0.25 por otros elementos.

1.2 Sobre las prácticas

- Son individuales.
- Usaremos dos librerías: OpenGL 1.2 y superior a 3.3 para implementar lo aprendido en teoría. •Sistema operativo Windows, se podrá usar Linux pero el profesor no solucionará problemas relacionados con compilación, etc.
- Lenguaje C/C++
- Compilador estándar (visual Studio). Podéis usar cualquier otro gcc, eclipse...
- Posiblemente para el proyecto o alguna práctica necesitéis alguna herramienta tipo Photoshop, Maya, Gimp, Blender...

EL proyecto no obligatorio se realizará de manera individual y debe cubrir todos los capítulos de la asignatura. En este se valorará su funcionamiento, programación y un informe final detallado. Ejemplos de proyectos: shooter, Google-earth, juegos de tanques y juegos de coches.

1.3 Material

Podéis encontrar información sobre el curso en la página Web de la asignatura.

- Transparencias de clase, semanalmente.
- Videos de prácticas.
- Páginas y tema de la bibliografía a los que hacer referencia.
- Ayudas/otros (para algunas prácticas).
- Otros elementos. Libros online.
- Redbook y manual de glut.

Libros:

- OpenGL Redbook version 8. OpenGL 4.5
- OpenGL Redbook version 2. OpenGL 1.1
- OpenGL en fichas (google books).
- Gráficos Por Computadora Con OpenGL Hearn Donald Baker M.
- Computer Graphics: Principles and Practice in C (2nd Edition) James D. Foley, Andries van Dam, Steven K. Feiner, John F. Hughes.
- Pauline.

Otras fuentes:

- www.opengl.org
- learnopengl.com
- <http://nehe.gamedev.net/> (ejemplos).

Otras que se irán indicando en las transparencias y el web

1.4 Conocimientos previos

Se necesitan conocimientos matemáticos tales como cálculo vectorial, producto vectorial, calcular ecuación de rectas, distancia de un punto a una recta, posición relativa de un punto, una recta y un plano. Además, será necesario saber realizar operaciones entre matrices y saber que la multiplicación de dos matrices no es conmutativa.

Pregunta 1: Sean $\vec{u}=(1,0,2)$ y $\vec{v}=(2,1,1)$ dos vectores. Entonces, ¿cuál es el ángulo que forman?

$$\vec{u} \cdot \vec{v} = |\vec{u}| |\vec{v}| \cos(x)$$

$$\vec{u} \cdot \vec{v} = (1,0,2) \cdot (2,1,1) = 2+0+2=4$$

$$|\vec{u}| = \sqrt{1^2+0^2+2^2} = \sqrt{5}$$

$$|\vec{v}| = \sqrt{2^2+1^2+1^2} = \sqrt{6}$$

$$4 = \sqrt{5} \sqrt{6} \cos(x)$$

$$\cos(x) = \frac{4}{\sqrt{5} \sqrt{6}}$$

2. Tema 1: Introducción

2.1 Historia de la computación gráfica

La computación gráfica empieza desde que existe la necesidad de tener una salida gráfica, que los ordenadores tengan una pantalla.

Años 50:

Se incorporaron los tubos de rayos catódicos (TRC). Las primeras televisiones con TRC eran las conocidas como pantallas con culo.

Uno de los primeros dispositivos de interacción eran los sistemas de control para sistemas aéreos: el radar, controlado por un operador que era capaz de detectar los aviones. Era tan malo que provocaban problemas de vista.

En 1956 aparecen también la primera aplicación de realidad virtual conocido como sensorama.

En el año 1958 aparece el primer videojuego, "Tenis for Two".

Años 60:

Aparecen los primeros videojuegos, el ratón con rueda (no era óptico como los actuales), los primeros sistemas CAD/CAM y el DVST y los primeros cascos de visión estereoscópica. Destaca Ivan Sutherland, considerado el padre de la computación gráfica moderna, que realizó trabajos en hardware y software.

Años 70

Aparecen los primeros computadores personales y las grandes empresas de SO y videojuegos tales como Apple, Microsoft, Atari (1972, Nolan Bushnell y Ted Dabney) y Lucas Film (1979).

Se recomienda ver [un documental sobre la historia de los videojuegos](#).

En 1972 se crea la primera videoconsola (Magnavox Odyssey).

Años 80:

Aparece el concepto de escritorio (desktop), los primeros guantes de realidad virtual, la tecnología BitMap, dando lugar a la creación de la GameBoy, las tarjetas gráficas, los primeros efectos especiales en el cine ("Tron" en 1982 y "Willow" (primer película que usa la técnica de morphing) en 1988) y Nintendo.

Años 90:

Aparece la primera versión de OpenGL, creado por SGI, que fue utilizado en películas como Terminator 2, y sistemas de visualización avanzados.

Se funda una de las principales empresas de las tarjetas gráficas, Nvidia, madre de todas las tarjetas gráficas actuales.

Historia de las tarjetas gráficas:

Tarjetas 1998 (1ª generación): Formado por TNT2, Range (ATI) y Voodoo3. Tenían una memoria propia que almacenaban texturas.

Tarjetas 1999-2000 (2ª Generación): Formada por GeForce256 y GeForce2 de nVidia. ATI. Las primeras GeForce con 2MB de memoria. Estas realizan el trabajo de las transformaciones por

vértice y la iluminación de la GPU. Además, se introdujo cierto nivel de configurabilidad, pero sin llegar en ningún momento a la posibilidad de programar la GPU.

Tarjetas 2001 (3ª Generación): Se utilizan en las Playstation 2 y Xbox One. Eran programables, lo que dio lugar a OpenGL versión 3.3. Estas, proporcionan ya la posibilidad de programación de vertex: permiten a la aplicación especificar una secuencia de instrucciones para procesar vértices.

DirectX 8 proporciona, mediante unos primitivos pixeles shaders (aunque estrictamente no puedan aún considerarse como tales), acceso a la configurabilidad a nivel de fragment.

En el 2002 aparece la versión 3.3 de OpenGL y la cuarta generación de tarjetas, con tarjetas ya programables. Incorporan fragment shaders y vertex shaders para programar la tarjeta, realizan transformaciones de vértices y operaciones de iluminación y sombreado.

Se utilizan diversos lenguajes de alto nivel para la programación de las tarjetas. Entre estos lenguajes se encuentran: C y CG (lenguaje propio de Nvidia), GLSL (estándar de OpenGL), DX HLSL(Microsoft), DirectX (desde v8.0) y CUDA (alto nivel).

Pregunta 2: ¿Quién patentó el primer videojuego?

Willy Higinbotham es el padre de los videojuegos.

Otras opciones:

Nolan Bushnell, es el que hizo la primera máquina recreativa.

Ted Dabney, es el ingeniero que ayudó a formar Atari.

Shigeru Miyamoto, es el padre del Mario Bros.

John Carmack, es el principal programador de la saga Doom y Quake. Actualmente trabaja en una empresa que trabaja en lanzar cohetes a Marte. Es uno de los grandes programadores de videojuegos de la historia.

2.2 Visión Humana

Los ojos tienen una cualidad, les llega una imagen, esta atraviesa la pupila y se refleja en la retina. Nuestro cerebro procesa esta imagen.

¿Cuántas imágenes se necesitan procesar?

Nuestro cerebro es capaz de procesar entre 24 y 30 imágenes/segundo (30Hz). La imagen queda reflejada un tiempo en la pupila. Este tiempo se conoce como persistencia.

2.3 Frames por segundo

En computación gráfica, el conjunto de imágenes creadas por segundo se denomina frames y al número de imágenes/segundo se le conoce como frames/segundo. Se supone que tenemos tiempo real cuando se generan más de 24f/s. Por ejemplo, el cine tiene 24 f/s.

Las televisiones antiguas tenían 50 f/s en Europa y 60 f/s en América, esto viene producido por 2 cosas: por el número de cosas que vemos y por la frecuencia de la electricidad, que es diferente en estos continentes.

En un videojuego se debe intentar tener tasas de entre 30 y 60 f/s.

La interactividad que se consigue realizando acciones sobre la pantalla se alcanza teniendo una tasa de f/s lo suficientemente grande, con el refresco de la pantalla y con un tiempo de respuesta con cierta velocidad. A veces hasta con 10 f/s puede parecer que es a tiempo real.

En resumen, se utilizan la persistencia de los fotogramas en nuestros ojos para generar imágenes en movimiento.

Un elemento importante es el número de frames que consigamos por segundo. Si un videojuego se atasca (no ha generado la imagen), se puede disminuir la calidad de las texturas para que vaya más rápido, consiguiendo así más f/s debido a que el dibujo que se necesita representar es menos complejo.

2.4 Hardware gráfico: dispositivos. Tubo de Rayos Catódicos

Los primeros monitores tenían cula y consistían en un haz de electrones que chocaban contra una pantalla y, entonces, iba generando la imagen. ¿Por qué generaba la imagen? Porque tenía un fósforo verde, un elemento fluorescente que produce luz una vez que se estimula. Cuando le daba

el haz de electrones, emitía luz durante un período de tiempo de 50 milisegundos, aproximadamente. Esto nos da la velocidad de refresco que tiene la pantalla. Al cabo de esos 50 ms, o se mantenía encendido, se apagaba o había que darle otra vez.

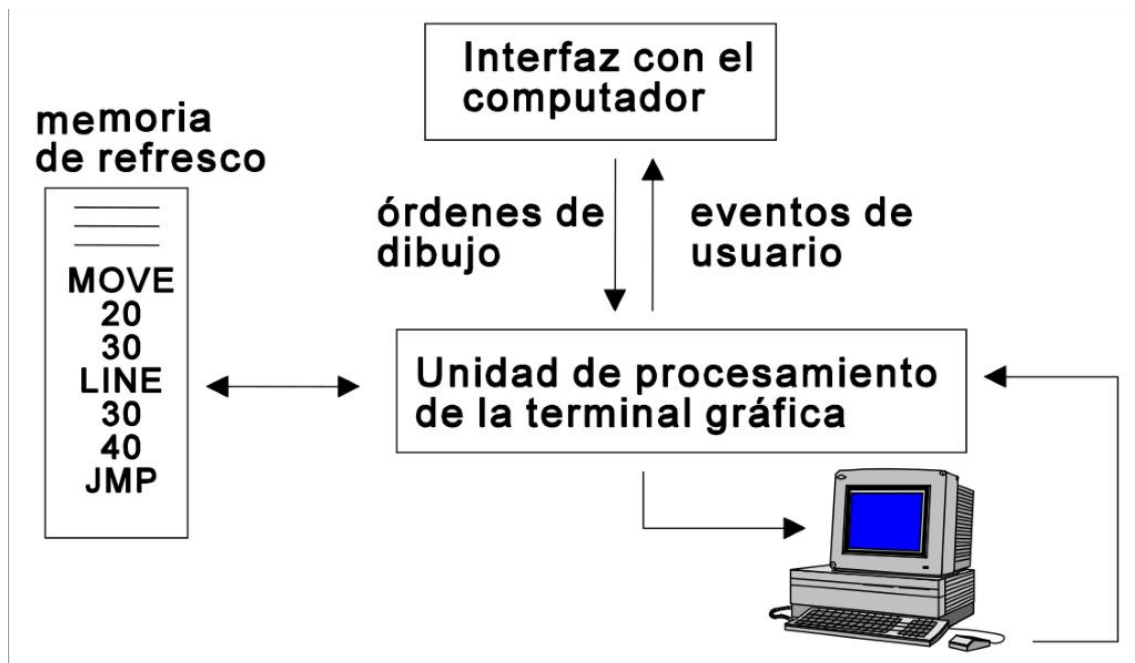
De esta forma generaban una imagen, haciendo que el barrido de la pantalla fuera lo suficientemente rápido como para llegar arriba otra vez. En esto se basaban las primeras pantallas creadas en 1875 por Carl Ferdinand Braum, que fueron las más usadas hasta 2005.

Con este tipo de tubos, como solo se tiene un haz que choca, se consigue un color en la pantalla, por lo que eran televisores monocromo (en blanco y negro). Para hacer televisión en color, se necesitan tres haces con tres tipos de receptor, cada uno de un color. Con esta mezcla, se conseguía tener el televisor en color.

La frecuencia mínima que podemos establecer está determinada por la persistencia del fósforo (tiempo que tarda en reducir un 10% su brillo inicial, normalmente de 10 a 60 microsegundos) y suele ser de 50Hz. Hablamos de CFF (Critical Flicker Frequency).

2.5 Terminales Vectoriales

Para introducir esto en un ordenador, al principio, como era más fácil, se conseguía especificando que imagen se necesitaba reproducir en la pantalla, desarrollándose así los terminales vectoriales. En estos terminales, se almacena información del dibujo que se va a dibujar, es decir, como hacerlo. El haz de electrones no tiene un orden particular, sino, que se va enfocando dependiendo del dibujo que se realizará.



Tenía una memoria en la que se almacenaban las instrucciones del dibujo y que se comunicaba con la pantalla y la interfaz del ordenador. El operador interactúa con la interfaz, que se lo enviaba a la unidad de procesamiento de la terminal gráfica, esta le indicaba las instrucciones a la memoria, dónde se almacenaban (principalmente movimientos) y se realizaban líneas por la pantalla. El correcto funcionamiento de estos terminales depende de la complejidad del dibujo.

2.5 Terminales Raster

En los años 70 aparecen los rasters, que son los terminales que llegan a nuestros días. Estos eran usados para la televisión y aplicados posteriormente en la computación. En estas terminales existe un direccionamiento de las líneas 625 EU

¿Cómo funcionan los terminales raster?

Los terminales raster, en vez de guardar la información de cómo realizar el dibujo, tiene una matriz bidimensional con todos los posibles puntos de la pantalla, denominados

pixeles. Cuando era la televisión en blanco y negro, solo se trataba de determinar si el píxel estaba apagado o encendido. Cuando la televisión pasó a color, el concepto de píxel se amplió. Esta matriz, se recorría de arriba a abajo y de izquierda a derecha, observando si los puntos se iluminaban o no. Esto es lo que realiza actualmente (concepto frame-buffer o color-buffer). El haz se genera ordenadamente (de arriba abajo y de izquierda a derecha).

Definición de píxel: Intersección de línea y columna.

2.6 Frame buffer

La zona de memoria en la cual se escribe el dibujo se denominaba frame buffer, debido a que era un buffer dónde se almacenaba la imagen que se iba a dibujar. Actualmente, es un área de la memoria la cual contiene toda la información necesaria para la generación de una imagen, pero, no es la imagen final. La imagen que se va a generar se almacena en otro buffer denominado color-buffer. No obstante, se utilizará indistintamente frame-buffer y color-buffer.

OpenGL → El frame buffer es donde se almacena toda la información de la escena que se debe dibujar y como renderizarla y el color buffer es el área de memoria de volcado directo al monitor (swap).

Importante, ¿cuanto ocupa el frame-buffer?

El frame buffer si se tiene una pantalla de 640×480 monocromo, ocupará 640×480 bits. Pero si se necesitan escalas de gris en lugar de blanco y negro, en lugar de 1 bit, se necesitará 1 byte, por lo que el tamaño del frame buffer va a crecer, pues se necesita más de un bit para definir un píxel. En este caso, se tiene 640×480 con 256 niveles de grises (que son 8 bits o 1 byte), se tendrán $640 \times 480 \times 8$, obteniendo el resultado en bytes. Esto para un color. El color aumenta x8 el tamaño de la pantalla. Entonces, el tamaño de la pantalla y del frame buffer dependerá del número de colores que se utilizarán.

Normalmente para dibujar el color se utiliza el concepto de RGB (rojo, verde y azul).

Cada uno de estos tres colores está representado por 1 byte, por lo tanto, cada píxel tiene 3 bytes asociados, uno para el rojo, otro para el verde y el último para el azul. Dependiendo de la complejidad de la paleta que se use para dibujar, se pueden indexar en una tabla, GLUT o paleta, o no. Nosotros vamos a suponer que son indexados.

Así, aparecen lo que se denomina profundidades de color, que son las siguientes:

La primera es un bit por píxel, que es encendido o apagado. Es el denominado monocromo.

La segunda es 2 bits por píxel → $2^2=4$ colores, denominado CGA.

La tercera es 3 bits por píxel → $2^3=8$ colores, denominado Zx spectrum que era uno de los primeros PC.

Después apareció el primer MAC que tenía 4 bits por píxel → $2^4=16$ colores.

Y, por último, 8 bits por píxel → $2^8=256$ colores, llamado VGA.

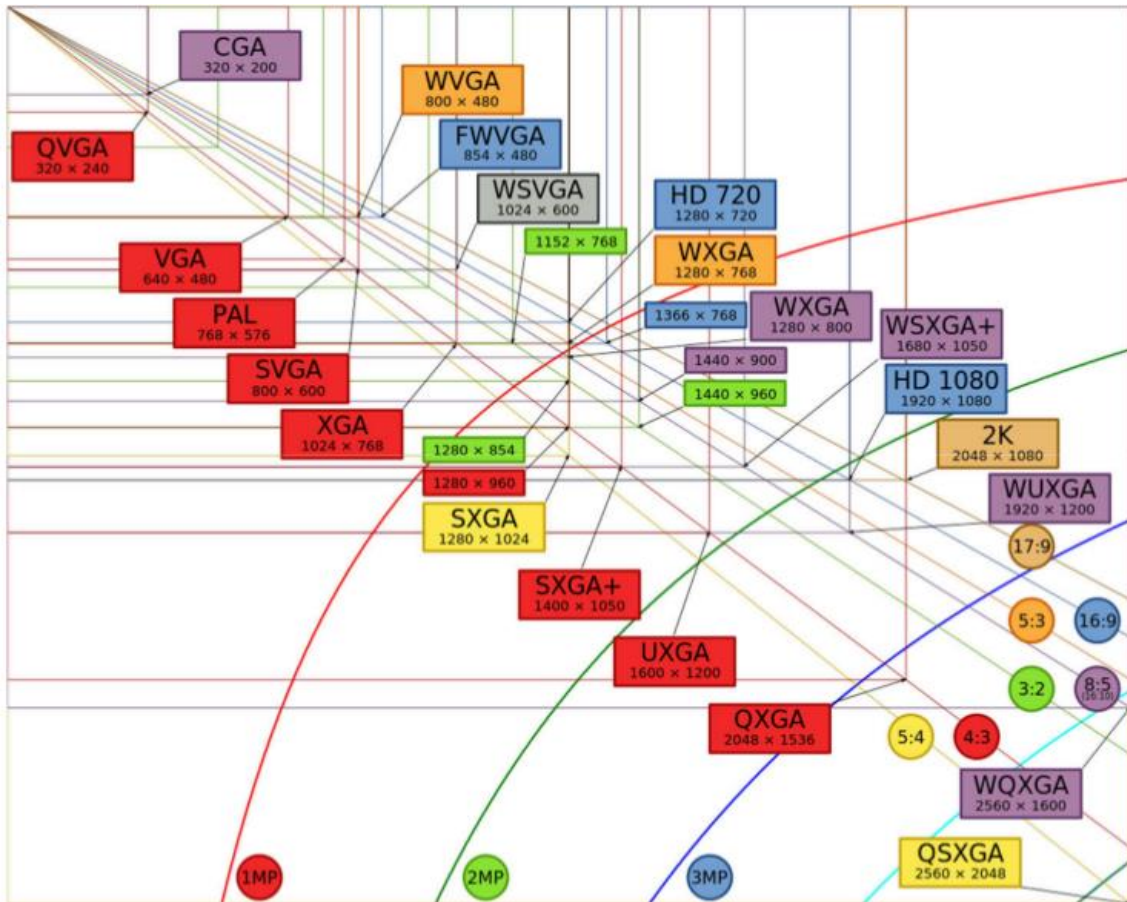
A partir de ese momento, se dieron cuenta que no eran necesarios tantos colores.

Se supone que el color real, actualmente, tenemos color real cuando nuestra tarjeta gráfica soporta 24 bits por píxel. Si son 24 bits por píxel y tenemos RGB, quiere decir que tenemos 8 bits por color o 1 byte por color.

2.7 Terminales LED (raster)

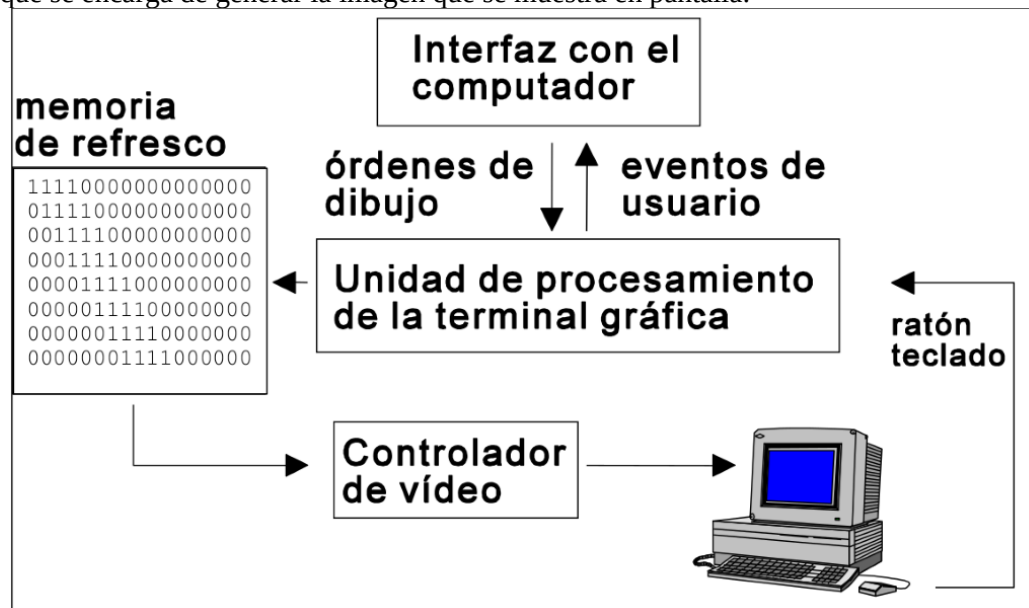
Actualmente esto se ve muy claro en las televisiones, ya que actualmente se utilizan pantallas con LED u otra tecnología que cuentan con píxeles muy pequeños, en donde cada píxel de la pantalla tiene unos LED con distinta configuración que dependiendo de si están encendidos o apagados, se genera la imagen. En general se suele utilizar 120 f/s (aprox 100 Hz) aunque solamente se necesitan 24f/s, para cansar menos la vista del espectador.

En los computadores, rápidamente se abandonaron las 625 líneas ya que estas se limitaban por la capacidad de emisión. Al poder procesar un mayor número de píxeles y aumentar la velocidad de refresco se disminuye el cansancio ocular del usuario.



2.8 Arquitectura de los terminales raster

El funcionamiento es similar al de los terminales vectoriales. A mayores, estos terminales raster cuentan con varias memorias dónde la final es el frame-buffer o color-buffer que almacena la imagen que se quiere dibujar. Además, tenemos una controladora de vídeo que se encarga de generar la imagen que se muestra en pantalla.



Pregunta 3: Si mi pantalla tiene una resolución VGA y color real cuanto ocupa el color buffer en MegaBytes? ¿y si la resolución de la pantalla es 1280*1024 píxeles?

VGA-> 640x480

$640 \times 480 \times 24/8 = 0.92 \text{ M}$

Pregunta 4: Tengo un juego en mi ordenador que tiene las siguientes tasas de refresco.

- 25 f/s durante 100 frames

- 50 f/s durante 100 frames

- 10 f/s durante 100 frames

- 25 f/s durante 100 frames

¿Cuál es la tasa media de refresco en frames por segundo?

Frames totales=400

Tiempo= 20 segundos

Media=400/20=20 frames/s

2.9 Proceso de rasterización

Hasta ahora usábamos un proceso muy simple: el terminal raster y teníamos nuestra aplicación y nuestra geometría (objeto que se desea dibujar) a la que llamamos escena. Esta geometría la poníamos en pantalla, al principio recorríamos la pantalla con unas líneas ordenadas con el concepto de píxel, entonces, cada uno de estos píxeles tenía un color dependiendo de la resolución y la profundidad del color.

El proceso mediante el cual transformamos una geometría en los píxeles de la pantalla, en general se denomina rasterización. Consiste en decidir como proyectar los puntos en la pantalla.

¿Como funciona un proceso de rasterizado?

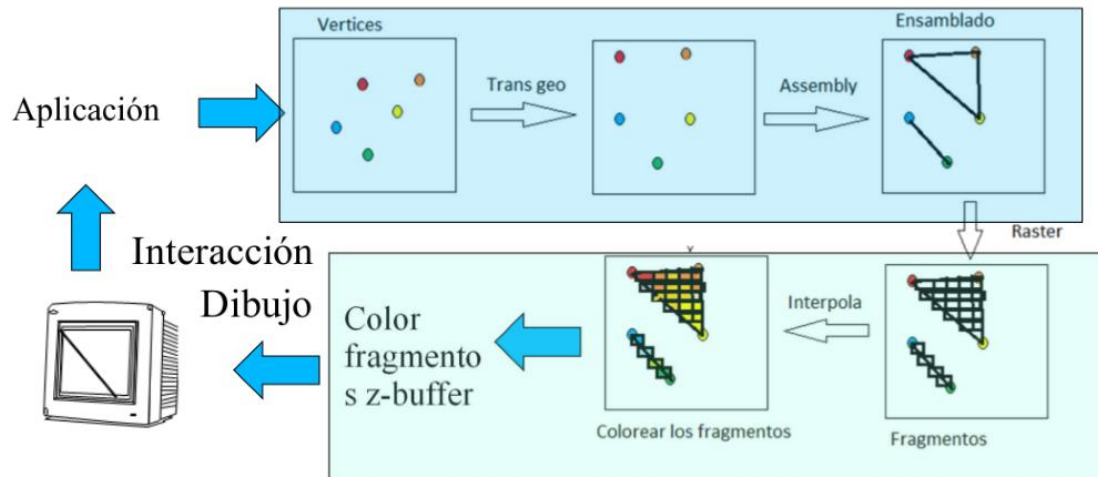
2.9.1 Modo inmediato de funcionamiento (Open GL hasta 1.2)

Las geometrías que se quieren realizar en la pantalla están compuestas por vértices. A base de puntos y uniendo estos se puede generar cualquier tipo de figura. Estos vértices forman parte de objetos. En una escena se pueden tener diversos objetos.

Si se tienen 2 objetos para dibujar, no se tienen por qué tener referenciados dichos objetos al mismo sistema de coordenadas. Todos los objetos tienen un origen de coordenadas diferente.

Al comienzo de la rasterización, todos los vértices que forman los objetos que componen nuestra escena están almacenados en un lugar denominado local space. Estos se deben mover al world space, que es compartido por todos los objetos.

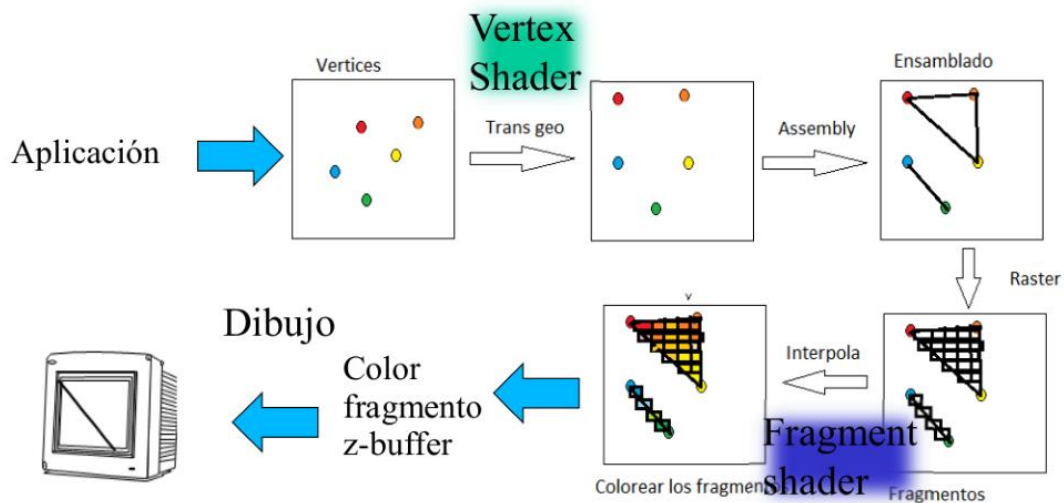
Una vez que los objetos se encuentran en el world space se deben unir los vértices para formar las caras de los objetos, lo que se denomina **ensamblado**. Una vez que se tienen los objetos ensamblados, lo que se hace es el proceso de **rasterización**, explicado anteriormente. Estas caras se transforman a píxeles, normalmente se transforman los vértices de las caras y después se recorren las líneas de píxeles para ver de qué color son cada uno, y entonces se les da un color mediante un proceso de **interpolación**. Este proceso da lugar a las caras de los objetos coloreadas, eso se manda a la pantalla o al color buffer, donde son dibujados. Este proceso se denomina pipe fija en OpenGL.



2.9.2 Vertex shader y fragment shader

El gran avance en OpenGL 3.3 es que se añaden unos programas antes y después de este proceso, los cuales nos permiten modificar muy rápidamente el proceso de coloreado y de mover los vértices. Dichos programas son:

- Vertex shader: Se encarga de mover los vértices.
- Fragment shader: Se encarga de colorear cada uno de los píxeles.



Pregunta 5: ¿El vertex shader y el fragment shader qué permiten?:

Vertex shader mueve los vértices y el fragment shader modifica los colores

2.10 Conceptos generales de la computación gráfica

Para evitar que el espectador pueda ver el cambio entre frames si no se realiza el dibujo lo suficientemente rápido, se hace uso del denominado **double buffer**.

Double buffer: Tiene 2 frame buffers: uno primario y otro secundario. El primario es lo que se ve y el secundario, en el que se dibuja. De esta forma, mientras veo una cosa estoy dibujando la siguiente y, cuando termino de dibujarla les hago un swap para pasar a ver esta. Este cambio es muy rápido y el espectador no lo ve.

Por lo tanto, cuando un videojuego se atasca, lo que se hace es bajar la calidad de las texturas para aumentar la velocidad de dibujo en el buffer secundario y que este intercambio sea más rápido. Las imágenes no se mostrarán hasta que esta no esté finalizada.

Si antes se tuviera, por ejemplo, 0,92MB de buffer, ahora con doble buffer se tendría 0,92x2 MB.

Depth Buffer:

Por ejemplo, si se está realizando un dibujo con varios elementos, y unos están delante de otros, ¿cómo se decide que elemento se dibuja antes o después?

Se tiene que dar alguna propiedad a esos elementos para que se dibujen de forma ordenada. Lo primero que se hacía, es utilizar el algoritmo del pintor. Este dibuja los objetos en el orden en que llegan. Pero, esto no resuelve algunos problemas, como, por ejemplo, cuando los elementos se cruzan.

Para solucionar el algoritmo del pintor, aparece el **z-buffer**.

El z-buffer o buffer de profundidad se utiliza para determinar si un objeto o parte de un objeto es visible en alguna escena. Entonces por cada píxel que se tenga que dibujar por cada objeto con unas coordenadas x e y sobre la pantalla se le da también una coordenada z. Esta coordenada se añade al frame buffer como buffer de profundidad y este buffer tiene un valor normalmente de 24 bits, pero también puede ser de 8, 16 o 32. Se le da un valor de profundidad al píxel que se quiere dibujar, de manera que cuando se desea comparar un objeto, se compara este valor.

Para asignar el valor del buffer de profundidad al píxel, se usa una función lineal.

En resumen, el buffer de profundidad permite ver los objetos que están detrás de otros.



Escola
Técnica
Superior
de Enxeñaría



Grado en Ingeniería Informática

2º curso – 2º cuatrimestre

Computación Gráfica

17-02-2021

Clase 02

Temas trabajados:

Introducción a OpenGL

(Teoría + Práctica 1)

Índice

1. Introducción a OpenGL.....	3
Concepto: ¿qué es OpenGL?	3
Características de OpenGL.....	3
Implementación de OpenGL.....	4
APP + GPU operaciones fijas (modo inmediato).....	4
APP + GPU operaciones no fijas (pipe modernas)	5
Otras características de OpenGL.....	6
GLUT: dirigida a eventos de ventana	6
Cómo comenzamos a trabajar	7
2. Parte práctica	8
Código main1a.c	8
Código main1b.c	11

1. Introducción a OpenGL

Concepto: ¿qué es OpenGL?

Es importante saber que casi todos los videojuegos soportan OpenGL, directx y últimamente Vulkan.

OpenGL es lo que se denomina una “API”, esto significa que es una interfaz de programación (de aplicaciones) con el hardware, por lo que no tiene una implementación específica. OpenGL indica qué tiene que hacer una determinada librería, y los fabricantes de tarjetas gráficas (o de software) deben desarrollar sus propias funciones para que la aplicación funcione.

Una ventaja de esto es que independientemente del hardware utilizado, OpenGL debería funcionar. Por ejemplo, si Nvidia o ATI, entre otras empresas, si dice que es compatible con OpenGL 4.6, significa que las funciones de 4.6 están implementadas, por lo que el programador sólo tiene la necesidad de saber cómo se llama la función, e invocarla si es necesario.

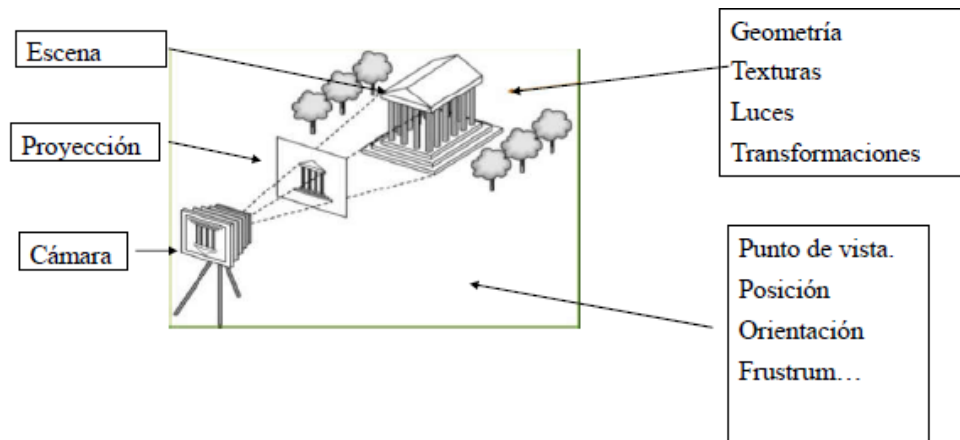
De esta forma, la implementación depende de los desarrolladores (permite implementaciones diferentes).

Características de OpenGL

- Mantenido por el grupo Kronos.
- Escrito en lenguaje C, aunque se puede utilizar bajo otros lenguajes.
- Diseñada para manipular escenas 3D.
- Tiene dos implementaciones: la clásica, de pipe fija (versión 1.3) y la pipe moderna, que no es fija, empieza en V2.0 (es la que introduce más cambios, no se suele utilizar). Se suele usar más las versiones a partir de la 3.3 hasta la actual, la 4.6. La única diferencia entre estas dos últimas es que la 4.6 soporta un mayor número de funciones (principalmente sobre el manejo de texturas).
- Al depender la implementación del programador, se puede utilizar bajo diferentes sistemas operativos. Es independiente de nuestro hardware.
- Da soporte a varios lenguajes para la programación de la GPU (programación). La gestión de los vértices y matrices cambia.
- Van añadiendo nuevas funcionalidades y extensiones.
- El objetivo de este curso es aprender las bases físicas y matemáticas de la computación gráfica, así como OpenGL, versión antigua y moderna.

Implementación de OpenGL

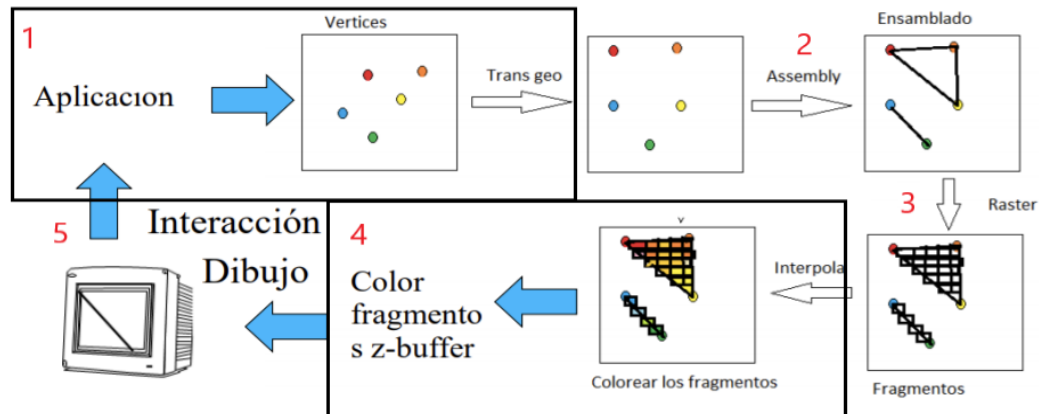
Trabaja con el modelo de la cámara sintética. Este consta de dos elementos principales: la cámara y la escena. La cámara la dividiremos en cámara y proyección:



1. La escena: es lo que nosotros vemos, donde están situados los objetos, o “aquello a lo que le vamos a hacer una foto”. Durante el curso veremos cómo se hacen las geometrías (los objetos), cómo se texturizan, cómo se les pone luces y cómo podemos mover esos objetos en el espacio.
2. La cámara: debemos colocar nuestra cámara. Se debe tener en cuenta la cámara en sí, cómo está situada la cámara, “cómo está y adónde mira”. Durante el curso miraremos el punto de vista: cómo se coloca, cómo está orientada, y cómo se diseña el frustum (el enfoque).
3. La proyección: el ajuste del zoom de la cámara.

APP + GPU operaciones fijas (modo inmediato)

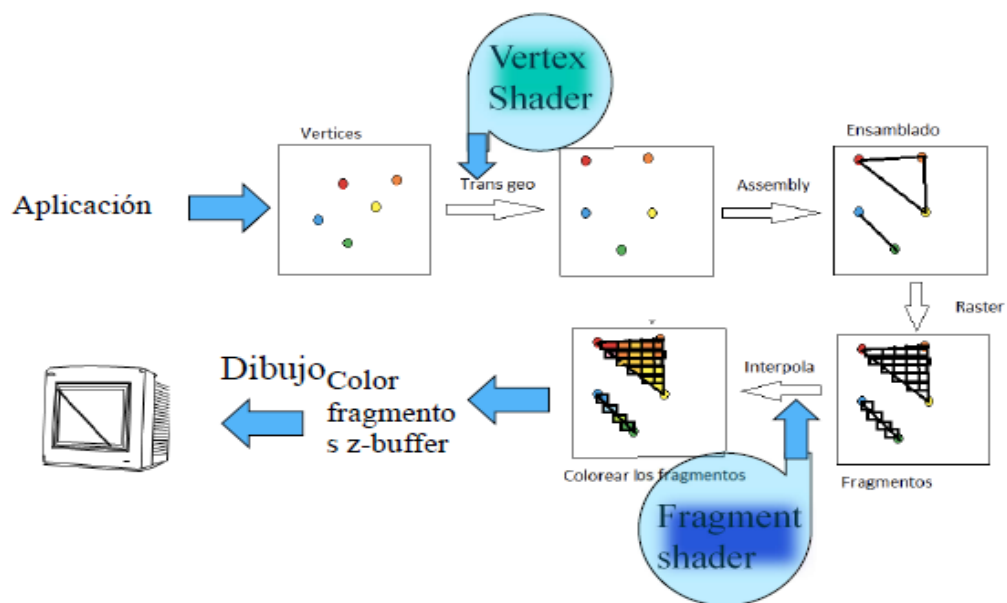
1. Tenemos nuestra aplicación, la cual le va a dar a OpenGL únicamente vértices. Estos compondrán una determinada figura. Debemos pensar dónde están referenciados. Por ejemplo, tenemos un vértice (1, 1, 1) pero ¿dónde está el origen? Normalmente cada objeto tiene su origen de coordenadas, que reciben el nombre de coordenadas locales del objeto. A partir de aquí tendrán lugar una serie de transformaciones que van a permitir colocar los vértices de nuestro objeto en la escena (world space).
2. Ensamblar: una vez tenemos colocados los vértices en nuestra escena, podemos ensamblarlos, es decir, enviaremos información de cómo unir los vértices para componer caras.
3. Rasterización: aplica la información que tiene de la cámara y busca a qué corresponde cada uno de los vértices de nuestro objeto sobre nuestra pantalla.
4. Como tenemos filas y columnas en toda la pantalla, se llevará a cabo un proceso de interpolación y, a los píxeles que componen la parte interior de las caras (fragmentos), se les dará un determinado valor. Una vez se les asigna dicho valor, lo que se hace es colorear. Una vez coloreados, aún no están en la pantalla, están en el frame buffer, se pasa el z-buffer, que permite elegir qué objetos se van a ver. Una vez tenemos todo esto, va al color buffer y se dibuja.
5. Feedback.



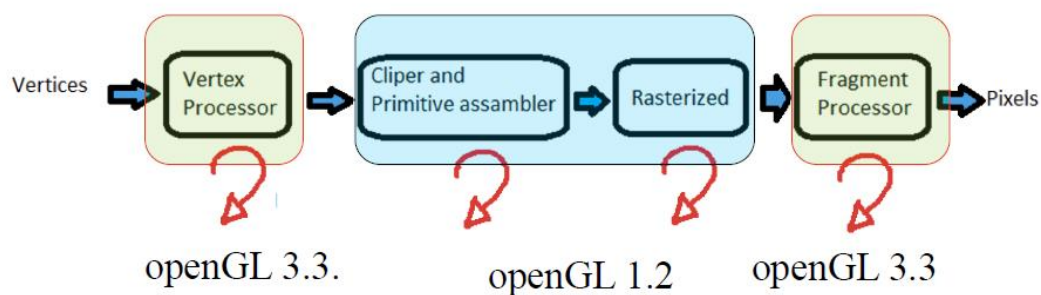
APP + GPU operaciones no fijas (pipe modernas)

Estas permiten incluir dos programas en GLSL:

- El **Vertex Shader**: realiza una serie de operaciones **antes** de las operaciones geométricas, es decir, él mismo se encarga de colocar los vértices.
- El **Fragment Shader**: colorea los píxeles.



Graphics Processing Unit (GPU)



Este esquema nos enseña dónde están colocados el Vertex Shader y el Fragment Shader. El primero realiza la transformación de la posición de vértice en la pantalla. Sus entradas son el vértice del triángulo junto con cualesquiera datos. La salida del sombreado de vértices es un vértice en una posición Transformada y posiblemente otra información, como la normal, color.

El Fragment Shader recibe la información del elemento a procesar (fragmentos), y la salida es el color del pixel (fragmentos coloreados).

Otras características de OpenGL

- **No tiene gestor de ventanas:** no nos permite abrir por sí mismo una ventana en Windows, Linux... Lo único que hace es coger vértices, moverlos y representarlos en un entorno de trabajo OpenGL. No puede controlar ningún elemento del ordenador (ni el teclado, ni el ratón...).
- **Se necesitan librerías auxiliares** (se pueden encontrar en Internet). Nosotros utilizaremos las dos más genéricas: glu, glaux, glut (hasta 2.0). Glfw 3.3 en adelante.
 - **Glut** (1996-2000) 3.7. Es la más común. The OpenGL utility toolkit. Hay una versión free que es más actual pero para nuestro caso no es necesario. Esta librería se utiliza para comunicar el OpenGL con el entorno de ventanas. (e.g. windows or Mac OS). Hay un paquete para cada sistema operativo. Glut.lib glut32.lib, glut32.dll. “Es útil sólo hasta la versión 1.2”. Escrita en C, con sus librerías y sus dlls (librerías de acceso dinámico).
 - **glaux Gl & Glu:** utiliza las cabeceras y librerías glaux, glu, .h, .lib, dll.
 - **Glfw:** es una biblioteca multiplataforma de código abierto para desarrollo de OpenGL, OpenGL ES y Vulkan. Proporciona una API simple para crear ventanas, recibir entradas y eventos. Lenguaje C y tiene soporte nativo para Windows, macOS y muchos sistemas similares a Unix que usan el sistema X Window, como Linux y FreeBSD. Versión OpenGL > 3.3.

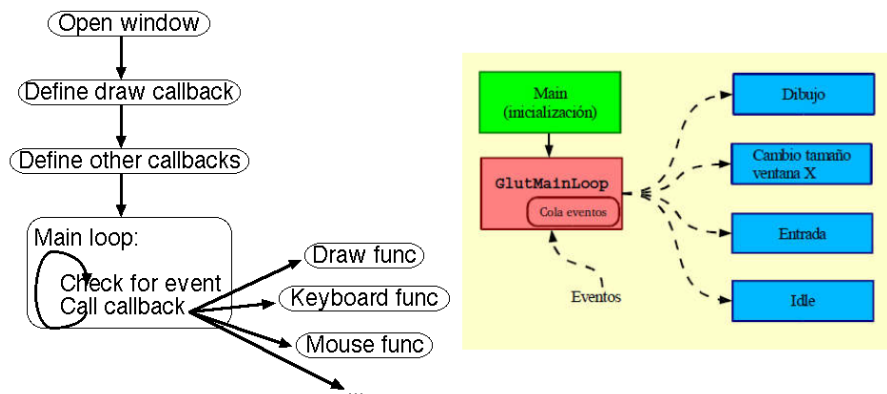
Glfw también se puede usar para V1.2. No usaremos solo la versión 3.3 porque existe mucho código en V1.2, su implementación es más fácil, y aprenderemos a usar el Glut además de Glfw.

GLUT: dirigida a eventos de ventana

Nos abre una ventana donde poder dibujar, definir unos callbacks (funciones que se repiten a lo largo del tiempo) donde controlaremos si alguien tocó una tecla o si la ventana ha hecho un resize, entre otros.

Tendremos un main con nuestras inicializaciones, lanzaremos el lazo principal de nuestra librería (en este caso GLUT), y esta controlará una serie de eventos. Uno de los eventos será el dibujo, que será donde estén nuestras librerías de OpenGL. Otro ejemplo es un cambio de tamaño, en el

que sí habrá un control de OpenGL, pero si tenemos una entrada de teclado o un proceso Idle (vago, se ejecuta cuando no hay otro proceso) no tendremos control de OpenGL.



Cómo comenzamos a trabajar

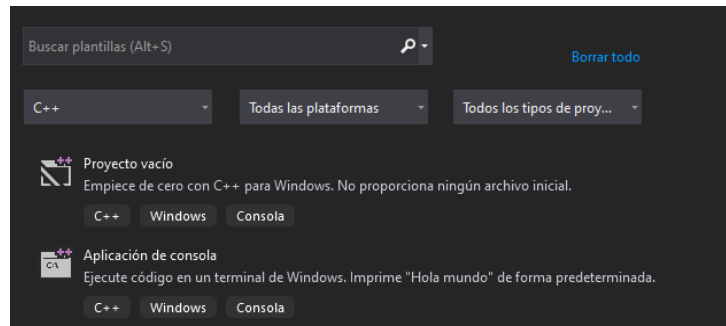
- Lo primero que debemos hacer para crear cualquier tipo de gráfico en OpenGL es crear un contexto de trabajo y una ventana de aplicación.
- Estas operaciones son específicas para cada sistema operativo.
- Debemos crear una ventana, definir un contexto y manejar la entrada del usuario por nosotros mismos o utilizar librerías ya desarrolladas.
- Veremos dos bibliotecas GLUT y GLFW (3.3). Hay bastantes más.
- Estas bibliotecas nos evitarán tener que trabajar directamente con el sistema operativo y nos dan una ventana y un contexto OpenGL para renderizar.
- La biblioteca GLUT proporciona soporte para C/C ++, FORTRAN y Ada. La distribución del código fuente de GLUT es portátil para casi todas las implementaciones y plataformas OpenGL. La versión actual es 3.7.(2000).

2. Parte práctica

Para la práctica se ha explicado cómo se preparan los proyectos y se ha explicado el código de cada uno de ellos, a continuación, se van a poner los pasos a seguir y cómo solucionar errores que han ido apareciendo:

Código main1a.c

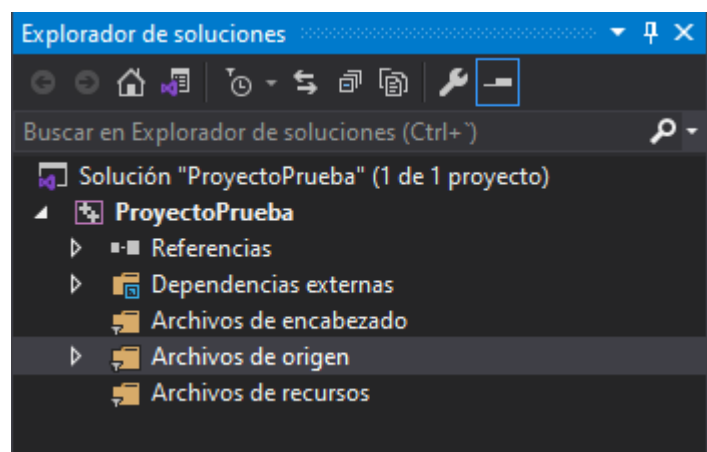
1. Descargamos el campus virtual el fichero “Packintro.zip”.
2. Abrimos Visual Studio (a partir de ahora, referenciado como VS).
3. Creamos un proyecto nuevo. Tendremos las opciones seleccionadas como se indica en la imagen y seleccionaremos “Aplicación de consola”.



4. Introducimos el fichero “main1a.c” en la carpeta del proyecto, este se encuentra siguiendo la ruta PackIntro -> p1_12

Disco local (C:) > Usuarios > itocd > source > repos > ProyectoPrueba				
Nombre	Fecha de modificación	Tipo	Tamaño	
.vs	17/02/2021 19:17	Carpeta de archivos		
ProyectoPrueba	17/02/2021 19:17	Carpeta de archivos		
main1a.c	17/02/2021 19:22	Archivo C	2 KB	
ProyectoPrueba.sln	17/02/2021 19:18	Visual Studio Solu...	2 KB	

5. Dentro de VS, damos click derecho a la carpeta que se encuentra en el “Explorador de soluciones” llamada “Archivos de origen”. Pulsamos en agregar -> Elemento existente... y añadimos el fichero “main1a.c” que acabamos de añadir a nuestra carpeta de proyecto.



6. Ahora vamos a instalar las librerías auxiliares que necesitamos para el funcionamiento del código:

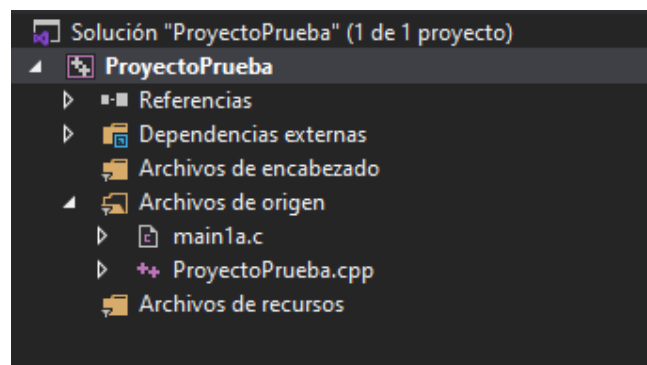
2. Parte práctica

- Creemos un directorio en nuestro proyecto (lo hacemos en la pantalla que se ve en el paso 4), este se llamará LIBRERIAS, aunque se puede llamar a gusto del usuario.
- Dentro vamos a crear otro directorio llamado GLUT, para que no se mezclen los archivos de la librería que vamos a añadir ahora, con los de otras librerías que pueden ser añadidas después.
- Dentro de GLUT, descomponemos el zip que se encuentra en “Packintro” llamado “glut.zip”.

Disco local (C:) > Usuarios > itocd > source > repos > ProyectoPrueba > LIBRERIAS > GLUT

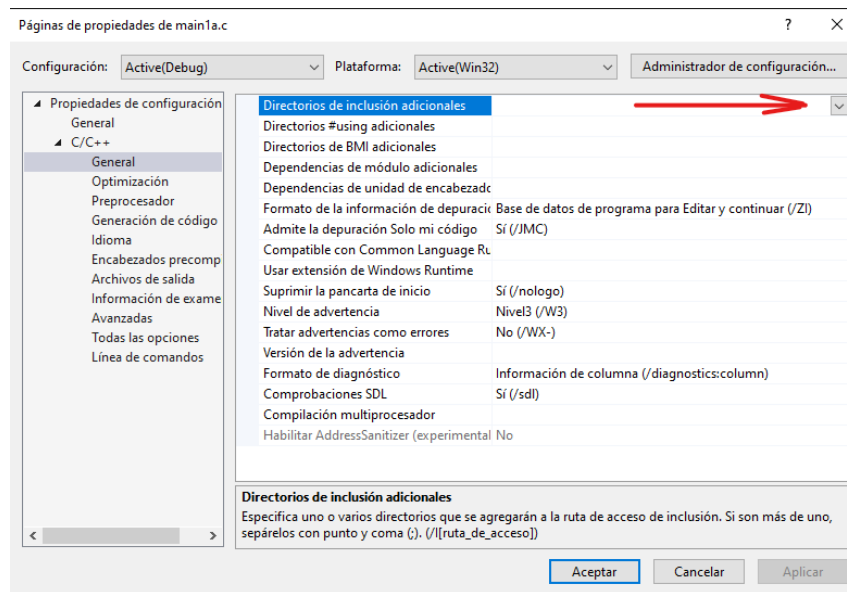
Nombre	Fecha de modificación	Tipo	Tamaño
eglew.h	17/02/2021 19:29	Archivo H	91 KB
GL.H	17/02/2021 19:29	Archivo H	68 KB
GLAUX.H	17/02/2021 19:29	Archivo H	12 KB
Glaux.lib	17/02/2021 19:29	Object File Library	1.282 KB
glew.h	17/02/2021 19:29	Archivo H	1.034 KB
glew32.dll	17/02/2021 19:29	Extensión de la ap...	334 KB
glew32.lib	17/02/2021 19:29	Object File Library	617 KB
glew32s.lib	17/02/2021 19:29	Object File Library	2.092 KB
GLFW.dll	17/02/2021 19:29	Extensión de la ap...	42 KB
glfw.h	17/02/2021 19:29	Archivo H	20 KB
GLFW.lib	17/02/2021 19:29	Object File Library	441 KB
GLFWDLL.lib	17/02/2021 19:29	Object File Library	16 KB
glm.h	17/02/2021 19:29	Archivo H	9 KB
GLU.H	17/02/2021 19:29	Archivo H	18 KB
glu32.dll	17/02/2021 19:29	Extensión de la ap...	114 KB
GLU32.LIB	17/02/2021 19:29	Object File Library	13 KB
glui.h	17/02/2021 19:29	Archivo H	54 KB
glui32.lib	17/02/2021 19:29	Object File Library	1.149 KB
glut.def	17/02/2021 19:29	Export Definition ...	3 KB
glut.h	17/02/2021 19:29	Archivo H	28 KB
glut32.dll	17/02/2021 19:29	Extensión de la ap...	473 KB
glut32.lib	17/02/2021 19:29	Object File Library	29 KB
glxew.h	17/02/2021 19:29	Archivo H	74 KB
libcd.lib	17/02/2021 19:29	Object File Library	5.359 KB
opengl32.dll	17/02/2021 19:29	Extensión de la ap...	670 KB
OpenGL32.LIB	17/02/2021 19:29	Object File Library	76 KB
README-win32.txt	17/02/2021 19:29	Documento de te...	26 KB
SOIL.h	17/02/2021 19:29	Archivo H	16 KB
SOIL.lib	17/02/2021 19:29	Object File Library	349 KB
wglew.h	17/02/2021 19:29	Archivo H	63 KB

- Dentro de VS, seguimos la siguiente ruta en los menús, importante, antes de ir a buscar en los menús, tener seleccionado el proyecto como en la imagen:

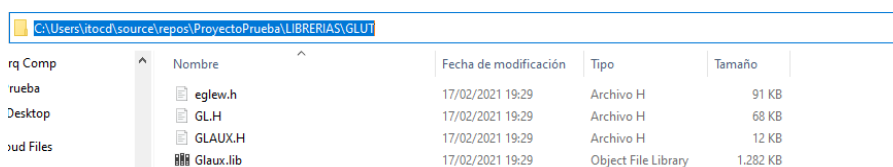


Proyecto -> Propiedades -> C/C++ -> General -> Directorios de inclusión adicionales

Al pulsar en “directorios de inclusión adicionales”, va a aparecer una flecha, la cual pulsaremos.



Pulsamos en “Editar”, en la nueva ventana que se va a abrir, copiaremos la ruta de nuestra carpeta GLUT.

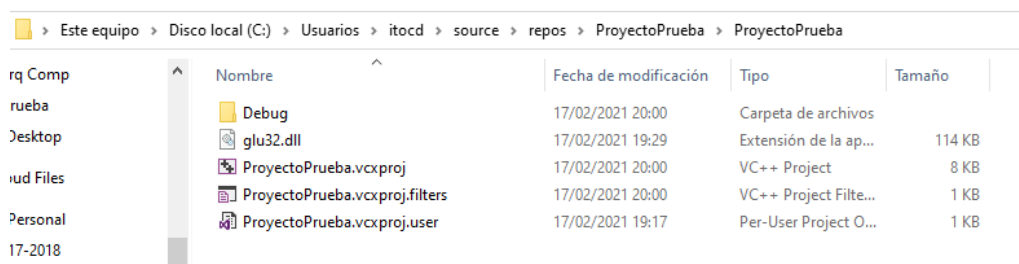


7. Ahora nos encontramos con errores debido al archivo “glut32.lib”. Para solucionarlo buscaremos en la siguiente ruta dentro de VS:

Proyecto -> Propiedades -> Vinculador -> General -> Directorios de bibliotecas adicionales

Y hacemos como en el paso 6, añadimos la ruta de GLUT.

8. Al compilar nos sale un error de ejecución. Tenemos que copiar el archivo “glu32.dll” en la carpeta con la siguiente ruta:



Si lo hemos hecho todo bien, debería de haber salido una ventana de color rojo en nuestra pantalla que será conocida como ventana de contexto, y otra que será la consola.

El código se encuentra explicado en los comentarios de este. Cosas destacadas del código que no están comentadas:

```
glutSwapBuffers();
```

```
glFlush();
```

Las líneas anteriormente descritas, se encargan de cambiar los buffers disponibles, si no están estas líneas podremos comprobar que no se cambia la pestaña de color. Lo mismo ocurre si no tenemos la línea “glClear(GL_COLOR_BUFFER_BIT|GL_DEPTH_BUFFER_BIT);” encargada de limpiar el buffer.

Código main1b.c

Seguimos los mismos pasos que para el proyecto “main1a.c”. A excepción de que, además de añadir el archivo en el directorio principal (el paso 4 anterior), añadiremos también el archivo “funciones_ejes.c”. Ambos se encuentran en el mismo sitio que “main1a.c”.

También tendremos que añadir el archivo “funciones_ejes.c” en Archivos de origen, dentro de Visual Studio.

En el paso 8, en vez de copiar el archivo “glu32.dll” se copia “glut32.dll”.

Sobre el código:

Cuando las coordenadas de los vértice son procesados estas son normalizadas, a la ventana de trabajo donde los valores de x, y, z varían de -1.0 a 1.0. Cualquier coordenada que caiga fuera de este rango será descartada / recortada y no será visible en la pantalla

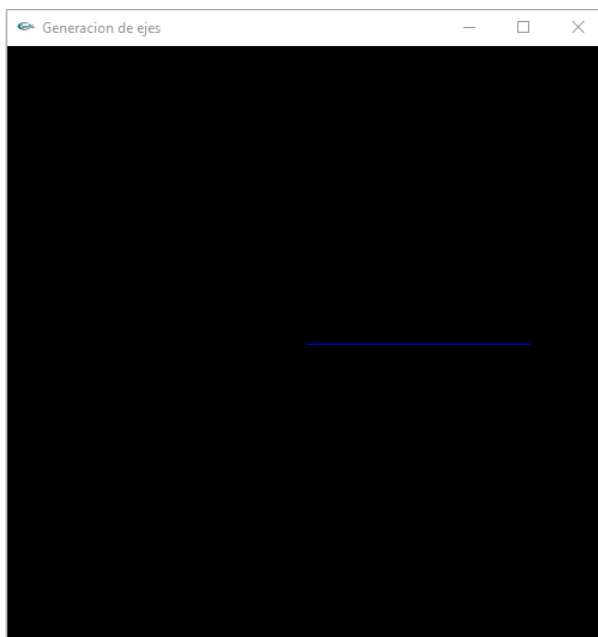
Función myCamara():

```
// Inicializamos la matriz del modelo a la identidad
glMatrixMode(GL_MODELVIEW);
glLoadIdentity();
myCamara();
```

La añadimos, esta función nos permitirá desplazarnos por el entorno y cambiar el punto de vista con las flechas del teclado.

Para crear los ejes de coordenadas:

Cada línea será creada con 3 líneas de código, veámoslo con la que viene creada por defecto:



```
glBegin(GL_LINES);
    glColor3f (0.0f,0.0f,1.0f);
    glVertex3f(0.0f,0.0f,0.0f);
    glVertex3f(.75f, 0.0f, 0.0f);
glEnd();
```

glColor3f() -> será el color de la línea

El par glVertex3f, será el primero para indicar el origen y el siguiente para indicar el destino.

Así con el código:

```
glBegin(GL_LINES);
```

```
    // eje x
```

```
    glColor3f (0.0f,0.0f,1.0f);
```

```
    glVertex3f(0.0f,0.0f,0.0f);
```

```
    glVertex3f(.75f, 0.0f, 0.0f);
```

```
    //eje y
```

```
    glColor3f(0.0f, 1.0f, 0.0f);
```

```
    glVertex3f(0.0f, 0.0f, 0.0f);
```

```
    glVertex3f(0.0f, 0.75f, 0.0f);
```

```
    //eje z
```

```
    glColor3f(1.0f, 0.0f, 0.0f);
```

```
    glVertex3f(0.0f, 0.0f, 0.0f);
```

```
    glVertex3f(0.0f, 0.0f, 0.75f);
```

```
glEnd();
```





Escola
Técnica
Superior
de Enxeñaría



Grado en Ingeniería Informática

2º curso – 2º cuatrimestre

Computación Gráfica

24-02-2021

Clase 03

Temas trabajados:

Tema 2

Índice

1. Modelado de Objetos.....	3
2. Modelado con OpenGL 1.2.....	6
3. Modelado con OpenGL 3.3.....	9
4. Ejercicios.....	15

1. Modelado de Objetos

▪ Introducción

Modelar consiste en dar forma a objetos. Debido a que en el mundo real los objetos son sólidos, existen diferentes tipos de modelado:

- De interior: No se estudiará en detalle. Resulta útil, por ejemplo, para simuladores de carácter médico. Suelen emplear representaciones basadas en Spatial-Partition (describen el interior dividiéndolo en pequeñas regiones sólidas que no se solapan entre sí, por ejemplo: marching-cubes).
- De exterior / De superficie: Se trabajará sobre representaciones de superficies que definen el entorno de un objeto sin referenciar a su volumen interno. En general, estas superficies serán cerradas (planos, cubos). En caso de trabajar con superficies abiertas, estas se descompondrán en estructuras más pequeñas y cerradas. Ejemplo: dos planos entrecortados se pueden modelar inicialmente como dos planos independientes y luego entrecortarlos para obtener la figura deseada.

Para ello se les proporcionarán propiedades a los objetos:

- Propiedades geométricas: Serán las que trataremos en este documento.
- Propiedades ópticas: luz, transparencia, textura...
- Propiedades físicas: elasticidad, actuación de la gravedad sobre el objeto...

NOTA: A la hora de modelar puede surgir el problema de que estemos dentro del objeto, por lo que este no se verá, lo que puede llevar a confusión.

▪ Ejes, Puntos y Vectores

Tanto los puntos, como los vectores y los objetos, se representarán sobre los tres ejes cartesianos.

- y, altura
- x, anchura
- z, profundidad (positivo hacia la cámara)

Los puntos informan sobre la ubicación del objeto (posición), mientras que los vectores indican la dirección hacia la que este se dirige (movimiento).

Una vez definido el espacio cartesiano se puede aplicar el modelo de cámara sintética, el cual permite dividir la escena de OpenGL en dos áreas diferenciadas:

- Escena: Compuesta por objetos situados en una determinada posición (coordenadas locales de los objetos)
- Cámara: Está determinada por su posición (0,0,0), su orientación y un foco (frustum).

▪ Triangulación

Una primitiva es la interpretación de un conjunto de vértices dibujados de una manera específica en pantalla. Ejemplos de primitivas: puntos, líneas, triángulos...

Para modelar cualquier objeto se emplea la técnica de teselación (o teselado), que consiste en la creación o transformación de figuras geométricas a partir de polígonos. Por lo general, se suele emplear el triángulo (triangulación), ya que esta es la figura geométrica más simple que delimita un plano.

En caso de utilizar otros polígonos para modelar superficies habría que comprobar que sus vértices fuesen coplanarios (y por ende, sus ejes).

Existe una relación empírica entre el número de triángulos necesarios para formar un polígono y el número de lados del mismo:

$$\text{número de triángulos} = \text{número de lados del polígono} - 2$$

Idealmente, una aproximación hecha con infinitos triángulos daría lugar a un círculo perfecto.

▪ Identificación de caras

Un plano está definido, entre otras cosas, por su vector normal (vector perpendicular al plano).

- Para calcular la normal se debe hallar el producto vectorial de dos vectores, $\vec{u}$ y $\vec{v}$, contenidos en el plano (en caso de no disponer de dichos vectores, se podrán hallar conociendo 3 puntos de dicho plano). El producto vectorial, $\vec{u} \times \vec{v}$, en $\mathbb{R}^3$ se calcula como el determinante de la matriz 3x3 conformada por las siguientes filas: vectores unitarios de los ejes cartesianos, componentes del vector $\vec{u}$, componentes del vector $\vec{v}$:

$$\begin{vmatrix} \vec{i} & \vec{j} & \vec{k} \\ \vec{u}_1 & \vec{u}_2 & \vec{u}_3 \\ \vec{v}_1 & \vec{v}_2 & \vec{v}_3 \end{vmatrix}$$

- Si simplemente se desea conocer la dirección del vector normal, bastaría aplicar la regla de la mano derecha según el recorrido de los vértices de la cara de una figura:
 - Si los vértices se recorren en sentido antihorario, la normal será positiva.
 - Si los vértices se recorren en sentido horario, la normal será negativa.

La estimación de la dirección del vector normal es muy importante, puesto que, por defecto, en OpenGL solo se ven los objetos cuya normal se dirige hacia la cámara (modelo de cámara sintética), por lo que hay que tener en cuenta el orden de creación de los vértices, ya que este indica el recorrido (horario o antihorario) que determinará la dirección de la normal. Este proceso de eliminación de caras (el interior del objeto no se renderiza) en función de la dirección de la normal se denomina Face Culling (Cull Face) que está habilitado por defecto en OpenGL, aunque se puede inhabilitar y mostrar todas las caras. Para ello existen las siguientes funciones:

- glEnable (GL_CULL_FACE)
- glDisable (GL_CULL_FACE)

En OpenGL, por defecto, la normal se dirigirá hacia la cámara en caso de que los vértices de la cara de un objeto se definiesen según un recorrido antihorario, y por tanto, se verá dicha cara (La función que establecería este funcionamiento sería glFrontFace(GL_CCW)). Aún así, esto se puede modificar con la función glFrontFace(GL_CW), para que la normal se dirija hacia la cámara cuando los vértices se definen según un sentido horario.

El vector normal de un plano puede ser normalizado (se puede transformar en un vector unitario). Esta característica es útil para procesos de iluminación de objetos (como calcular el valor de un

rayo reflejado), por lo que conviene normalizar dicho vector mediante `glEnable(GL_NORMALIZE)` al inicio del código.

Normalizar un vector es tan sencillo como dividir cada una de sus componentes por su módulo.

■ Tablas

Para la representación de un objeto en base a polígonos se necesita conocer información sobre sus vértices y sus atributos (propiedades físicas: color, luz, textura...), almacenando dichos datos de alguna forma, por ejemplo, mediante tablas, geométricas y de atributos, respectivamente. A su vez, estas tablas se pueden clasificar de la siguiente forma:

- Sin Indexar: Se especifica cómo están compuestos los vértices, y posteriormente por qué vértices está compuesto el polígono.
- Indexadas: Esta representación es la más usada y es la empleada por el formato .obj de `aliasWavefront`. La ventaja que ofrece esta representación con respecto a la anterior, es que en caso de que un vértice se emplee varias veces, basta con escribirlo una única vez en la tabla de vértices, pudiendo referenciarlo todas las veces que se requiera para cada polígono. Se compone de 2 tablas:
 - Tabla de polígonos
 - Tabla de vértices
- Doblemente Indexadas:
 - Tabla de polígonos (Cada polígono se conforma por aristas)
 - Tabla de aristas (Cada arista se conforma por 2 vértices y, si se desea, se puede indicar cada polígono que la comparte)
 - Tabla de vértices (Cada vértice se conforma por 3 coordenadas)

En la siguiente imagen se ve un ejemplo de representación mediante tablas con indexación simple:

```

• # Wavefront OBJ file: cubo.obj
• # Bounding box of geometry = (-0.5,-0.5,-0.5) to (0.5,0.5,0.5).

• mtlub cubo.mtl
• # Coordinates for object 'cubo'
• v -0.500000 -0.500000 0.500000
• v 0.500000 -0.500000 0.500000
• v -0.500000 0.500000 0.500000
• v 0.500000 0.500000 0.500000
• v -0.500000 0.500000 -0.500000
• v 0.500000 0.500000 -0.500000
• v -0.500000 -0.500000 -0.500000
• v 0.500000 -0.500000 -0.500000
• # 8 vertices
• vt -1.000000 0.0 0.0
• vt -1.000000 1.000000 0.0
• vt 0.0 0.0 0.0
• vt 0.0 1.000000 0.0
• vt 0.0 2.000000 0.0
• vt 0.0 3.000000 0.0
• vt 0.0 4.000000 0.0

•vt 1.000000 0.0 0.0
•vt 1.000000 1.000000 0.0
•vt 1.000000 2.000000 0.0
•vt 1.000000 3.000000 0.0
•vt 1.000000 4.000000 0.0
•vt 2.000000 0.0 0.0
•vt 2.000000 1.000000 0.0
•# 14 texture vertices

•usemtl default_2

•s off
•f 1/3 2/8 3/4
•f 2/8 4/9 3/4
•f 3/4 4/9 5/5
•f 4/9 6/10 5/5
•f 5/5 6/10 7/6
•f 6/10 8/11 7/6
•f 7/6 8/11 1/7
•f 8/11 2/12 1/7
•f 2/8 8/13 4/9
•f 8/13 6/14 4/9
•f 7/1 1/3 5/2
•f 1/3 3/4 5/2
•# 12 polygons

```

.obj

- Bounding box: Cubo mínimo que contiene al objeto. Se usa para las colisiones.
- mtlub: ubica la tabla de materiales

- Como se pretende generar un cubo, se especifican 8 vértices.
- A continuación, se especifican las coordenadas de las texturas (no se entró en detalle).
- Por último, se forman las caras mediante triangulación (ya que un cubo tiene 6 caras cuadradas, se requerirá el doble de triángulos). Se especifica el índice del vértice y el de la textura, de manera que la primera cara f 1/3 2/8 3/4, estaría compuesta por los vértices 1 (-0.5, -0.5, 0.5), 2 (0.5, -0.5, 0.5) y 3 (-0.5, 0.5, 0.5) y por las texturas 3 (0, 0, 0), 8 (1, 0, 0) y 4 (0, 1, 0).

2. Modelado con OpenGL 1.2

▪ Primitivas

Los vértices son el denominador común en cualquiera de las primitivas de OpenGL. Con ellos se definen puntos, líneas y polígonos. La función que define vértices es glVertex, y puede tomar de dos a cuatro parámetros de cualquier tipo numérico.

Existen diez primitivas en OpenGL, especificadas por diez macros distintas:

- | | |
|-----------------|---------------------|
| - GL_POINTS | - GL_TRIANGLE_STRIP |
| - GL_LINES | - GL_TRIANGLE_FAN |
| - GL_LINE_STRIP | - GL_QUADS |
| - GL_LINE_LOOP | - GL_QUAD_STRIP |
| - GL_TRIANGLES | - GL_POLYGON |

Para especificar una primitiva es necesario escribir el comando glBegin(Macro), definir los vértices necesarios y por último escribir la sentencia glEnd().

□ Puntos

Ejemplo de definición en OpenGL 1.2:

```
glBegin(GL_POINTS);
    glVertex3f(0.0f, 0.0f, 0.0f);
    glVertex3f(10.0f, 5.0f, 3.0f);
glEnd();
```

NOTA: Es posible alterar el tamaño de los puntos con la función glPointSize().

□ Líneas

Los vértices se toman de dos en dos para formar segmentos.

Ejemplo de definición en OpenGL 1.2:

```
glBegin(GL_LINES);
    glVertex3f(0.0f, 0.0f, 0.0f);
    glVertex3f(10, 10, 0.0f);
glEnd();
```

NOTA: Si se especifica un número impar de vértices, el último se ignora.

□ Tira de Líneas (Line Strip)

Los vértices están encadenados.

Ejemplo de definición en OpenGL 1.2:

```
glBegin(GL_LINE_STRIP);
    glVertex3f(0.0f, 0.0f, 0.0f);
    glVertex3f(2.0f, 1.0f, 0.0f);
    glVertex3f(2.0f, 2.0f, 0.0f);
glEnd();
```

NOTA: Con *GL_LINE_LOOP* el último vértice se encadenaría con el primero.

□ Triángulos

Cada tripleta de vértices especifica un triángulo. El color se puede configurar para cada triángulo o para cada vértice de un triángulo.

Ejemplo de definición en OpenGL 1.2:

```
glBegin(GL_TRIANGLES);
    glVertex3f(0.0f, 0.0f, 0.0f);
    glVertex3f(1.0f, 1.0f, 0.0f);
    glVertex3f(2.0f, 0.0f, 0.0f);
glEnd();
```

NOTA: El orden en el cual son introducidos los vértices es muy importante, ya que influye en la dirección del vector normal. Además, si solo se introducen 2 vértices, no ocurrirá nada.

□ Tira de Triángulos (Triangle Strip)

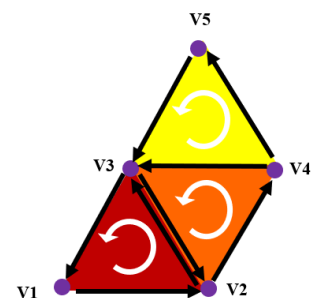
El primer triángulo se forma con los tres primeros vértices en el orden en el que son dados. A partir de ahí:

- Si el siguiente vértice es par, los dos vértices anteriores en la ristra de vértices dada se invierten. Esto es para mantener el sentido antihorario que garantiza que la normal se dirija a la cámara y se vea el objeto. Ejemplo: Dada la ristra de vértices V1, V2, V3 y V4:

Primer triángulo: V1, V2, V3
 Segundo triángulo: V3, V2, V4 (en vez de V2, V3, V4)

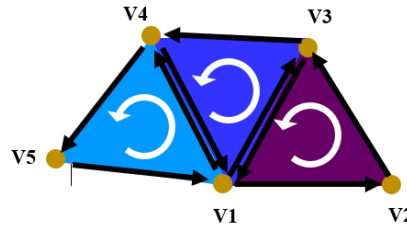
- Si es impar, se mantienen los 3 vértices consecutivos de la ristra inicial. Ejemplo: Dada la ristra de vértices V1, V2, V3, V4 y V5:

Primer triángulo: V1, V2, V3
 Segundo triángulo: V3, V2, V4 (en vez de V2, V3, V4)
 Tercer triángulo: V3, V4, V5



□ **Abanico de Triángulos (Triangle Fan)**

Una vez formado el triángulo inicial, se van añadiendo vértices que formen un nuevo triángulo cuyo vértice inicial sea siempre el vértice inicial del primer triángulo formado y como segundo vértice, el vértice final del último triángulo generado.



Ejemplo: Dada la ristra de vértices V1, V2, V3, V4 y V5, los triángulos formados mediante este método serían:

Primero triángulo: V1, V2, V3
 Segundo triángulo: V1, V3, V4
 Tercer triángulo: V1, V4, V5

□ **Tiras de cuadriláteros (Quad Strip)**

Para formar una tira de cuadriláteros dada una ristra de vértices, se toman los 4 primeros vértices para formar el primer cuadrado (los 2 primeros en el orden dado, mientras que los 2 siguientes en orden invertido). Para formar el resto de cuadriláteros se van uniando nuevos pares de vértices en orden inverso a los del último cuadrilátero creado.

□ **Polígonos**

Para la formación de polígonos se debe tener en cuenta que:

- Los polígonos deben ser necesariamente convexos, sin cavidades.
- Los lados no pueden cortarse.

MODOS POLIGONALES (glPolygonMode()): Función que permite especificar si una cara es visible o no, si se debe rellenar con líneas o con puntos... (Se usa para efectos especiales).

▪ **Listas de visualización**

Permite agrupar todos los vértices bajo un mismo índice entero.

Utilizando listas de visualización se puede renderizar un conjunto de vértices, previamente almacenados, simplemente invocando el índice de la lista en cuestión.

- **glGenLists(valor entero):** Lista correlativa de índices vacíos para almacenar tantas listas como se indican por argumentos.
- **glNewList(Id_Lista, GL_COMPILE):** A partir de la invocación de esta función se pueden insertar los vértices que manejará la lista entre las sentencias glBegin() y glEnd(). Para terminar, se debe invocar a la función glEndList(Id_Lista, GL_COMPILE) o glEndList(Id_Lista, GL_COMPILE_AND_EXECUTE).

- glCallList(Id_Lista): Invoca la lista creada con el índice pasado como argumento de la función y dibuja sus vértices.

Otra ventaja de estas listas, es que se suelen almacenar en la GPU, evitando la circulación de información mediante buses, que ralentizaría el proceso. Sin embargo, no pueden almacenar vértices definidos en función de algún cálculo, sino que deben estar definidos de manera fija.

3. Modelado con OpenGL 3.3

▪ Tipos de Shaders

En OpenGL 3.3 aparecen el Vertex Shader y el Fragment Shader:

- Vertex Shader: Es una función que recibe como parámetro una serie de vértices y que permite manipularlos (modificar su color, coordenadas de textura, orientación y tamaño de vértices). El objetivo consiste en transformar las coordenadas 3D de dichos vértices en otras que se obtendrán como salida de la función.
- Fragment Shader: Es una función que recibe información de cada fragmento (información necesaria para dibujar un píxel), con el propósito de obtener el color de cada píxel como salida. Se aplica en la última etapa de rasterizado. Posterior al Fragment Shader se realizan una serie de comprobaciones relativas al Alpha test, Blending test y Depth test.

Cada uno de estos Shaders debe ser creado (`glCreateShader()`), leído (`glShaderSource()`) y finalmente compilado (`glCompileShader()`). Una vez generados, se procede con la creación del Program Shader (`glCreateProgram()`), que recoge todos los Shaders con los que se desea trabajar, mediante `glAttachShader()`. Estos Shaders se vinculan (con `glLinkingProgram()`) y se prepara el Program Shader para ser usado (`glUseProgram()`).

De esta manera, cuando se quiera emplear un Shader en un programa, este se lee del Program Shader y posteriormente se utiliza.

▪ Vértices

Para generar figuras en OpenGL 3.3 se emplean Vertex Array Object (VAO). Estos objetos almacenan los vértices (y, si se desea, otros atributos) especificados en un vector, ya que no existen las funciones `glVertex()`, que permitían su definición.

Los VAO son contenedores de información del objeto a representar (como mínimo incluyen los vértices, pero también pueden incluir colores, texturas...). Esa información se almacena a su vez en buffers. Los VBO (Vertex Buffer Object) almacenan la información relativa a los vértices, colores.... Será por tanto en los VBO donde se indique el vector con los vértices, de manera que ese VBO se asignará al VAO con el que estemos trabajando.

En el caso de que trabajemos con el único atributo, *vértices*, un ejemplo de código para definir un VAO sería el siguiente:

```
void medianteArray() {
    float vertices[] = { 0.0f, 0.0f, 0.0f, // x
                        .5f, 0.0f, .0f,
                        0.0f, 0.0f, 0.0, // y
                        0.0f, .5f, 0.0f,
                        0.0f, 0.0f, 0.0f, // z
                        0.0f, 0.0f, .5f };

    glGenVertexArrays(1, &VAO);
    glBindVertexArray(VAO);

    glGenBuffers(1, &VBO);
    glBindBuffer(GL_ARRAY_BUFFER, VBO);

    glBufferData(GL_ARRAY_BUFFER, sizeof(vertices), vertices, GL_STATIC_DRAW);

    glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 3*sizeof(float), (void*)0);
    glEnableVertexAttribArray(0);

    glBindBuffer(GL_ARRAY_BUFFER, 0);
    glBindVertexArray(0);
}
```

Se crea el vector de floats que recoge los vértices

Se genera el VAO

Se activa el VAO, de manera que las funciones invocadas posteriormente afectarán a dicho VAO

Se genera el buffer donde se almacenarán los vértices

Se activa el buffer, de manera que las funciones invocadas posteriormente afectarán a dicho buffer

Se asigna el vector de vértices al buffer y al VAO que están “escuchando” y se detalla el modo de dibujo

Se especifica el número que identifica al atributo “vértices”, cuántos elementos del vector compone un vértice, la distancia de salto al próximo y dónde empieza el primer vértice, entre otras cosas

Se activa el buffer con el atributo “vértices” para su uso

Se desactiva el buffer (ya no “escucha” modificaciones)

Se desactiva el VAO (ya no “escucha” modificaciones)

Para emplear el VAO, una vez invocada la función `medianteArray()` que crea el VAO, se puede emplear el siguiente fragmento de código:

```
glUseProgram(shaderProgram);
glBindVertexArray(VAO);
glDrawArrays(GL_LINES, 0, 6);
glBindVertexArray(0);
```

Se indica cuál es el Program Shader a emplear

El VAO especificado se “pone a la escucha” (se activa)

Se dibujan las aristas del objeto especificando desde dónde empezar y el número de vértices a dibujar del VAO

Se desactiva el VAO para su uso (ya no “escucha”)

En caso de trabajar con más de un atributo, por ejemplo, almacenar colores en el vector, el código de generación de un VAO añadiría un par de líneas más:

```
void medianteArray() {
```

Se crea el vector de floats que recoge los vértices y los colores

```
float vertices[] = {
    //Vertices          //Colores
    0.0f, 0.0f, 0.0f,    1.0f, 1.0f, 1.0f, // 0
    .5f, 0.0f, 0.0f,    1.0f, 0.0f, 0.0f, //x
    0.0f, 0.0f, 0.0f,    1.0f, 1.0f, 1.0f, // 0
    0.0f, .5f, 0.0f,     0.0f, 1.0f, 0.0f, // y
    0.0f, 0.0f, 0.0f,    1.0f, 1.0f, 1.0f, // 0
    0.0f, .5f, 0.0f,     0.0f, 0.0f, 1.0f, // z
    0.0f, 0.0f, 0.0f,    1.0f, 1.0f, 1.0f, // 0
    .5f, .5f, 0.5f,      1.0f, 1.0f, 1.0f // mitad de 1,1,1
};
```

```
glGenVertexArrays(1, &VAO);
glBindVertexArray(VAO);
```

```
glGenBuffers(1, &VBO);
glBindBuffer(GL_ARRAY_BUFFER, VBO);
```

```
glBufferData(GL_ARRAY_BUFFER, sizeof(vertices), vertices, GL_STATIC_DRAW);
```

Se genera el primer atributo (“vértices”), identificado con 0, indicando cuántos elementos del vector compone un vértice, el salto al siguiente vértice y dónde empieza el primer vértice. Posteriormente, se activa este atributo para su uso.

```
glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 6*sizeof(float), (void*)0);
glEnableVertexAttribArray(0);
```

Se genera el primer atributo (“colores”), identificado con 1, indicando cuántos elementos del vector compone un color, el salto al siguiente color y dónde empieza el primer color. Posteriormente, se activa este atributo para su uso.

```
glVertexAttribPointer(1, 3, GL_FLOAT, GL_FALSE, 6*sizeof(float), (void*)(3*sizeof(float)));
glEnableVertexAttribArray(1);
```

```
glBindBuffer(GL_ARRAY_BUFFER, 0);
glBindVertexArray(0);
glDeleteBuffers(1, &VBO);
}
```

Para dibujar, el código en este caso sería el siguiente, una vez invocada la función medianteArray() que crea el VAO:

```
glUseProgram(shaderProgram);
glBindVertexArray(VAO);
glDrawArrays(GL_LINES, 0, 8);
```

▪ Vértices indexados

Para evitar escribir varias veces el mismo vértice, se pueden emplear vértices indexados. En este caso, se requiere, además del vector de vértices, un vector de índices, por lo que el código en este caso variaría ligeramente.

Ejemplo de generación de VAO con un único atributo:

```
void medianteArray() {
    float vertices[] = {    0.0f, 0.0f, 0.0f, // 0
                          .5f, 0.0f, 0.0f, //x
                          0.0f, .5f, 0.0f, // y
                          0.0f, 0.0f, .5f  // z
    };

    unsigned int indices[] = {0, 1, 0, 2, 0, 3 };
    // Se genera el vector de índices, que recoge el
    // orden en el que se recorren los vértices

    glGenVertexArrays(1, &VAO);
    glBindVertexArray(VAO);

    glGenBuffers(1, &VBO);
    glGenBuffers(1, &EBO);
    // Se generan los buffers donde se almacenarán los vértices e
    // índices, respectivamente

    // Se activa el buffer de vértices y se especifica el vector al que referencia
    glBindBuffer(GL_ARRAY_BUFFER, VBO);
    glBufferData(GL_ARRAY_BUFFER, sizeof(vertices), vertices, GL_STATIC_DRAW);

    // Se activa el buffer de índices y se especifica el vector al que referencia (ELEMENT_ARRAY_BUFFER
    // indica que se trata de un array de índices)
    glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, EBO);
    glBufferData(GL_ELEMENT_ARRAY_BUFFER, sizeof(indices), indices, GL_STATIC_DRAW);

    glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 3 * sizeof(float), (void*)0);

    glEnableVertexAttribArray(0);
    glBindVertexArray(0);
}
```

Para dibujar dicho VAO, una vez invocada la función medianteArray() que crea el VAO:

```
glUseProgram(shaderProgram);
glBindVertexArray(VAO);
glDrawElements(GL_LINES, 6, GL_UNSIGNED_INT, 0);
glBindVertexArray(0);
// La función de dibujo es diferente con
// indexación
```

Un ejemplo de código para un VAO con varios atributos y vértices indexados podría ser el siguiente:

```
void medianteArray() {
float vertices[] = {
    //Vertices          //Colores
    0.0f, 0.0f, 0.0f,    1.0f, 1.0f, 1.0f, // 0
    .5f, 0.0f, 0.0f,    1.0f, 0.0f, 0.0f, //x
    0.0f, .5f, 0.0f,    0.0f, 1.0f, 0.0f, // y
    0.0f, .5f, 0.0f,    0.0f, 0.0f, 1.0f, // z
    .5f, .5f, 0.5f,    1.0f, 1.0f, 1.0f // mitad de 1,1,1
};

unsigned int indices[] = {0, 2, 0, 3, 0, 4};

glGenVertexArrays(1, &VAO);
glBindVertexArray(VAO);

glGenBuffers(1, &VBO);
glGenBuffers(1, &EBO);

glBindBuffer(GL_ARRAY_BUFFER, VBO);
glBufferData(GL_ARRAY_BUFFER, sizeof(vertices), vertices, GL_STATIC_DRAW);

glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 6 * sizeof(float), (void*)0);
glEnableVertexAttribArray(0);

glVertexAttribPointer(1, 3, GL_FLOAT, GL_FALSE, 6 * sizeof(float), (void*)(3 * sizeof(float)));
glEnableVertexAttribArray(1);

glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, EBO);
glBufferData(GL_ELEMENT_ARRAY_BUFFER, sizeof(indices), indices, GL_STATIC_DRAW);

glDeleteBuffers(1, &VBO);
glDeleteBuffers(1, &EBO);
}
```

Para dibujar dicho VAO, una vez invocada la función `medianteArray()` que crea el VAO:

```
glUseProgram(shaderProgram);
glBindVertexArray(VAO);
glDrawElements(GL_LINES, 8, GL_UNSIGNED_INT, 0);
glBindVertexArray(0);
```

Los vértices especificados en un VAO se almacenan en la pipe y posteriormente son gestionados en el Vertex Shader y Fragment Shader. El código de un Vertex Shader podría ser el siguiente:

```
#version 330 core
layout (location = 0) in vec3 aPos;

void main() {
    gl_Position = vec4(aPos.x, aPos.y, aPos.z, 1.0);
}
```

Sentencias como “`layout (location = 0) in vec3 aPos`” permiten almacenar los elementos del atributo identificado con 0 del VAO activo en un vector denominado *aPos* de 3 componentes.

glPosition es una variable predefinida que podemos modificar para alterar la posición de los vértices. En este caso, la posición de los objetos no varía, ya que indicamos que sea igual que la de entrada.

Por otro lado, el código de un Fragment Shader podría ser el siguiente:

```
#version 330 core
out vec4 FragColor;

void main() {
    FragColor = vec4(1.0f, 0.5f, 0.2f, 1.0f);
}
```

En él, especificamos el color que deben poseer los píxeles, ya que sino no habría ningún color en un principio. Podría especificarse que se reciba un color de entrada proveniente del Vertex Shader como en los siguientes fragmentos de código de ambos Shaders:

```
#version 330 core
layout (location = 0) in vec3 aPos;
layout (location = 1) in vec3 aColor;
out vec3 ourColor;

void main() {
    gl_Position = vec4(aPos, 1.0);
    ourColor = aColor;
}
```

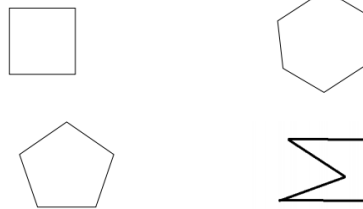
```
#version 330 core
in vec3 ourColor;

void main() {
    gl_FragColor = vec4 (ourColor,1.0f);
}
```

4. Ejercicios

▪ Sin Soluciones

1. Si se coloca un cubo de lado 1 centrado en el origen en el punto $(0,0,-3)$ ¿Es visible? ¿Y si se coloca en el $(0,0,3)$?
2. ¿Cuál es el número mínimo de triángulos necesarios para formar las siguientes figuras?

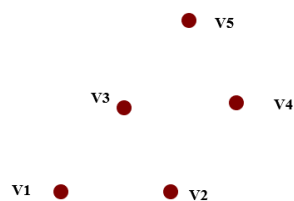


3. Dados los vértices: 1 $(0,0,-1)$ 2 $(1,0,-1)$ 3 $(1,1,-1)$ 4 $(0,1,-1)$
Y dadas las siguientes secuencias de dichos vértices, indica qué caras, resultantes de dichas secuencias, se verían y cuáles no:

- a) 2,3,1
- b) 3,2,1
- c) 1,4,3
- d) 1,2,4
- e) 4,3,2

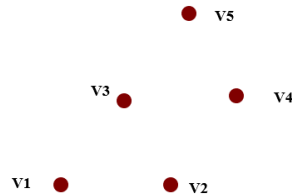
4. ¿Qué figuras generan las siguientes secuencias de vértices usando triángulos?

- a) 1, 2, 3, 4, 5
- b) 3, 2, 1, 3, 4, 5
- c) 2, 1, 3, 4, 3, 2, 3, 5, 4



5. ¿Qué figuras generan las siguientes secuencias de vértices usando tiras de triángulos?

- a) 1, 2, 3, 4, 5
- b) 2, 1, 3, 4, 5



6. ¿Cómo puedo generar la siguiente figura en OpenGL?



7. Rellena los fragmentos de código en blanco:

```
float vertices_esfera[] =
{-0.0247692f, 0.987385f, -0.156387f, 0.0250000f, 1.00000f, 0.00100000f, 1.00100f, -
0.00100000f,-0.0567020f, 0.931997f, -0.358003f, -5.96046e-008f, 0.900000f,0.000999987f,
0.952057f, -0.310017f,-0.118514f, 0.940755f, - 0.317702f, 0.0500000f, 0.900000f, -0.0944915f,
0.952057f, -0.294893f,.....}

void dibuja_esfera() {

    unsigned int VBO;
    glGenVertexArrays(1, &VAO_esfera);
    glGenBuffers(1, &VBO);
    glBindVertexArray(VAO_esfera);
    glBindBuffer(GL_ARRAY_BUFFER, VBO);
    glBufferData(GL_ARRAY_BUFFER,sizeof(vertices_esfera),vertices_esfera, GL_STATIC_DRAW);
    // Normales
    glVertexAttribPointer(____,____, GL_FLOAT, GL_FALSE, ____* sizeof(float), (void*)____);
    glEnableVertexAttribArray(1);

    // Texturas
    glVertexAttribPointer(____,____, GL_FLOAT, GL_FALSE, ____* sizeof(float), (void*)____);
    glEnableVertexAttribArray(2);

    // Vértices
    glVertexAttribPointer(____,____, GL_FLOAT, GL_FALSE, ____* sizeof(float), (void*)____);
    glEnableVertexAttribArray(0);

    glBindBuffer(GL_ARRAY_BUFFER, 0);
    glBindVertexArray(0);
    glDeleteBuffers(1, &VBO);
}
```



```

#version 330 core

layout (location = ____ ) in vec3 aPos;

layout (location = ____ ) in vec3 aNorm;

layout (location = ____ ) in vec2 aTexCoord;

out vec3 ourColor;
uniform mat4 transform;

void main() {
    gl_Position = transform * vec4(aPos, 1.0);
    ourColor = aNorm;
}

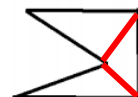
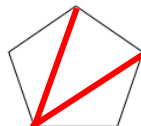
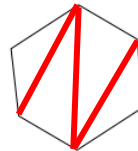
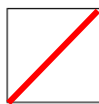
```

▪ Con Soluciones

1. Si se coloca un cubo de lado 1 centrado en el origen en el punto (0,0,-3) ¿Es visible?
¿Y si se coloca en el (0,0,3)?

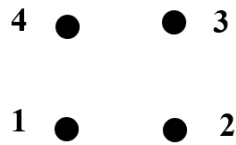
Un cubo con $z = -3$ se colocaría delante de la cámara, por lo que el objeto se vería.
Mientras que un cubo con $z = 3$, estaría detrás de la cámara y, por tanto, no se vería.

2. ¿Cuál es el número mínimo de triángulos necesarios para formar las siguientes figuras?

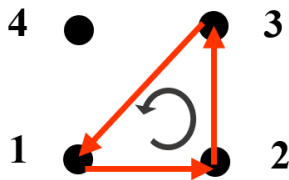


3. Dados los vértices: 1 (0,0,-1) 2 (1,0,-1) 3 (1,1,-1) 4 (0,1,-1)
Y dadas las siguientes secuencias de dichos vértices, indica qué caras, resultantes de dichas secuencias, se verían y cuáles no:

Los vértices representados sobre un plano, en función de sus coordenadas, se ven tal que así:

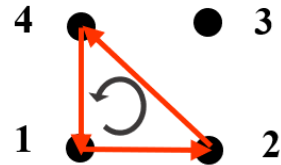


a) 2,3,1



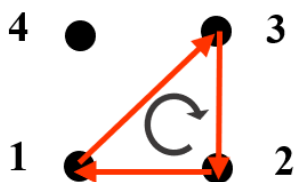
Se ve, el recorrido es antihorario

d) 1,2,4



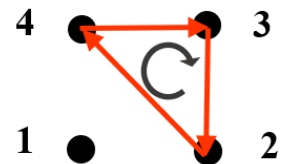
Se ve

b) 3,2,1



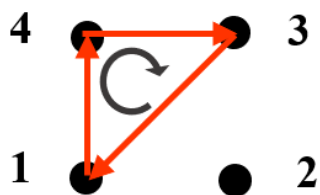
No se ve, el recorrido es horario

e) 4,3,2



Se ve

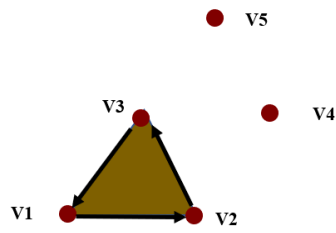
c) 1,4,3



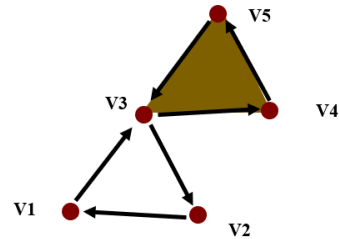
No se ve

4. ¿Qué figuras generan las siguientes secuencias de vértices usando triángulos?

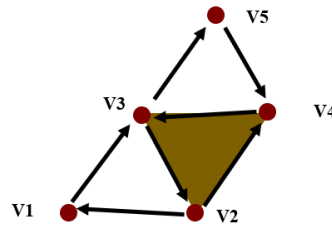
a) 1, 2, 3, 4, 5



b) 3, 2, 1, 3, 4, 5



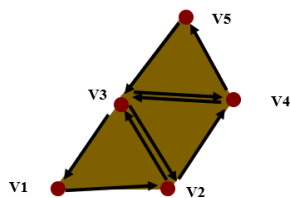
c) 2, 1, 3, 4, 3, 2, 3, 5, 4



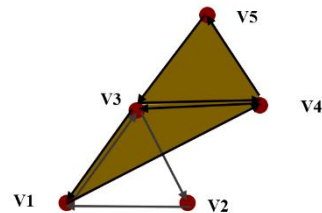
Las caras coloreadas son las únicas que se ven

5. ¿Qué figuras generan las siguientes secuencias de vértices usando tiras de triángulos?

a) 1, 2, 3, 4, 5



b) 2, 1, 3, 4, 5



6. ¿Cómo puedo generar la siguiente figura en OpenGL?



Se debe dibujar el objeto dos veces:

Primero se dibuja el objeto con `GL_FILL` (no se dibujan las aristas, pero sí el objeto) y posteriormente, se dibuja el objeto con `GL_LINES`.

7. Rellena los fragmentos de código en blanco:

```

float vertices_esfera[] =
{-0.0247692f, 0.987385f, -0.156387f, 0.0250000f, 1.00000f, 0.00100000f, 1.00100f, -
0.00100000f,-0.0567020f, 0.931997f, -0.358003f, -5.96046e-008f, 0.900000f,0.000999987f,
0.952057f, -0.310017f,-0.118514f, 0.940755f, - 0.317702f, 0.0500000f, 0.900000f, -0.0944915f,
0.952057f, -0.294893f,.....}

void dibuja_esfera() {

    unsigned int VBO;
    glGenVertexArrays(1, &VAO_esfera);
    glGenBuffers(1, &VBO);
    glBindVertexArray(VAO_esfera);
    glBindBuffer(GL_ARRAY_BUFFER, VBO);
    glBufferData(GL_ARRAY_BUFFER, sizeof(vertices_esfera), vertices_esfera, GL_STATIC_DRAW);
    // Normales
    glVertexAttribPointer(1, 3, GL_FLOAT, GL_FALSE, 8* sizeof(float), (void*)0);
    glEnableVertexAttribArray(1);

    // Texturas
    glVertexAttribPointer(2, 2, GL_FLOAT, GL_FALSE, 8* sizeof(float), (void*)3);
    glEnableVertexAttribArray(2);

    // Vértices
    glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 8* sizeof(float), (void*)5);
    glEnableVertexAttribArray(0);

    glBindBuffer(GL_ARRAY_BUFFER, 0);
    glBindVertexArray(0);
    glDeleteBuffers(1, &VBO);
}

```

```

#version 330 core

layout (location = 0) in vec3 aPos;

layout (location = 1) in vec3 aNorm;

layout (location = 2) in vec2 aTexCoord;

out vec3 ourColor;
uniform mat4 transform;

void main() {
    gl_Position = transform * vec4(aPos, 1.0);
    ourColor = aNorm;
}

```



Escola
Técnica
Superior
de Enxeñaría



Grado en Ingeniería Informática

2º curso – 2º cuatrimestre

Computación Gráfica

03-03-2021

Clase 04

Temas trabajados:

Tema 4: Transformaciones

Índice

1. Modelo de representación Transformaciones Geométricas.	4
GPU operaciones fijas.....	4
1.1 Transformaciones.....	4
1.2 Transformaciones Geométricas.....	4
1.2.1 TRASLACIÓN	5
TRASLACIÓN MATRICIAL	5
1.2.2 ROTACIÓN	5
ROTACIÓN GENERAL	6
ANGULOS DE EULER	6
1.2.3 Escalado.....	6
1.2.4 Otras transformaciones.....	7
1.2 Preguntas	7
1.3 Reflexión.....	8
1.4 Coordenadas homogéneas	8
1.5 Transformaciones 2D	8
1.6 Transformaciones 3D	9
1.7 Zonas de una matriz	9
1.7.2 Ejercicio	9
1.8 Matrices en OpenGL	10
1.9 Representación de la matriz OpenGL	10
1.10 OpenGL 1.2.....	10
1.11 Pila, varias Transformaciones, Identidad 1.2	10
1.12 Problema I	11
1.13 Movimientos concatenados-jerarquías	11
1.14 Problema II.....	12
1.15 Recordatorio, OpenGL	13
1.16 Cómo trabajamos en OpenGL	13
1.17 Lectura de una matriz.....	14
1.18 Transformaciones en OpenGL 3.3	14
¿Cómo lo controlamos?.....	14
¿Cómo calculamos la matriz?	15

1. Modelo de representación Transformaciones Geométricas.

Ejemplo de código.....	15
1.19 Prácticas	15

1. Modelo de representación Transformaciones Geométricas.

GPU operaciones fijas

Las transformaciones geométricas se realizan en la pipe fija.

Tenemos los vértices referidos a donde están ellos localizados y luego componemos la escena.

*En PIPE 3.3: El Vertex Shader coloca los vértices en la escena.

1.1 Transformaciones

En el pipe se definen varios espacios de trabajo, se llaman Local Space a las coordenadas a las que están referidos los vértices de los objetos.

A partir de ahí se compondrá la escena en el World Space.

Para pasar de Local Space a World Space hay que aplicarle a cada objeto una transformación mediante la multiplicación por una matriz de transformación.

DEFINICIONES

- Local Space: son coordenadas locales. Son de su objeto en relación con su origen local. Los vértices están referenciados a estas coordenadas.
- La matriz del modelo transforma las coordenadas locales en coordenadas de la escena (mundo).
- En estas coordenadas es donde aparecen representados todos los objetos el World Space.
- La matriz del modelo traslada, escala y rota los objetos hasta colocarlos en su posición. Las coordenadas de los vértices tendrán unos nuevos valores: que será el resultado de multiplicar cada uno de sus vértices por la matriz de transformación.

1.2 Transformaciones Geométricas

- Identidad: la matriz identidad, el objeto no se mueve.
- Translación: el objeto se traslada.
- Rotación: el objeto rota conforme al origen de coordenadas.
- Escalado:
 - Isotrópico: si en 'x', 'y' y 'z' se escala la misma cantidad

1.2.1 TRASLACIÓN

Tenemos un punto (x,y) y lo queremos trasladar a una coordenada (x', y'), para ello se sumará la cantidad necesaria:

- $x' = x + T_x$
- $y' = y + T_y$

Es decir, si queremos trasladar un objeto, se trasladará cada uno de sus vértices.

- **glTranslatef(T_x,T_y,T_z):** se le pasan tres parámetros float que es la cantidad que se quiere trasladar cada uno de los puntos.
- **glTransatev():** misma funcionalidad que la anterior, pero en lugar de float se le pasa números enteros.

TRASLACIÓN MATRICIAL

Se puede expresar como:

- $x' = x + T_x$
- $y' = y + T_y$

Dado lo anterior cogemos los vectores columna quedando lo siguiente:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} T_x \\ T_y \end{bmatrix} = \begin{bmatrix} x + T_x \\ y + T_y \end{bmatrix}$$

1.2.2 ROTACIÓN

En este caso, queremos girar un punto respecto al origen, cuando tenemos un objeto lo que hacemos es rotar todos los puntos.

- Llamamos rotación a desplazar un ángulo las coordenadas de un punto respecto al origen.
- Rotamos una línea u objeto rotando cada uno de sus vértices

- $x = r \cos(\alpha)$;
- $y = r \sin(\alpha)$;
- $x' = r \cos(\alpha + \theta)$;
- $y' = r \sin(\alpha + \theta)$;

$$\begin{aligned} x' &= x \cos(\alpha) - y \sin(\theta); \\ y' &= x \sin(\alpha) + y \cos(\theta); \end{aligned}$$

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix} \cdot \begin{bmatrix} x \\ y \end{bmatrix}$$

En las ecuaciones anteriores el ángulo alfa representa el grado del ángulo que forma cuando unimos el eje de coordenadas con el punto con el eje de las x. Por otro lado, el θ ángulo representa el ángulo que queremos rotar el punto.

La demostración de los pasos anteriores no entra en el examen.

- **glRotated((45.0, 0,1,1):** se le pasa el número de grados que se quiere girar y tres coordenadas que especifican el eje en el que se quiere girar.

ROTACIÓN GENERAL

- Fórmula general cuando el punto sobre el cual se rota no es el origen, sino un punto cualquiera (X_c, Y_c).

Cuando el eje sobre el que se quiere rotar no es el eje de coordenadas entonces se hará una composición de traslaciones.

Primero trasladamos el objeto al origen, luego lo giramos sobre el origen los grados que queremos que gire sobre sí mismo y finalmente trasladaremos de nuevo al punto inicial el objeto, de forma que para conocer las coordenadas del objeto se realizará la siguiente ecuación:

$$P' = P \cdot T(-x_c, -y_c) \cdot R(\theta) \cdot T(x_c, y_c)$$

Son transformaciones cuerpo rígido ya que no cambia su forma.

ANGULOS DE EULER

Nosotros vamos a trabajar con rotaciones en ángulos de Euler

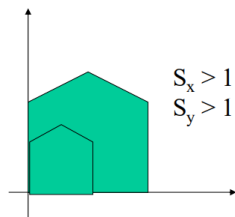
- Dados dos sistemas de coordenadas xyz y XYZ con un mismo origen, es posible determinar la posición de un sistema en términos del otro usando tres ángulos α , β y γ (yaw, pitch and roll). En nuestro caso serán los ejes de un objeto respecto a un sistema de referencia general.
- Los giros son acumulativos y los ángulos se encuentran ya girados.
- glRotatef (angle, vx, vy, vz); //ángulo, los ángulos de Euler (1,0,0) (0,1,0), (0,0,1), Pero puede ser cualquier otro. (1,1,1)

1.2.3 Escalado

- Llamamos escalado a cambiar el tamaño de un objeto. No es transformación cuerpo rígido ya que la geometría forma-volumen varía.
- Para escalar multiplicamos por los coeficientes de escalado que son la razón entre las coordenadas nuevas y las viejas. Escalado respecto al origen.
 - $S_x = \text{Anchura Nueva} / \text{Anchura vieja}$ $0 < S_x < 1$

1. Modelo de representación Transformaciones Geométricas.

- $S_y = \text{Altura Nueva} / \text{Altura vieja}$ $0 < S_y < 1$
- $S_x = S_y$ Escalado uniforme, $S_x \neq S_y$. Escalado no-uniforme
- Si queremos escalar un objeto escalamos cada vértice multiplicamos



$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} S_x & 0 \\ 0 & S_y \end{bmatrix} \cdot \begin{bmatrix} x \\ y \end{bmatrix}$$

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} S_x \cdot x \\ S_y \cdot y \end{bmatrix}$$

glScalef(Sx,Sy,Sz); //escalada en cada uno de los ejes, x, y,z.

Nota: Escalar un objeto será mover todos sus puntos a un tamaño que nosotros indiquemos.

El escalado presenta el mismo problema que la rotación, ya que si realizamos el escalado de forma directa el objeto se moverá.

Para realizar un escalado cuando no se encuentra en el origen se realizará la siguiente composición:

- Traslado al origen
- Escalado al doble
- Traslado al punto de partida

1.2.4 Otras transformaciones

Denominaremos otras transformaciones a aquellas que hacen uso de la matriz (2D). Reflexión y Shear.

*Se usan mucho con la cámara, pero nosotros no las vamos a ver.

1.2 Preguntas

¿Dónde es mejor situar los objetos en el local space?

Respuesta: en el origen, porque así tenemos el centro del objeto más localizado y será más sencillo alguna de las transformaciones como puede ser rotación o escalado.

¿Cuáles son las matrices de transformación en lenguaje natural para el siguiente movimiento?



Repuesta: tenemos que transformar nuestro objeto desde la posición inicial a la segunda, lo primero que se haría sería trasladarlo al origen con `glTranslatef(-2,-2,0)`, luego se gira 90 grados y por último se volverá trasladar con `gltranslate(0,2,0)`

1.3 Reflexión

- Cuando se realiza una traslación tenemos la primera ecuación.

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} T_x \\ T_y \end{bmatrix} = \begin{bmatrix} x + T_x \\ y + T_y \end{bmatrix}$$
- Para realizar una rotación tenemos la segunda.

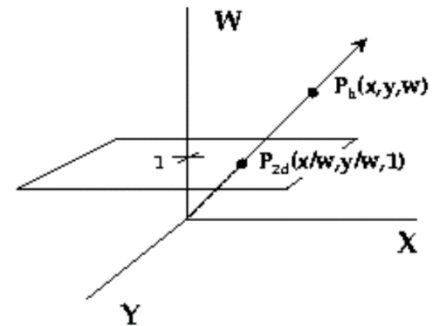
$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix} \cdot \begin{bmatrix} x \\ y \end{bmatrix}$$
- Para realizar un escalado tenemos la tercera

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} S_x & 0 \\ 0 & S_y \end{bmatrix} \cdot \begin{bmatrix} x \\ y \end{bmatrix}$$

1.4 Coordenadas homogéneas

Nos queda por resolver el problema de las traslaciones mediante una suma para poder utilizar solo multiplicaciones para lo cual introducimos unas nuevas coordenadas las homogéneas en estas coordenadas.

- Punto P (x, y, z, w)
- Vector P (x, y, z, 0) (es una dirección).
- Coordenadas homogéneas w = 1
- En general w = 1
- No existe (0,0,0), (0,0,0,1).
- Punto (x, y, z, 1), vector (x, y, z, 0)



1.5 Transformaciones 2D

$$\begin{aligned} \begin{bmatrix} x' \\ y' \end{bmatrix} &= \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} T_x \\ T_y \end{bmatrix} = \begin{bmatrix} x + T_x \\ y + T_y \end{bmatrix} \\ \begin{bmatrix} x' \\ y' \end{bmatrix} &= \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix} \cdot \begin{bmatrix} x \\ y \end{bmatrix} \\ \begin{bmatrix} x' \\ y' \end{bmatrix} &= \begin{bmatrix} S_x & 0 \\ 0 & S_y \end{bmatrix} \cdot \begin{bmatrix} x \\ y \end{bmatrix} \end{aligned} \quad \rightarrow \quad \begin{pmatrix} 1 & 0 & T_x \\ 0 & 1 & T_y \\ 0 & 0 & 1 \end{pmatrix}$$

$$\begin{pmatrix} \cos(\alpha) & -\sin(\alpha) & 0 \\ \sin(\alpha) & \cos(\alpha) & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

$$\begin{pmatrix} S_x & 0 & 0 \\ 0 & S_y & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

Con esto vemos que mediante una multiplicación de matrices y aplicando las coordenadas homogéneas obtenemos que la traslación de objetos puede realizarse mediante multiplicaciones de matrices, quitando la suma.

1.6 Transformaciones 3D

- Traslación

$$T(d_x, d_y, d_z) = \begin{bmatrix} 1 & 0 & 0 & d_x \\ 0 & 1 & 0 & d_y \\ 0 & 0 & 1 & d_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

- Escalado

$$S(s_x, s_y, s_z) = \begin{bmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Notación en columna M[16].

Por ejemplo los valores de traslación
estarán en M(13,14,15,16)

- Rotación alrededor del eje Z

$$R_z(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta & 0 & 0 \\ \sin \theta & \cos \theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

- Rotación alrededor del eje X

$$R_x(\theta) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \theta & -\sin \theta & 0 \\ 0 & \sin \theta & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

- Rotación alrededor del eje Y

$$R_y(\theta) = \begin{bmatrix} \cos \theta & 0 & \sin \theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin \theta & 0 & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

No hay que saberlo de memoria.

Cuando se quiera realizar cualquier tipo de operación se pondrá el vector en vertical al lado derecho de la matriz y se multiplicará.

1.7 Zonas de una matriz

Si nosotros aplicamos varias transformaciones siempre podemos saber dónde lo hemos trasladado porque aparece en N41, N42, N43. Pero no sabemos cada giro o escalado que hemos realizado.

1.7.2 Ejercicio

Vamos aoger el punto (0,0,1) y trasladarlo 1 unidad en X y rotarlo 90 grados en z.
 $\sin(\pi/2.0)=1$; $\cos(\pi/2.0)=0$;

$$\begin{pmatrix} 1 & 0 & T_x \\ 0 & 1 & T_y \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} \cos(\alpha) & -\sin(\alpha) & 0 \\ \sin(\alpha) & \cos(\alpha) & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

Rotación Translación

$$\begin{pmatrix} X' \\ Y' \\ 1 \end{pmatrix} = \begin{pmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix}$$

$$\begin{pmatrix} X' \\ Y' \\ 1 \end{pmatrix} = \begin{pmatrix} 0 & -1 & 1 \\ 0 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix}$$

Tenemos un vértice, lo queremos trasladar en el eje x y después girarlo. La primera es la matriz de traslación y la segunda la de rotación.

Si primero se traslada y después se gira no es lo mismo, quedará en distinta posición.

1.8 Matrices en OpenGL

`glMatrixMode(GLenum mode);`

En OpenGL existen tres matrices: la matriz de modelo (que se multiplica por los vértices), la de proyección (por la cámara) y la de texturas (se verá más adelante).

Las operaciones que se hagan sobre una matriz se harán sobre la matriz que en ese momento esté activo.

Cuando le quiero decir sobre qué matriz quiero trabajar es cargar la matriz identidad (inicializar la matriz).

1.9 Representación de la matriz OpenGL

La matriz es almacenada en un array de 16 elementos 4*4.

$$M = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 50 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Todo cero menos:

```
ScaleMatrix[0] = 1.0;
scaleMatrix[5] = 1.0;
scaleMatrix[10] = 1.0;
scaleMatrix[13] = 50.0;
scaleMatrix[15] = 1.0;
```

```
Float scaleMatrix[16]
/*Set matrix to zero*/
for(i = 0; i<16; i++){
    scaleMatrix[i] = 0;
}
scaleMatrix[0] = 1.0;
scaleMatrix[5] = 1.0;
scaleMatrix[10] = 1.0;
scaleMatrix[13] = 50.0;
scaleMatrix[15] = 1.0;

glMatrixMode(GL_MODELVIEW);
glLoadIdentity();
/*Load in the scale matrix*/
glLoadMatrixf(scaleMatrix);
```

DIBUJO;

Notación en columna M[16]. Por ejemplo los valores de traslación estarán en M(13,14,15,16)

27

1.10 OpenGL 1.2

Funciones de transformación:

- `glTranslatef(x,y,z)`
- `glRotatef(angle, x,y,z)`
- `glScalef(x,y,z)`

1.11 Pila, varias Transformaciones, Identidad 1.2

Las operaciones sobre traslación no tienen que ser esas tres, todas ellas se van acumulando en una cola LIFO.

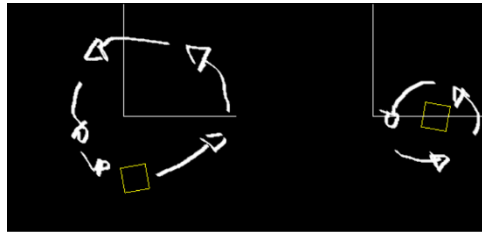
Las diferentes transformaciones mediante matrices se van “almacenando” multiplicando las transformaciones hasta obtener `glMatrixMode(GL_MODELVIEW)` que nos interese; Según van llegando se van almacenado en una pila y finalmente se multiplican. Por tanto, hay que usar de forma adecuada `glTranslate`, `glRotate` y `glScale`.

Problemas: la rotación y la traslación no se pueden poner de forma aleatoria, ya que el resultado cambiará según el orden en el que se apliquen.

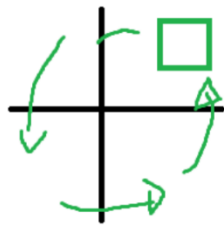
*La multiplicación de matrices no es conmutativa.

1.12 Problema I

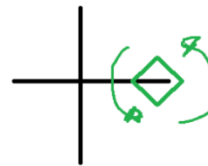
Supongamos un cubo centrado en el origen. Indica la secuencia que traslaciones para realizar estos giros.



SOLUCIÓN



```
glRotatef(angulo, Ex,Ey,Ez);  
glTranslatef(Tx, Ty, Tz);  
glScalef(Sx, Sy, Sz);
```



```
glTranslatef(Tx, Ty, Tz);  
glRotatef(angulo, Ex, Ey, Ez);  
glScalef(Sx, Sy, Sz);
```

1.13 Movimientos concatenados-jerarquías

Son movimientos que dependen o posiciones que depende unos de otros, y pueden ser heredados consecutivamente.

El sistema solar, los planetas giran alrededor del sol, y los satélites alrededor de un planeta.

Por ejemplo, para calcular la posición de la luna se sumará la posición del sol, la de la tierra y la suya propia con respecto a la tierra.

glPushMatrix(): hace una copia de la matriz activa.

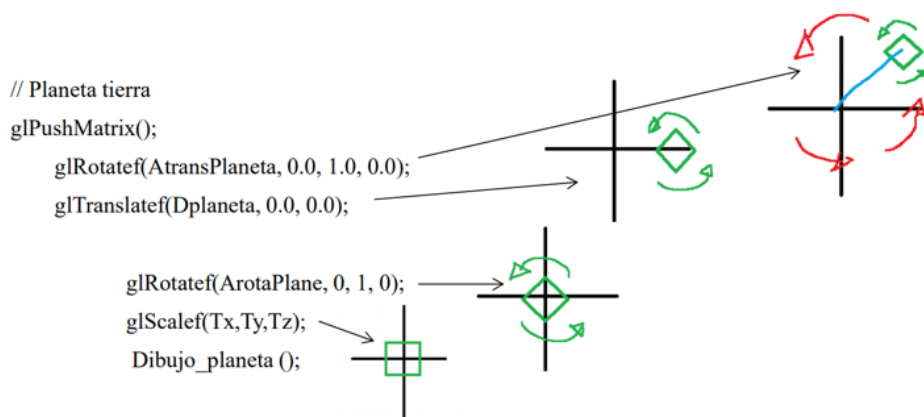
glPopMatrix(): recupera la matriz activa.

Si quiero dibujar el sistema solar, cogeré el sol y una vez calculada su posición se utilizará la primera función, después se calcula otra y se vuelve a utilizar esa misma, se realiza un pop para recuperarla.

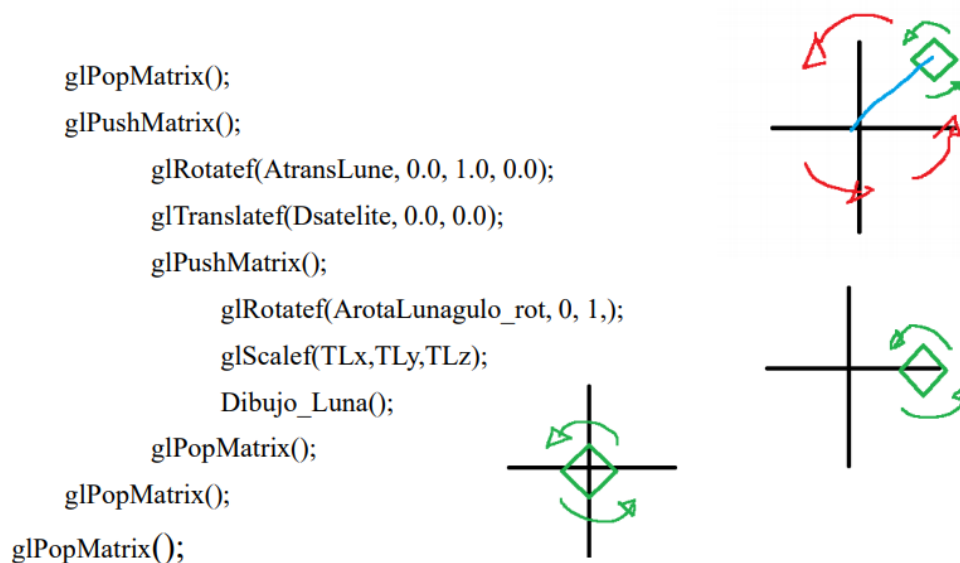
La función pop recupera la primera matriz anterior al push.

1.14 Problema II

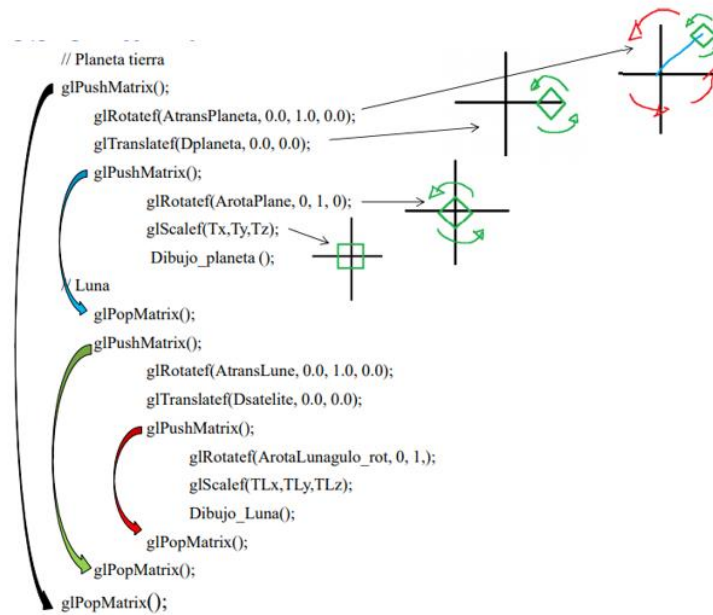
- ¿Y si el objeto girara sobre el origen y sobre sí mismo a diferentes velocidades? ¿código?
- ¿Y si además otro objeto girara alrededor de él? ¿Cuál sería el código?



- Dibujo de la luna



- Juntamos código



La luna no hereda la rotación sobre si misma de la tierra por eso, se realiza un pop al empezar la construcción de la matriz.

El siguiente push es para guardar la posición de la tierra en el espacio

1.15 Recordatorio, OpenGL

- OpenGL funciona en base a matrices:
 - o Son matrices 4*4, glMatrixMode(...)
 - GL_PROJECTION como los proyectados sobre la pantalla *(la veremos más adelante).
 - GL_MODELVIEW que se multiplica por el valor de los vértices y se combina con la anterior para general la escena.
 - o Matriz identidad: glLoadIdentity();
 - o OpenGL Utiliza un sistema de coordenadas homogéneo.
 - o Las matrices se aplican de derecha a izquierda
 - o Las matrices no son conmutativas.
 - o Con las funciones propias se almacenan en una FILO.
 - o Es posible copiar el estado de la matriz.

1.16 Cómo trabajamos en OpenGL

Hay dos caminos, matriz activa:

- Matriz activa
 - o Crear nuestras propias transformaciones.
 - o Calcular la transformación Matriz

- Meter la Matriz en un array 4*4.
- Leer la matriz (model_view)
- Cargarla la misma.
- Dibujar el objeto.
- Funciones Propias
 - Leer la matriz (model_view).
 - Usar funciones propias de OpenGL.
 - Dibujar el objeto.

1.17 Lectura de una matriz

No tiene mucha importancia, solo saber que tenemos dos funciones que nos permiten cargar la matriz (push) y para recuperarla (pop).

1.18 Transformaciones en OpenGL 3.3

Las transformaciones se van a realizar en el Vertex shaders.

Tipos:

- **Vertex shaders**
 - Función que recibe como parámetro vértices.
 - Manipula vértices en 3D; color, coordenadas de textura, orientación y tamaño de vértices.
 - Su propósito es transformar las coordenadas 3D de los vértices en otras coordenadas que ya veremos cuales son.
 - La salida debe ser la posición de los vértices.
- **Fragment shaders (OpenGL)**
 - Recibe la información de cada fragmento.
 - Un fragmento es toda la información necesaria para dibujar un píxel. – Se aplica en la última etapa de rasterizado.
 - Su propósito es obtener el color de cada píxel.
 - La salida es el color de cada píxel.

¿Cómo lo controlamos?

Tenemos que meter en el nuevo shader una nueva matriz 4x4 que multiplicada por el vértice devuelva la posición final del vértice en el espacio después de la transformación.

Esta matriz se calcula en el main, y se le mandará al shader, utilizando una variable uniform.

¿Cómo calculamos la matriz?

Para ello vamos a utilizar unas librerías (glm) que vienen en cabeceras, y a parte de muchas operaciones incluyen la traslación, la rotación y el escalado.

El orden de las transformaciones también es importante.

Ejemplo de código

```
glm::mat4 transform; // es la matriz de transformacion
transform = glm::mat4(); // carga identidad
transform = glm::translate(transform, glm::vec3(0.25f, 0.0f, 0.0f));
transform = glm::rotate(transform, angulo, glm::vec3(0.0f, 0.0f, 1.0f));
transform = glm::scale(transform, glm::vec3(.1, .1, .1));

unsigned int transformLoc = glGetUniformLocation(shaderProgram, "transform");
glUniformMatrix4fv(transformLoc, 1, GL_FALSE, glm::value_ptr(transform));

glBindVertexArray(VAOCuadrado); // Lo dibujo
glDrawElements(GL_TRIANGLES, 36, GL_UNSIGNED_INT, 0);

#version 330 core
layout (location = 0) in vec3 aPos;
uniform mat4 transform;
void main() {
    gl_Position = transform * vec4(aPos, 1.0f);
}
```

1.19 Prácticas

Practica 2: puede hacerse una en 3.3 y otra en 1.2 o ambas en 3.3.



Escola
Técnica
Superior
de Enxeñaría



Grado en Ingeniería Informática

2º curso – 2º cuatrimestre

Computación Gráfica

10-03-2021

Clase 5

Temas trabajados:

Proyecciones, modelo de cámara sintética

Índice

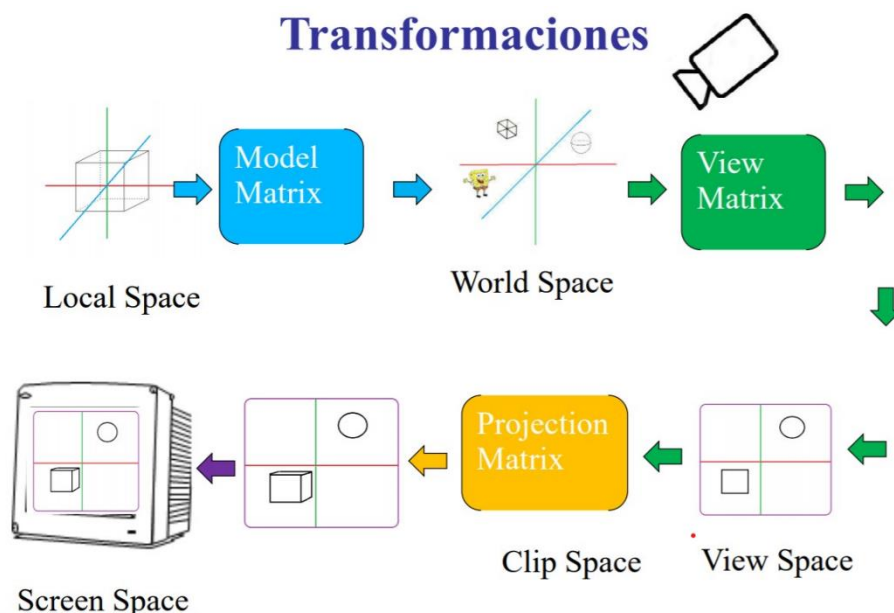
1. Modelo de la cámara Sintética	3
Visión 2D y 3D	3
2. Proyección	4
Tipos de proyecciones.....	5
Clasificación de las Proyecciones	5
3. Como generar las matrices de proyección en Opengl 1.2.....	6
4. Como generar las matrices de proyección en Opengl 3.3.....	8

1. Modelo de la cámara Sintética

Visión 2D y 3D

Dentro del pipe fija, la cámara se podría englobar dentro de las transformaciones geométricas, o después de ellas. Sin embargo, tiene más sentido situarlo dentro de las transformaciones geométricas, ya que, teniendo en cuenta que las transformaciones geométricas componen la escena, y esa composición de la escena la determinan la posición de los objetos en el espacio, cuando hay una escena compleja con varios objetos, dependiendo de la colocación de la cámara, el zoom y otros factores, también se va a modificar la posición de los vértices de los objetos, por lo que el proceso de colocación de la cámara y visualización.

Dentro del pipe móvil se hacen en el Vertex Shader.



Mediante la matriz de vista, o View Matrix, se transforma el World Space en el View Space.

El **View Space** se corresponde con el espacio de visualización de la cámara, resultado de transformar las coordenadas de la escena World Space en la de usuario/cámara. Esto se consigue colocándola en un punto en el espacio, determinando hacia donde mira mediante una combinación de traslaciones y rotaciones. Estas transformaciones combinadas es lo que se almacena dentro de la matriz de proyección, que transforma las coordenadas de la escena en las de la cámara.

Por otro lado, se tiene la matriz de proyección, o Projection Matrix, cuya funcionalidad es determinar cómo se enfoca la escena, que se realiza mediante proyecciones (ortogonal o perspectiva). De esta manera se obtiene el Clip Space.

El **Clip Space** es todo el espacio obtenido de la matriz de proyección, donde se sitúan los objetos en sus posiciones correspondientes tras realizar la transformación. Sin embargo, se debe definir el frustum, que se corresponde a la parte del Clip Space que se ve y en la cual estarán aquellos objetos visibles. Aquellos objetos que se encuentren fuera del frustum se eliminarán mediante un proceso llamado “clipping”. Generalmente, el frustum se corresponde con ese cubo en el cual se pueden ver los objetos, aunque más adelante se comprobará que suele ser una pirámide.

Una vez determinado el frustum se mapean las coordenadas 3D a un espacio 2D similar a sacar una foto. Estas coordenadas normalizadas resultantes se asignarán a las coordenadas de la pantalla

Pregunta.

Si un objeto queda dentro y fuera del frustum que pasa con el.

- a) Lo veo.
- b) No lo veo.
- c) Solo veo la mitad que se encuentra dentro.

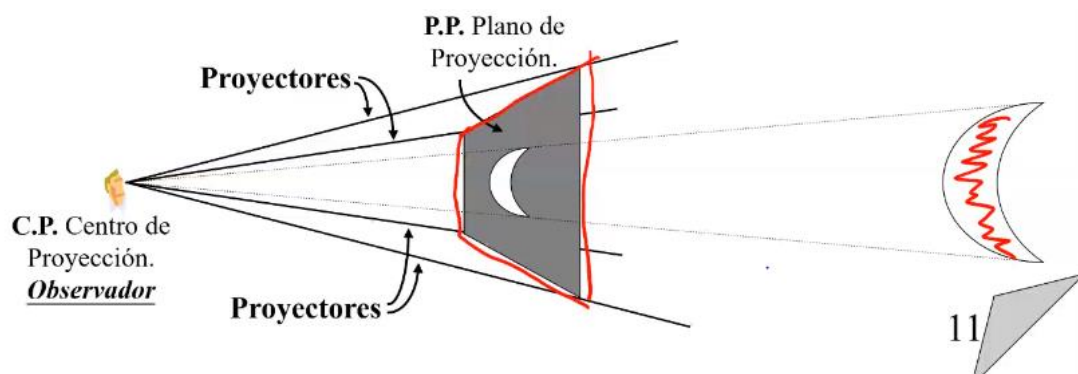
Depende. Si el objeto sale por un lado, seremos capaces de observar la mitad de este que pertenezca al frustum. Sin embargo, si por donde sale el objeto es por la región del frustum más próxima a la cámara, acercándose mucho al objeto, no se verá nada, ya que la cámara se situará dentro del objeto y la dirección de las normales no nos permitirían verlo.

2. Proyección

Una proyección es una transformación de un espacio de n dimensiones en otro de menor dimensión, $3 \rightarrow 2$. Consta de una serie de elementos:

- Los objetos a procesar (escena)
- El plano de proyección, que es donde se representan los objetos.
- El punto de vista o centro de proyección, que es donde se sitúa la cámara.

La proyección se realiza trazando unos rayos, que se denominan proyectores, desde los laterales del objeto hasta el centro de proyección. Estos proyectores intersecan el plano de proyección y conforman la imagen.



Tipos de proyecciones

Paralela (ortográfica): El centro de proyección está en el infinito, por lo que los proyectores son paralelos. En este caso, se define una dirección de proyección (D.P.)

Perspectiva: Existe un centro de proyección (C.P.) en el espacio, donde intersecan todos los proyectores.

Pregunta

Nuestra visión es:

- a) Ortográfica
- b) Perspectiva

Las fotos de nuestro teléfono son:

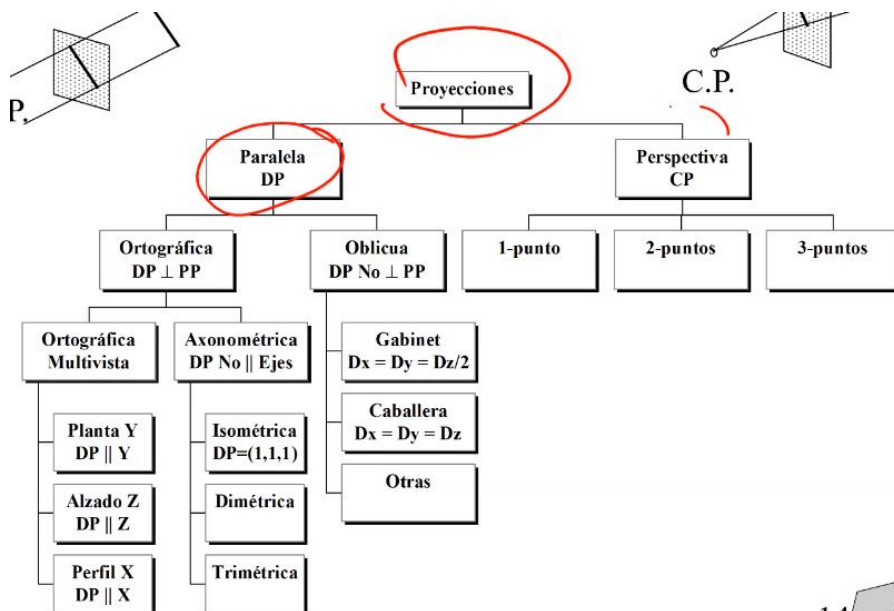
- a) Ortográfica.
- b) Perspectiva.

Los planos realizan proyecciones:

- a) Ortográfica.
- b) Perspectiva.

- 1: Nuestra visión es perspectiva, ya que existe un centro de proyección, en nuestros ojos
- 2: La visión de un teléfono es perspectiva, al ser el centro de proyección la lente de la cámara
- 3: Los planos realizan proyecciones ortográficas, ya que al ser planos el centro de proyección está en el infinito.

Clasificación de las Proyecciones



Dentro de las proyecciones paralelas, podemos diferenciar si los proyectores cortan perpendicularmente al plano de proyección (Ortográficas), o lo hacen con un determinado ángulo (Oblicuas).

Las proyecciones Ortográficas Multivistas permiten realizar vistas de planta alzado y perfil de los objetos, dependiendo de si la dirección de proyección (D.P.) es paralela al eje y, z o

x, respectivamente. Sin embargo, si la dirección de proyección no es paralela a ninguno de los ejes, forma proyecciones Axonométricas de tipo Isométricas, Dimétricas o Trimétricas, dependiendo si forma ángulos iguales con los tres ejes, con dos o con uno.

Por último, en las proyecciones en perspectiva, se pueden clasificar dependiendo de si tienen un punto de fuga, dos o tres.

El plano de proyección es paralelo a los ejes x, y, z, dentro de las ortográficas multivista.

Ortografía Multivista

Permite realizar las vistas de planta, alzado y perfil. Presenta las ventajas de que es posible realizar medidas y comparaciones precisas y que además todas las vistas tienen la misma escala. Sin embargo, no es posible saber la distancia a la que se encuentran los objetos entre sí y no se obtiene una visión en 3D.

Perspectiva

La proyección perspectiva no mantiene la escala ni la forma a diferencia de la ortografía multivista, pero proporcionan realismo virtual y sensación de tridimensionalidad.

3. Como generar las matrices de proyección en OpenGL 1.2

En OpenGL 1.2 únicamente se tiene una matriz, resultado de la multiplicación de la matriz de vistas (View Matrix) y la matriz de proyección (Projection Matrix). Esta matriz de proyección se llama `GL_PROJECTION`, y se llama mediante la función `glMatrixMode(GL_PROJECTION)`.

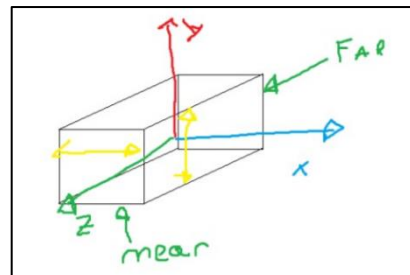
Una vez está cargada la matriz de proyección, se necesita colocar la cámara. Para ello se llama a la función `gluLookAt`, la cual recibe 9 parámetros:

`gluLookAt(eyeX, eyeY, eyeZ, atX, atY, atZ, upX, upY, upZ)`

- eyeX, eyeY, eyeZ: Especifican la posición de la cámara
- atX, atY, atZ: Especifican hacia dónde mira la cámara
- upX, upY, upZ: Especifican la orientación de la cámara

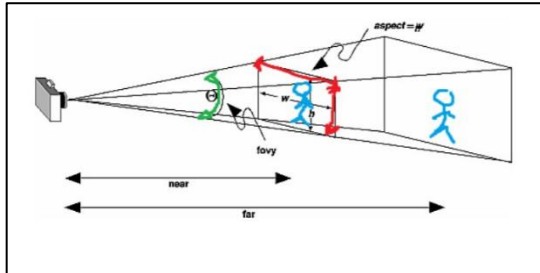
Por defecto, la cámara está en el punto 0,0,0, dirección 0,0,-1, y orientación 0,1,0.

Para la **proyección ortográfica** se usa la función `glOrtho3D`, que recibe como parámetros izquierda, derecha, arriba, abajo, cerca y lejos. Estos parámetros delimitan un cubo, que es el frustum, por lo que si un objeto se encuentra dentro de este cubo lo veremos, y si está por fuera no. Por defecto, como la cámara está situada en el 0,0,0, se trata de un cubo de una unidad hacia dentro.



Para la **proyección perspectiva** se usa la función gluPerspective, a la que se le pasan 4 parámetros: gluPerspective(fovy, aspect, near, far)

- Fovy se trata del ángulo con el que se abre el foco, normalmente 45 o 60 grados.
- Aspect es el tamaño de la ventana, que suele ser 4:3, 16:9... Tratándose del alto partido por el ancho
- Near y Far determinan desde donde ve la cámara, hasta donde ve la cámara



En este tipo de proyecciones es donde se aprecia que el frustum toma forma piramidal en vez de la de un cubo.

Aquí se representa un ejemplo de cómo se configurarían, representando las proyecciones, ortogonal o en perspectiva, dependiendo de la función que se use, y finalmente haciendo el gluLookAt para establecer la cámara.

Ejemplo

```
void Camara() {  
    //Configuración de la matriz de proyeccion  
    glMatrixMode(GL_PROJECTION);  
    glLoadIdentity(); //La ponemos a uno  
    glOrtho(-1000.0,1000.0f,-1000.0,1000.0f,1.0,1500.0f);  
  
    void gluPerspective(45,4.0/3.0, 0, 1500);  
  
    gluLookAt(0,0,0,0,0,100, 0,1,0);  
}
```

Pregunta.

Imaginemos un shooter en un laberinto o habitaciones. El programador ha puesto un *near* de 1m pero yo me puedo acercar a las paredes .5m. ¿Que pasará al acercarme a la pared?

- a) Se verá todo negro.
- b) Se verá la habitación de al lado.
- c) Chocare y me quedará atascado.

Se verá la habitación del otro lado, ya que al si nos acercamos a 0.5m no se verá la pared ya que estaría fuera del frustum, al ser near = 1m, por lo que seremos capaces de ver la habitación de al lado.

Pregunta.

Si quiero dividir mi ventana en 4 secciones para visualizar planta alzado y perfil. ¿Cómo hago?

- a) Genero 4 ventanas.
- b) Genero 4 viewports.
- c) Muevo la cámara y elijo la proyección.
- d) Otros.

Se puede hacer de varias formas, pero la más correcta es generar un viewport para cada una.

Pregunta.

Como hago para ver la planta alzado y perfil de un objeto.

- a) Genero un viewport.
- b) Giro el objeto para verlo desde diferentes puntos de vista.
- c) Giro la cámara para ver diferentes puntos de vista.

Se puede hacer de dos formas, o girar el objeto, o la cámara. Aunque se suele girar más la cámara para evitar movimientos raros en los objetos.

4. Como generar las matrices de proyección en OpenGL 3.3

La principal ventaja a la hora de generar matrices en la versión 3.3 es que se tiene mayor control.

A diferencia que en OpenGL 1.2, en la versión 3.3 se tienen 2 matrices: por un lado, la matriz de vista y por otro la matriz de proyección. Para calcular la posición de los vértices, se multiplican las matrices de proyección, de vista, de modelo y los vértices en la posición en la que estaban.

$$Vclipposition = Mprojection * Mview * Mmodel * Vertices$$

Pregunta

Debo hacer siempre $Vclipposition = Mprojection * Mview * \dots$ o puedo hacer $Vclipposition = Mview * Mprojection * \dots$

- a) Si es lo mismo.
- b) No el orden es importante.
- c) Depende de como las envíe al shader.

El orden es importante, ya que la multiplicación de matrices no es conmutativa.

Como es el shader

$$Vclip = Mprojection * Mview * Mmodel * Vertices$$

```
glm::mat4 model; // Se calcula
glm::mat4 view; // Se Calcula
glm::mat4 projection; // Se calcula

unsigned int modelLoc = glGetUniformLocation(IDShader, "model");
glUniformMatrix4fv(modelLoc, 1, GL_FALSE, glm::value_ptr(model));

unsigned int viewLoc = glGetUniformLocation(IDShader, "view");
glUniformMatrix4fv(viewLoc, 1, GL_FALSE, glm::value_ptr(view));

unsigned int projectionLoc = glGetUniformLocation(IDShader, "projection");
glUniformMatrix4fv(projectionLoc, 1, GL_FALSE, glm::value_ptr(projection));

#version 330 core
layout (location = 0) in vec3 aPos;

uniform mat4 model;
uniform mat4 view;
uniform mat4 projection;

void main() {
    gl_Position = projection * view * model * vec4(aPos, 1.0);
}
```

En el shader, se buscan las tres matrices que se le pasan, que en la figura anterior serían model, view y projection, y se multiplican para obtener la posición de los vértices. Para cargar valores en las matrices de proyección de vista se usan las funciones de antes, lookAt y ortho respectivamente.

Las matrices de proyección y de vista se pueden calcular a mano.

En el caso de la matriz de proyección ortográfica: **GLOrtho(left, right, Top, down, near, far)**

R = right L = left T = top B = botton N = near F = far

$$\begin{bmatrix} \frac{2}{R - L} & 0 & 0 & -\frac{(R + L)}{R - L} \\ 0 & \frac{2}{T - B} & 0 & -\frac{(T + B)}{T - B} \\ 0 & 0 & \frac{2}{F - N} & -\frac{(F + N)}{F - N} \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Para la matriz de proyección perspectiva: **gluPerspective (fovy, aspect, near, far)**

$$\begin{bmatrix} 1/\text{Aspt} \cdot \tan(\text{fovy}/2) & 0 & 0 & 0 \\ 0 & 1/\tan(\text{fovy}/2) & 0 & 0 \\ 0 & 0 & N+F/N-F & 2 \cdot F \cdot N / (N-F) \\ 0 & 0 & -1 & 0 \end{bmatrix}$$

Ejercicio:

- Supongamos **gluPerspective (90, 1, 1, 11);**
- Calcula la matriz de proyección.
- Calcula el punto de proyección de (0,0,-1), (0,11,-11), (0,4,-6); Razona la respuesta

$$\begin{bmatrix} 1/\text{Aspt} \cdot \tan(\text{fovy}/2) & 0 & 0 & 0 \\ 0 & 1/\tan(\text{fovy}/2) & 0 & 0 \\ 0 & 0 & N+F/N-F & 2 \cdot F \cdot N / (N-F) \\ 0 & 0 & -1 & 0 \end{bmatrix}$$

Punto	P. Coordenadas Homogéneas	Resultado	Resultado Normalizado
(0,0,-1),			
(0,11,-11),			
(0,4,-6);			

Tenemos que calcular la matriz de proyección a partir de estos valores.

Las coordenadas homogéneas son las mismas, añadiendo un 1 en omega.

Posteriormente se multiplica la matriz de proyección por el vértice, obteniendo los valores de la columna Resultado.

Para obtener el resultado normalizado, se dividen los valores del punto obtenido entre el valor de omega, obteniendo así las coordenadas normalizadas.

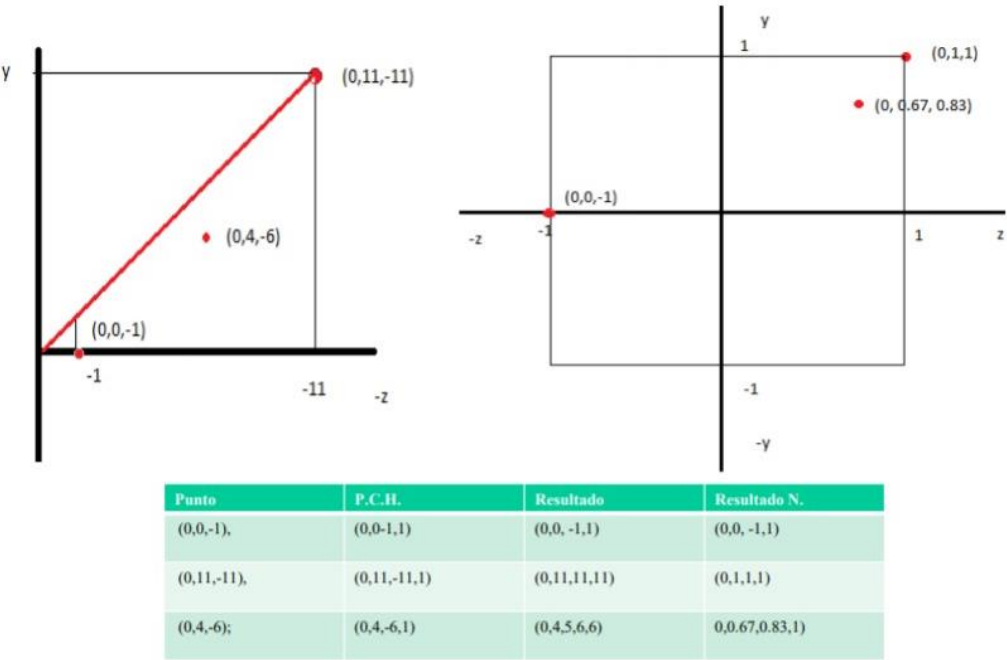
Resultados:

Matriz Proyección perspectiva.

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -1.2 & -2.2 \\ 0 & 0 & -1 & 0 \end{bmatrix}$$

Punto	P. C. Homogéneas	Resultado	Resultado Normalizado
(0,0,-1),	(0,0,-1,1)	(0,0, -1,1)	(0,0, -1,1)
(0,11,-11),	(0,11,-11,1)	(0,11,11,11)	(0,1,1,1)
(0,4,-6);	(0,4,-6,1)	(0,4,5,6)	0,0.67,0.83,1)

A continuación, se representan los puntos una vez normalizados, tal y como se obtendría al calcular la escena.





Escola
Técnica
Superior
de Enxeñaría



Grado en Ingeniería Informática

2º curso – 2º cuatrimestre

Computación Gráfica

21-04-2021



Clase 06

Temas trabajados: Tema 5. Iluminación.

Índice

1. Introducción	3
2. ¿Qué es la luz?	3
3. Caracterización humana de la luz.....	3
4. Teoría del color. Los colores aditivos	4
5. Interpolación del color	4
6. Modelo de iluminación.....	5
7. Modelo local de iluminación.....	5
8. Modelo de iluminación simple	5
9. Modelo de iluminación de Phong. (1973).....	6
10. La luz ambiental (modelo empírico).	6
11. La luz difusa (Lambert).....	6
12. La luz especular (blinn-phong)	7
13. Modelo de iluminación simple	7
14. Luces en Opengl 1.2.....	9
15. Normales/Caras OpenGL 1.2	14
16. Modelo Gouraud	14
17. PHONG shading.....	16
18. Luces en OpenGL	16
19. Materiales en OpenGL:	17

1. Introducción

Vamos a ver como coloreamos los vértices. Hay que tener en cuenta el concepto de fragmento: trozo de información que contiene todo lo necesario para proyectarse en la pantalla. ¿Cómo obtenemos los fragmentos? Una vez realizado el ensamblado de los vertices de los objetos que vamos a proyectar se pasa al proceso de **rasterizado**, que es cuando se transforman los objetos a las imágenes que vemos en pantalla. En este proceso se van trazando una serie de líneas para pasar de las matrices a pixeles, obteniendo la correspondencia de los puntos de los objetos con los puntos de la pantalla. Esa información compone un fragmento. Al realizar la proyección, lo hacemos a la inversa: desde la pantalla miramos a que pixel correspondería cada vértice del objeto, realizando la interpolación. A partir de cada punto de los objetos calculamos su valor y esta información se almacena en el fragmento. Dependiendo de la profundidad de los objetos podemos no alcanzar ningún punto en la rasterización, por lo que en ese fragmento se almacenaría el color del buffer de profundidad (clear color). Este trabajo lo puede realizar la tarjeta gráfica o se puede hacer por software siguiendo el algoritmo de ray tracing (el profe lo llama ray matching pero en internet sale como ray tracing), que es bastante efectivo. Los primeros fragmentos que se calculan son los de los vertices de los objetos y después se interpolan paraa tener la información completa de las caras.

Con operaciones fijas (1.2) no tenemos control o tenemos control limitado basado en ciertas funciones de alto nivel.

En 3.3 tenemos el fragment shader en el que podemos indicar de qué color es cada fragmento: podemos coger los fragmentos y trabajar sobre ellos, pudiendo colorearlos del color del vertice o del color que queramos. Gracias al fragment shader podemos trabajar a muy bajo nivel sobre esos fragmentos, pero requiere mucho más cuidado por parte del programador. Sin embargo, la versión 3.3 nos da mucha más flexibilidad que 1.2.

2. ¿Qué es la luz?

La luz es una onda electromagnética que viaja por el espacio, es percibida por el ser humano y cuya frecuencia o suma de frecuencias determina su color. El conjunto de las frecuencias o longitudes de onda se llama espectro.

3. Caracterización humana de la luz

Dos tipos fundamentales de detectores, que son los bastones y los conos.

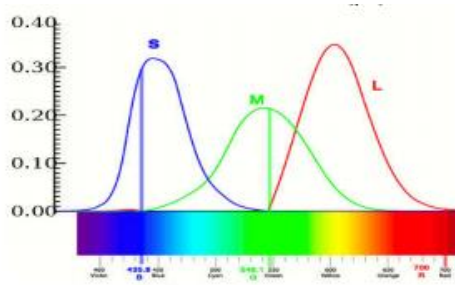
– Los bastones se utilizan en la visión nocturna y responden básicamente a la luminosidad, por lo que no afecta a la distinción entre los diferentes colores.

– Tres tipos de conos que responden con diferente sensibilidad a las bandas de radiación, se denominan detectores de rojo, verde y azul.

•El espectro, en función de la longitud de onda, de la intensidad luminosa será:

$$I(\lambda) = R \cdot r(\lambda) + V \cdot v(\lambda) + A \cdot a(\lambda)$$

Por esta razón, en la imágenes de síntesis se caracteriza un color por medio de sus componentes R, G, B (Red, Green, Blue). Sin embargo, existen modelos más precisos que realizan una discretización mucho más fina del espectro luminoso.



En el eje Y de la gráfica tenemos la intensidad luz y en el x, la longitud onda. Percibimos más intensidades del azul y del rojo. Por esto, cuando representamos los 24 bits de color no se hace 8 bits para cada color, sino que el verde suele llevar menos bits, normalmente 10 para el rojo y el azul y 4 para el verde.

4. Teoría del color. Los colores aditivos

– Los colores obtenidos directamente, por descomposición de la luz solar o artificialmente mediante focos emisores de luz de una longitud de onda determinada se denominan colores aditivos.

– No es necesaria la unión de todas las longitudes del espectro visible para obtener el blanco, ya que si mezclamos solo rojo, verde y azul obtendremos el mismo resultado. Por ello el 0,0,0 es negro y 1,1,1 blanco.

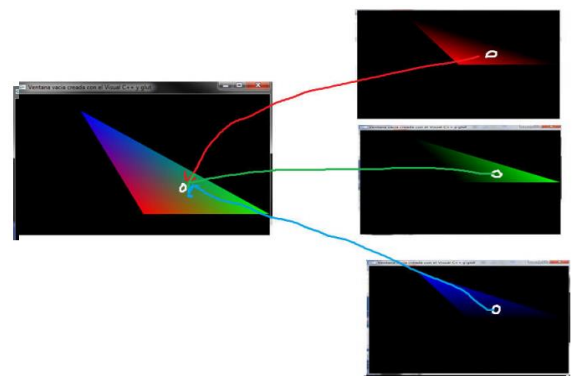
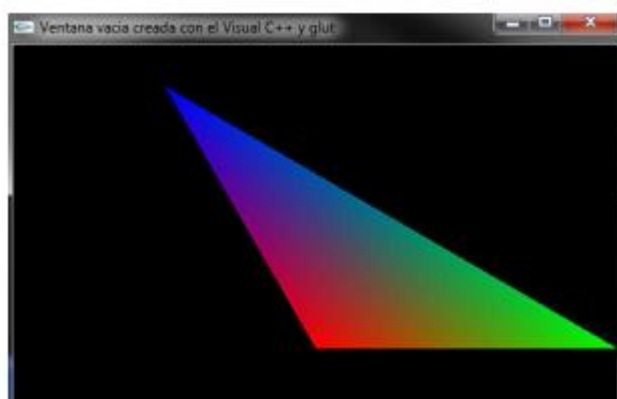
– El rojo, verde y azul son denominados colores primarios, porque la suma de los tres produce el blanco y todos los colores del espectro pueden ser obtenidos a partir de ellos.

• Para OpenGL usamos RGB con valores de 0-1 o de 0-255: `glColor3f(R,G,B)`

5. Interpolación del color

En OpenGL dándole un color antes de cada vértice dibujamos cada vértice de un color, interpolando el color a lo largo de los vértices.

Para obtener la siguiente imagen dibuja 3 triángulos y la intensidad final es la suma de los tres. En esto entran elementos como el rasterizado, la teoría aditiva del color y la percepción humana del color. Esto es el coloreado básico de los objetos.



6. Modelo de iluminación.

A partir del coloreado básico se iluminan los objetos. OpenGL tiene una luz por defecto que se corresponde con alguno de los modelos que veremos más adelante.

Hay dos tipos principales de modelos de iluminación.:

- o Locales: se utilizan para la visualización estándar y se caracterizan por su velocidad. Destaca el **modelo de Phong**. Son con los que vamos a trabajar. Se suelen usar en videojuegos.
- o Globales: consideran la escena en un conjunto, suelen ser lentos, aunque sus resultados son mucho mejores. Los más conocidos son los de radiosidad y trazado de rayos. Unreal y Crytek (videojuegos) ya implementan estos modelos de iluminación en tiempo real, pero se suelen utilizar para cine para que exista coherencia en la integración de las luces y sombras del cine live-action y de animación.

7. Modelo local de iluminación.

Utiliza un modelo de iluminación local en el cual los cálculos se realizan en función de:

- Fuentes de luz
 - Posición: Localizada o infinitamente alejada.
 - Intensidad y Color de la fuente.
 - Número: Número de luces.
 - Distribución lumínica: uniforme, focalizada, direccional, etc.
 - Geometría: puntual, esférica, lineal, etc.
- Objetos:
 - Distancias: al observador y a la fuente de luz
 - Material/Propiedades ópticas: transparencia, refracción, etc.
 - Cromaticidad: Color superficial propio.
 - Interacción: lumínica con otros objetos de la escena.
- Observador:
 - Dirección de observación: cálculo de la intensidad.

La luz viene de una o más fuentes de luz e interacciona con los objetos de la escena.

8. Modelo de iluminación simple

Sintetizando lo anteriormente mencionado, se puede decir que la iluminación viene dada por la siguiente función:

$$I = f(p, pv, \{MGMO\}, \{MGCL\})$$

I: Intensidad luminosa observada en el punto p

P: Punto de cálculo de la iluminación

Pv: Posición del punto de vista del observador

{MGMO}: Modelo geométrico - material de los objetos

{MGCL}: Modelo geométrico - cromático de la luz

En un principio, se puede simplificar usando fuentes puntuales (luces son redondas y pequeñas, objetos opacos y sin existencia de inter-reflexiones: la luz que llega a un objeto no se refleja sobre otro, atraviesa los objetos y los objetos no generan sombras sobre otros.

9. Modelo de iluminación de Phong. (1973)

Es un modelo de empírico de iluminación local muy usado a día de hoy en videojuegos. Se caracteriza por:

- Fuentes de luz puntuales.
- Los objetos (no fuentes) no emiten su propia luz.
- La luz esta formada por:
 - Luz ambiental: Luz no direccional, no identificable. Su efecto solo depende de su color y del objeto. Su variación implica cambios uniformes en la iluminación de los objetos. No genera sombras ni matices de color. Por ejemplo, un tubo fluorescente.
 - Luz Lamber o difusa: Está asociada a un foco de luz. Su variación está asociada a cambios de intensidad o posición del foco. Genera sombras y matices de color en función del ángulo con el que incide la luz en la superficie. Por ejemplo, una lámpara.
 - Luz especular o de Phong: Esta asociada a un foco intenso de luz. Se aprecia en objetos con brillo, el caso extremo es un espejo. Su variación está asociada a cambios de intensidad, posición del foco y punto de vista.
 - Luz Emisiva: la que da apariencia de que objeto emite luz. Poco usada.

10. La luz ambiental (modelo empírico).

La luz ambiental es aquella que no proviene de una dirección concreta, incide sobre todas las partes del objeto por igual. Su intensidad en un punto viene dada por:

$$I = I_a * K_a$$

Donde:

- K_a (constante) coeficiente de reflexión ambiental [0,1] dependiente de cada objeto. Si es 1, la intensidad de la luz en el objeto será igual a la intensidad de la luz que llega.
 - I_a intensidad ambiental en todo punto del espacio.
- Muestra los objetos como siluetas ya que esta k_a es una constante en todas las caras, por lo que todas las caras se ven igual.

11. La luz difusa (Lambert)

Depende de la normal y el ángulo de incidencia.

- La luz difusa es la que proviene de una dirección particular y es reflejada en todas direcciones según la ley de Lambert. Afecta a aquellas partes del objeto en las que la luz incide.
- La luz difusa es reflejada por la superficie de un objeto de manera direccional.
- Se basa en la ley de Lambert: “la componente difusa de la luz reflejada por una superficie es proporcional al coseno del ángulo de incidencia con la normal a la superficie”

$$I = I_L * K_D * \cos(\theta) = I_L * K_D * (\vec{N} \cdot \vec{L})$$

I_L :Intensidad de la fuente

θ :Ángulo de incidencia $[0^\circ, 90^\circ]$

K_D :Coeficiente de reflexión para la luz difusa $[0,1]$

$\vec{N}$: Normal a la superficie (unitario)

$\vec{L}$:Vector de iluminación (unitario)

12. La luz especular (blinn-phong)

La luz especular procede de una dirección concreta y depende de la posición del observador y las propiedades del objeto.

- Esta asociada a objetos brillantes o pulidos.
- Las reflexiones especulares suelen ser del mismo color que la fuente de luz.
- Caso ideal: Espejo, la luz se refleja exactamente en el ángulo incidente.
 - Depende del ángulo de la luz incidente.
 - En superficies imperfectas la luz se refleja dentro de un patrón tridimensional.
- En superficies perfectas rayo incidente y reflejado coplanares.
- En superficies imperfectas patrón de reflexión. Hay luz fuera del plano y el ángulo ideal de reflexión.

$$I = I_L * K_S * \cos^n(\alpha) = I_L * K_S * (\vec{R} \cdot \vec{V})^n$$

Si n muy grande, especularidad muy concentrada. $\vec{R} \cdot \vec{V}$ entre -1 y 1 , al estar elevado a n se reduce.

13. Modelo de iluminación simple

¿Como seria el modelo de iluminación de Phong? Correspondería a una ecuación del siguiente estilo:

$$I = I_a * K_a + I_L * (K_S * (\vec{R} \cdot \vec{V})^n + K_d * (\vec{N} \cdot \vec{L}))$$

A parte de esto vamos a añadirle unos cuantos efectos como son la atenuación, múltiples fuentes y color.

Hasta ahora se había hablado de la intensidad de la luz, pero, cuando hablamos de la intensidad de la luz tenemos 3 colores, entonces, para calcular la intensidad de la luz que tenemos en un punto, el color real de la luz será la componente de la luz roja, más la luz verde más la luz azul.

- Luz roja: $I_R = I_{aR} * K_a * C_{dR} + I_{LR} * f_{at}(K_s * C_{eR} * (\vec{R} \cdot \vec{V})^n + K_d * C_{dR} * (\vec{N} \cdot \vec{L}))$
- Luz verde: $I_G = I_{aG} * K_a * C_{dG} + I_{LG} * f_{at}(K_s * C_{eG} * (\vec{R} \cdot \vec{V})^n + K_d * C_{dG} * (\vec{N} \cdot \vec{L}))$
- Luz azul: $I_B = I_{aB} * K_a * C_{dB} + I_{LB} * f_{at}(K_s * C_{eB} * (\vec{R} \cdot \vec{V})^n + K_d * C_{dB} * (\vec{N} \cdot \vec{L}))$

Con lo cual automáticamente, la complejidad de las ecuaciones se ha multiplicado por 3:

$$R = R_R + R_G + R_B$$

Calcular esto en tiempo real no es sencillo.

A modo de resumen se pueden decir las características más importantes de esta luz:

- La luz ambiente no depende de la normal de los objetos y no depende la posición del observador.
- La luz difusa depende de la normal, pero, no depende de la posición del observador.
- La luz especular depende tanto de la normal como de la posición del observador:

Modelo	Depende de la Normal	Depende del Observador
Ambiente	NO	NO
Difusa	SI	NO
Especular	SI	SI

Problema:

Supongamos una luz de lambert de $I=1$ de dirección $(0,-1,0)$ que incide sobre la cara y una cara formada por los vértices $(0,0,0)$ $(1,0,0)$ $(0,0,1)$. Cual es la Intensidad de luz resultante.

Qu pasa si la cara es (0,0,0) (2,0,0) (0,0,2). Razone su respuesta

Para comenzar, se debe calcular la normal de la cara:

$A=(0,0,0)$

$B=(1,0,0)$

$C=(0,0,1)$

La normal sería:

$$\vec{N} = \|(A - B) \times (A - C)\|$$

14. Luces en OpenGL 1.2

¿Como usamos las luces en OpenGL? Va a ser exactamente igual en los dos casos.

Primero tenemos que iluminar la escena, lo que se le denomina, encender los interruptores, controlando yo mismo las luces. Después se debe indicar como es el modelo de iluminación y por último indicar las propiedades de los materiales, es decir, indicar cuáles son los materiales. Recordar que el modelo de la iluminación local dependía de las fuentes de luz, del material de los objetos y de dónde está el usuario.

Para encender la luz se necesitan realizar unas habilitaciones que se realizan con glEnable. Es recomendable (puesto que el cálculo de iluminación es costoso) que se garantice que está habilitado el buffer de profundidad (glEnable(GL_DEPTH_TEST)) y está habilitado la ocultación de caras (glEnable(GL_CULL_FACE)), entonces ya se dibujan la mitad de las caras y los objetos que se encuentran por detrás de otros no se van a dibujar. Para habilitar los cálculos de las luces se utiliza glEnable(GL_LIGHTING).

En OpenGL 1.2 se tienen 8 luces, y cuándo se pone una luz, se nota que va bajando el rendimiento.

Estas luces se identifican con un número que va de 0 a 7. Para habilitar una luz concreta, por ejemplo, la 0 se utilizaría: glEnable (GL_LIGHT0);

Para habilitar lo que se denomina seguimiento de material, es decir, para que mezcle la luz incidente y los materiales, se habilita GL_COLOR_MATERIAL (glEnable(GL_COLOR_MATERIAL)).

Con glColorMaterial(face,Mode) se indica que el modelo de material se va a aplicar sobre las dos caras.

Con el modelo de sombreado primero se ilumina la cara y luego se pueden poner sombras sobre esta, necesitando dos modelos de sombreado: GL_FLAT y GL_SMOOTH que vendrán relacionados en cómo se calcula las normales en las caras de los objetos.

Con glShadeMode(Model) se habilita el modelo de sombreado.

Para habilitar GL_FLAT o GL_SMOOTH:

```
glShadeMode(GL_FLAT);
glShadeMode(GL_SMOOTH);
```

Tipos de luz

En OpenGL 1.2 se tienen 4 tipos de luces:

- Luces “básicas” que es una luz ambiente.
- Luces direccionales.
- Luces locales
- Focos

La luz básica es una luz ambiente. Es la que se tiene por defecto.

OpenGL permite definir un modelo de iluminación general (sin poner luces) que definirá:

- Una luz ambiente básica.
- Especificará como se realiza la iluminación de la escena en cuanto a si las luces afectan a las dos partes de un objeto.

Propiedades para definir del modelo básico de iluminación, luz ambiente:

Los parámetros de la luz por defecto se pueden cambiar mediante la función `glLightModelfv`, aplicándole diferentes directivas

- `glLightModelfv(GL_LIGHT_MODEL_AMBIENT,ambiente);` → Indica que el color de la luz ambiente es el color que se almacena en la variable ambiente.
- `glLightModelfv(GL_LIGHT_MODEL_TWO_SIDE,TRUE);` → Se utiliza para que las luces afecten a ambas caras de un material.
- `glLightModelfv(GL_LIGHT_MODEL_LOCAL_VIEWER,TRUE);` → Coloca el modo de cálculo con el observador local, si no todos los cálculos se hacen como si el observador estuviese situado sobre el eje z

Estos valores anteriores, son los valores establecidos por defecto y no se suelen tocar, solo cuándo se desea cambiar el modelo de iluminación de OpenGL

Las componentes de la luz:

Anteriormente, se mencionó que cuándo se tenía una luz existían tres componentes: ambiente, difusa y especular. Entonces para todas las luces se deben especificar cuáles son las componentes RGB del ambiente, difusa y especular, marcando que color tiene cada una de esas componentes. Para esto, simplemente se definen tres componentes y se le pasa al modelo de iluminación. Ejemplo:

```
Gfloat light_ambient[]={1.0,0.0,1.0,1.0};
```

```
Gfloat light_diffuse[]={1.0,1.0,0.0,1.0};
```



```
GLfloat light_specular[]={0.0,1.0,1.0,1.0};
```

```
glLightfv(GL_LIGHT0, GL_AMBIENT, light_ambient);
glLightfv(GL_LIGHT0, GL_DIFFUSE, light_diffuse);
glLightfv(GL_LIGHT0, GL_SPECULAR, light_specular);
```

Después, se pueden tener diferentes tipos de luces:

- **Luces direccionales:** Una luz direccional situada en el infinito. No hay atenuación. Dirección que por defecto es (0,0,1,0);
- Vienen de un lugar del espacio, pero, sin ninguna aplicación. Es decir, si te mueves en la dirección de la luz hacia el infinito la luz siempre sigue alumbrando, nunca pasas el foco.
- Esas luces se caracterizan por una dirección. Lo importante para que una luz sea direccional, es que el valor de la última variable sea 0, porque entonces en vez de ser un punto, es un vector que indica de dónde viene.
- Indicando la dirección de la luz, se ilumina el objeto con los colores que se han definido previamente.
- Ejemplo:
 - o `float direccion[4]= {1.0,1.0,1.0,0.0};`
 - o `glLightfv(GL_LIGHT0, GL_POSITION, direccion);`
- **Luces locales:** Vienen de un punto e iluminan en una dirección. Por detrás de ese punto no se iluminan, por tanto, están caracterizadas por una posición y una dirección, por lo que el valor de la última componente será un 1, como contraposición de las luces anteriores.

Ejemplo:

```
float posicion[4]={1.0,1.0,1.0,1.0};
float direccion[4]={1.0,1.0,1.0,1.0};
glLightfv(GL_LIGHT0, GL_POSITION, posicion);
glLightfv(GL_LIGHT0, GL_SPOT_DIRECTION, direccion); → Marca dirección de
la fuente de luz
```

Focos de luz: En el caso anterior se tenía una luz local que venía de un punto y de una dirección. Ahora, se desea realizar una linterna con un foco de luz que alumbrar en un determinado haz. Esto básicamente es una luz direccional que se recorta. Entonces se caracteriza por la posición de la

luz, por su dirección y por el ángulo de corte. Este último, es cuánto mide cada uno de los sectores en grado. Es cuánto abre y cuánto cierra. Para esto se usa:

- `glLightfv(GL_LIGHT0, GL_POSITION, posicion);`
- `glLightfv(GL_LIGHT0, GL_SPOT_DIRECTION, direccion);`
- `glLightfv(GL_LIGHT0, GL_SPOT_CUTOFF, angulo);`

Por último, existe el parámetro `GL_SPOT_EXPONENT`, que se utiliza cuándo se degrada el cono de luz, viéndose más intensa en el centro y menos intensa en los lados. Puede valer entre 0 y 128.

A modo de resumen, para las fuentes de luz se tienen las funciones:

- `glLightf(SourceNumber, Parameter, Value)(escalar)`
- `glLightfv(SourceNumber, Parameter, PointerToArray)(vector)`

La primera es para float y la segunda para vectores.

SourceNumber: Fuente de luz (`LIGHT0`, `LIGHT1...`)

Parameter: Macros o parámetros que indican a cuáles se pueden hacer referencia:

Nombre del parámetro	Valor por defecto	Significado
<code>GL_AMBIENT</code>	<code>(0.0,0.0,0.0,1.0)</code>	Intensidad ambiente RGBA de la luz.
<code>GL_DIFFUSE</code>	<code>(1.0,1.0,1.0,1.0)</code>	Intensidad difusa RGBA de la luz
<code>GL_SPECULAR</code>	<code>(1.0,1.0,1.0,1.0)</code>	Intensidad especular RGBA de la luz
<code>GL_POSITION</code>	<code>(0.0,0.0,1.0,0.0)</code>	Posición(x,y,z,w) de la luz
<code>GL_SPOT_DIRECTION</code>	<code>(0.0,0.0,-1.0)</code>	Dirección (x,y,z) del foco de la luz
<code>GL_SPOT_EXPONENT</code>	<code>0.0</code>	Exponente del foco de luz

GL_SPOT_CUTOFF	180.0	Ángulo del foco de luz
GL_CONSTANT_ATTENUATION	1.0	Factor de atenuación constante
GL_LINEAR_ATTENUATION	0.0	Factor de atenuación lineal
GL_QUADRATIC_ATTENUATION	0.0	Factor de atenuación cuadrático

Ejemplo:

Variables

```
Gfloat light_ambient[]={1.0,1.0,1.0,1.0};
```

```
Gfloat light_specular[]={1.0,1.0,1.0,1.0};
```

```
Gfloat light_diffuse[]={1.0,1.0,1.0,1.0};
```

```
Gfloat light0_pos[]={100.0,200.0,0.0,1.0};
```

Define los valores

```
glLightfv(GL_LIGHT0, GL_POSITION, light0_pos);
```

```
glLightfv(GL_LIGHT0, GL_AMBIENT, light_ambient);
```

```
glLightfv(GL_LIGHT0, GL_DIFFUSE, light_diffuse);
```

```
glLightfv(GL_LIGHT0, GL_SPECULAR, light_specular);
```

Se deben definir estos parámetros para cada variable luz.

15. Normales/Caras OpenGL 1.2

Al iluminar las caras, se tiene el siguiente problema: cuándo se indica la normal de una cara, se coge la cara y se le indica la normal directamente, no obstante, esto en OpenGL no funciona así, sino que considera, puesto que tiene los vértices, la posible especificación de las normales para cada uno de los vértices, obteniendo así una normal por cara. Por esto, OpenGL implementa 2 tipos de sombreado: el modelo de iluminación plana (FLAT) y el modelo Gouraud

Iluminación plana (FLAT)

La iluminación plana es la más simple y eficiente. En ella todo el polígono presenta el mismo color pues OpenGL evalúa sólo un color para todos sus puntos.

Se activa con `glShadeModel(GL_FLAT);`

El modelo FLAT lo que hace es especificar una única normal para toda la cara. Esta normal puede ser un valor indicado por el programador o la puede calcular OpenGL automáticamente, normalizándola en función de los vértices. Todo el color de esa cara va a ser constante, lo que se denomina un sombreado plano.

El valor de la normal se puede especificar con la función `glNormal3fv`.

Ejemplo:

```
glNormal3fv(normals[1]);
glVertex3fv(vertices[1]);
glVertex3fv(vertices[5]);
glVertex3fv(vertices[6]);
glVertex3fv(vertices[2]);
```

16. Modelo Gouraud

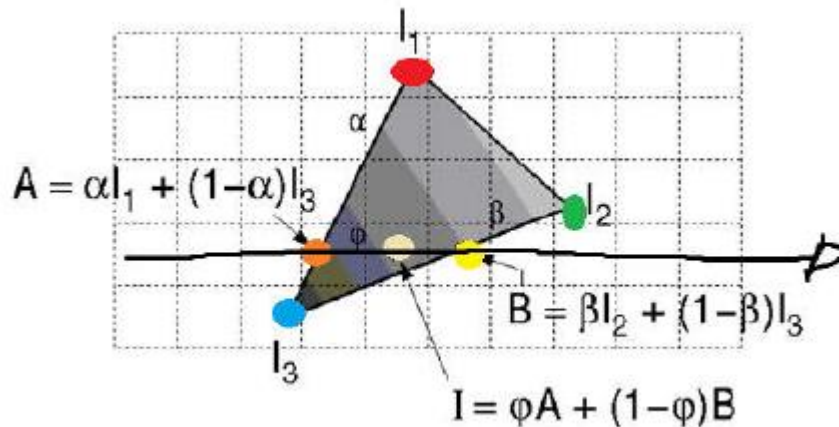
A cada vértice se le asigna una normal y, entonces, promedia los valores de color en toda la superficie del objeto para así alcanzar un efecto de sombreado suave que parece más realista sobre superficies curvas.

Es el denominado sombreado suave. Se puede habilitar con:

- `glEnable(GL_SMOOTH);`
- `glShadeModel(GL_SMOOTH);`

Cálculo:

Se supone que los objetos están compuestos por varias caras. Por ejemplo, si se tiene una figura con varias caras y no tiene especificadas las normales, lo que se hace es, a todos los vértices de cada cara, se les aplica la normal que se calcula a través de los vértices que compone la cara. El problema surge cuando aparece un vértice en el que hay varias normales de varias caras. En este caso lo que se hace es calcular una normal como la media de las caras.



En caso de indicar las normales de forma forzada, se debe especificar la normal con glVertex3fv

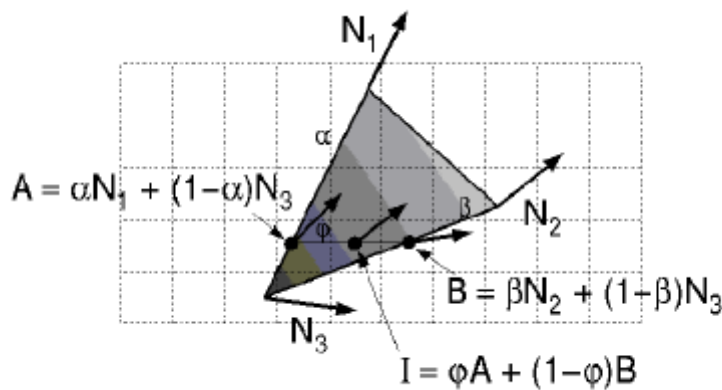
Ejemplo:

```
for (i=0; i<3; i++){
    glNormal3fv(normAvg[i]); /*first face of pyramid*/
    glVertex3fv(vertices[i]);
}
glNormal3fv(normAvg[0]);
glVertex3fv(vertices[0]);/*Second face*/
glNormal3fv(normAvg[2]);
glVertex3fv(vertices[2]);
glNormal3fv(normAvg[3]);
glVertex3fv(vertices[3]);//etc
```

Esto significa que se tiene la normal de cada vértice, la luz que llega y el material del objeto en cada cara. En función de esto, se puede calcular el color de cada uno de los vértices. Lo que falta es calcular el color de cada uno de las caras, ya que la normal de cada vértice es diferente. Si se tiene el modelo FLAT, toda la cara será igual, pero, si se tiene el modelo Gouraud calcula el color de cada uno de los vértices por separado y después, en la línea de píxeles que realiza el rasterizado, va interpolando los colores, es decir, el color de cada punto es calculado mediante la interpolación bilinear entre los colores del mismo.

Esto lleva a que exista una degradación en las caras porque, aunque las caras tengan todas el mismo color y la luz incida más o menos igual, las normales de cada una de las caras van a variar. Para que se vean igual como si fuera FLAT, las normales de las caras deberían ser iguales.

17. PHONG shading



Este modelo ya no se encuentra implementado en las tarjetas gráficas.

Funciona de manera similar al de Gouraud, pero con un cambio. El Gouraud consideraba el color en los vértices, en función de la luz y de la normal y calculaba por interpolación el color en un punto. En PHONG solo interesa saber cuáles son las normales, entonces, calcula la normal de un punto del interior de la cara por la interpolación de las normales de los vértices. Una vez que hace esto, con el valor de la normal, aplica el modelo de iluminación y calcula el color. En el modelo Gouraud se interpolaban los colores y en el modelo de PHONG lo que se hace es calcular las normales y después calcular el color, es lo que se llama interpolación bi-lineal de las normales. A cada punto del polígono se le asocia una normal distinta, con lo cual, su color asociado será muy cercano a la realidad.

Esto lo hace automáticamente OpenGL, aunque, no lo implementa directamente.

El programador es el encargado de interpolar las normales y asociarlas a todos los puntos.

El modelo PHONG se considera mejor que las soluciones anteriores, sin embargo, consume más recursos.

18. Luces en OpenGL

Se debe también caracterizar el material del objeto. El objeto depende de la luz con la que se ilumine. Por ejemplo, si la luz es blanca y el objeto es rojo, este es rojo, pero, si se ilumina con una luz azul se verá negro, puesto que, si se ve rojo, quiere decir que le llegan todas las frecuencias de la luz blanca y refleja la roja, absorbiendo la azul. Esto lleva a pensar que también se deben tener unas características de los materiales de los objetos, para saber cómo se van a comportar dependiendo de la luz. Básicamente, esto trata de especificar cómo se va a reflejar la luz que aparece sobre ellos. El modelo de iluminación de OpenGL realiza una aproximación del color de un material en función de la luz que incide sobre él y la luz que es capaz de reflejar.

Los objetos van a tener una propiedad que es cómo se comportan frente a la luz ambiente, difusa y especular. Lo que se va a hacer es multiplicar las diferentes propiedades por el color de la luz incidente. Es decir, si se tiene una luz ambiente (1,0,0) y la luz que llega es una luz verde (0,1,0), si se multiplican estos vectores, se tiene que el resultado es 0. Esto es lógico, ya que, si se tiene un objeto azul y se ilumina con una luz verde, se ve negro.

19. Materiales en OpenGL:

OpenGL ofrece un conjunto de primitivas que se utilizarán para especificar diferentes parámetros del material en cada polígono.

```
void glMaterial{if}{v}(GLenum face, GLenum pname, TYPE *param);
```

Los materiales se pueden especificar en función de:

GL_enum face	GLenum pname	Param	Valores por defecto
GL_FRONT	GL_AMBIENT	(R,G,B,1.0f)	(0.2,0.2,0.2,1.0)
GL_BACK	GL_DIFFUSE	(R,G,B,1.0f)	(0.8,0.8,0.8,1.0)
GL_FRONT_AND_BACK	GL_AMBIENT_AND_DIFFUSE	(R,G,B,1.0f)	
	GL_SPECULAR	(R,G,B,1.0f)	(0.0,0.0,0.0,1.0)
	GL_SHINNIES	[0,128]	0.0
	GL_EMISSION	(R,G,B,1.0f)	(0.0,0.0,0.0,1.0)
	GL_COLOR_INDEXES	(R,G,B)	(0,1,1)

--	--	--	--

Por defecto, los parámetros son blancos o grises, haciendo así, que todos los objetos sean blancos y que solo se dependa de la luz incidente.

En OpenGL solo hay 8 luces debido a que esto es computacionalmente costoso. Normalmente se usan 2 o 3 luces.

Ambient: comportamiento respecto a la luz Ambient. Se refiere al color por defecto del material.

Diffuse: comportamiento respecto a la luz Diffuse

Capacidad de emisión y reflexión

Respecto a la especularidad y shininess, para los objetos no se suele usar y es exactamente igual al que se utilizaba hasta ahora.

- Specular: Indica cómo se va a comportar el material en función de la componente Specular del rayo de luz incidente.

- o La propiedad especular se suele definir blanca o gris o del color con el que se ilumine, de forma que los reflejos del material se degradan desde el color base de la luz incidente hasta el color definido en esta propiedad.

- Shininess

- o Determina la concentración de los puntos que van a reflejar la luz.

- o Puede ser un único punto o toda la superficie del objeto [0,128] por defecto es igual a 0. Cuanto mayor, más metálico, coseno de el “n”.

- Emission

- o Esta propiedad define la capacidad de simular la emisión de luz.

- o No es una fuente de luz

- o Se definen los colores de la luz que emite

- o Sirve para simular estrellas, lámparas u otros objetos similares.

Ejemplo:

```
GLfloat ambientl[] = { 0.0, 0.0, 0.0, 1.0 };
```

```
GLfloat diffusel[] = { 1.0, 1.0, 1.0, 1.0 };
```

```
GLfloat specularl[] = { 1.0, 1.0, 1.0, 1.0 };
```

```
GLfloat position[] = { 0.0, 3.0, 2.0, 0.0 };
```

```
glClearColor(0.0, 0.1, 0.1, 0.0);
```

```
glEnable(GL_DEPTH_TEST);
```



```

glShadeModel(GL_SMOOTH);
glLightfv(GL_LIGHT0, GL_AMBIENT, ambientl);
glLightfv(GL_LIGHT0, GL_DIFFUSE, diffusel);
glLightfv(GL_LIGHT0, GL_POSITION, positionl);
glEnable(GL_LIGHTING);
glEnable(GL_LIGHT0);
void display(void)
{
    GLfloat ambient[] = { 0.7, 0.7, 0.7, 1.0 };
    GLfloat diffuse[] = { 0.1, 0.5, 0.8, 1.0 };
    GLfloat specular[] = { 1.0, 1.0, 1.0, 1.0 };
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
    glMaterialfv(GL_FRONT, GL_AMBIENT, ambient);
    glMaterialfv(GL_FRONT, GL_DIFFUSE, diffuse);
    glMaterialfv(GL_FRONT, GL_SPECULAR, specular);
    glMaterialfv(GL_FRONT, GL_SHININESS, shininess);
}

```

Cálculo del color por vértice

$$\begin{aligned}
 vertexcolor = & emission_{material} + ambiente_{lightmodel} * ambiente_{material} \\
 & + \sum_1^{n-1} \left(\frac{1}{a + bd + cd^2} \right) * spotlight_effects * \\
 & [ambiente_{light} * ambiente_{material} + diffuse_{light} * diffuse_{material} + specular_{light} * specular_{material}]
 \end{aligned}$$

El color de cada vértice, que al final es lo que se obtiene, pues el modelo de Phong y de Gouraud al final lo que hacen es la interpolación de cada vértice, depende de: el material de emisión más la luz ambiente por el material.

En la fórmula anterior:

- $\sum_1^{n-1} (\frac{1}{a+bd+cd^2})$ Sería el parámetro de atenuación.
- `ambiente_lightmodel` → Luz básica.
- `spotlight_effects` → Ángulo.
- `diffuse_light * diffuse_material` → Luz*material

El parámetro de atenuación tiene un sumatorio debido a que pueden existir varias luces. Se pueden atenuar las luces en función de la distancia.



Escola
Técnica
Superior
de Enxeñaría



Grado en Enxeñaría Informática

2º curso – 2º cuadrimestre

Computación gráfica

24-03-2021



Clase 07

Temas traballados: Tema 5 (Iluminación en 3.3)

Índice

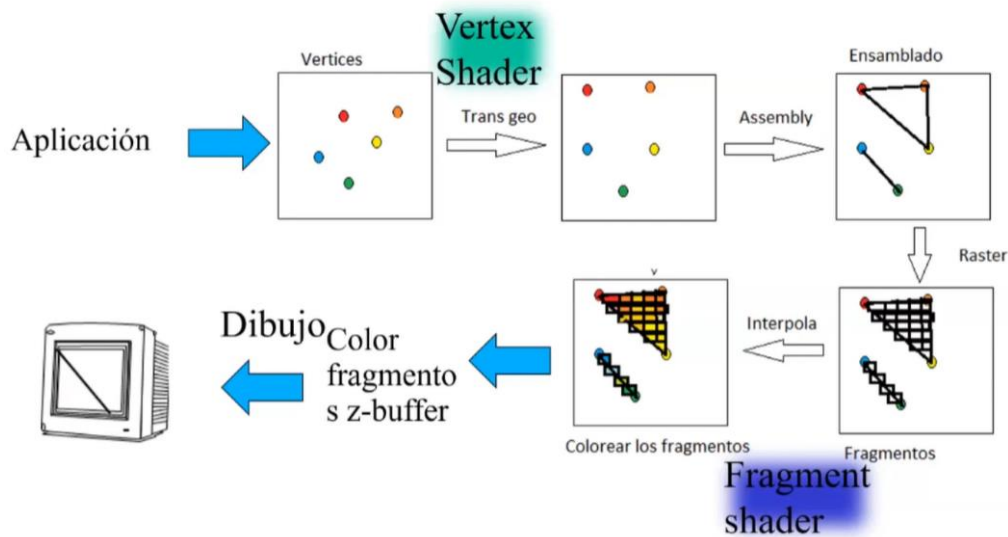
1. Iluminación en 3.3.....	3
Pipe 3.3.....	3
Iluminación básica	3
Implementación da iluminación básica	3
Luz difusa.....	4
Implementación da iluminación difusa	4
Luz especular	5
Implementación da luz especular	6
Implementación da Luz en cono	6
2. Anexo: fórmulas de iluminación	7
Luz ambiental.....	7
Luz difusa ou de Lambert.....	7
Luz especular ou de phong.....	7
Modelo de iluminación simple con cor	7

1. Iluminación en 3.3

Pipe 3.3

Na pipe 3.3 non nos podemos axudar dalgúnhas das función que usabamos na versión 1.2, se non que **realizamos as operacións a partir do vertex shader e do fragment shader**, o cal nos dá unha maior liberdade, pero a cambio de ter un maior traballo de implementación. En concreto **no vertex shader temos que facer operacións relativas as normais e ás posicións dos obxectos**, que son tidas en conta nalgúns modelos de iluminación. **No fragment shader é onde faremos os cálculos da cor e da iluminación propiamente.**

Pipe 3.3



Iluminación básica

“El color del objeto es el color del objeto por la luz que lo ilumina” Julián Flores (24-03-2021)

A luz ambiental é aquela que **non provén dunha dirección concreta**, incidindo sobre todas as partes do obxecto por igual.

Implementación da iluminación básica

Para a implementar a iluminación básica utilizamos o seguinte fragment shader, no cal temos unha variable vector RGB para a cor da luz, outra para a cor do obxecto e unha constante de atenuación para incrementar ou decrementar a intensidade da luz. Por último multiplicamos a cor da luz pola do obxecto.

```
uniform vec3 lightColor;
uniform vec3 objectColor;
void main()
{
    float ambientI = 0.1;
    vec3 ambient = ambientI * lightColor;
    vec3 result = ambient * objectColor;
    gl_FragColor = vec4(result, 1.0);
}
```

(A implementación da imaxe está obsoleta, temos que usar un vector out para almacenar o vector FragColor, non usar gl_FragColor). Faríase da seguinte maneira:

```

1  #version 330 core
2  out vec4 FragColor;
3
4  in vec3 Normal;
5  in vec3 FragPos;
6
7  uniform vec3 lightPos;
8  uniform vec3 lightColor;
9  uniform vec3 objectColor;
10
11 void main()
12 {
13     // ambient
14     float ambientI = 0.5;
15     vec3 ambient = ambientI * lightColor;
16     vec3 result = (ambient) * objectColor;
17     FragColor = vec4(result, 1.0);
18
19
20 }
21

```

Luz difusa

Este tipo de luz ven dunha dirección concreta e é reflectida en todas as direccións. A luz reflectida pola superficie depende da normal e é proporcional ó coseno do ángulo de incidencia coa normal da superficie. Ademais tamén ten unha intensidade e unha constante da atenuación. O ángulo de incidencia calcúlase a partir da posición da luz e da posición na que incide. **Hai que normalizar as normais para que non dependan do tamaño do obxecto, só da súa posición e orientación.**

Implementación da iluminación difusa

Para a implementación da luz difusa podemos usar o seguinte vertex shader. Neste caso temos dous vectores out (Normal e Fragpos), un para almacenar as novas normais unha vez recalculadas, xa que estas van variar en función da orientación do obxecto, e outro para almacenar as coordenadas dos fragmentos dentro da propia escena. Estes dous vectores enviaranse ó fragment shader.

```

#version 330 core
layout (location = 0) in vec3 aPos;
layout (location = 1) in vec3 aNormal;
out vec3 Normal;
out vec3 FragPos;
uniform mat4 model;
uniform mat4 view;
uniform mat4 projection;

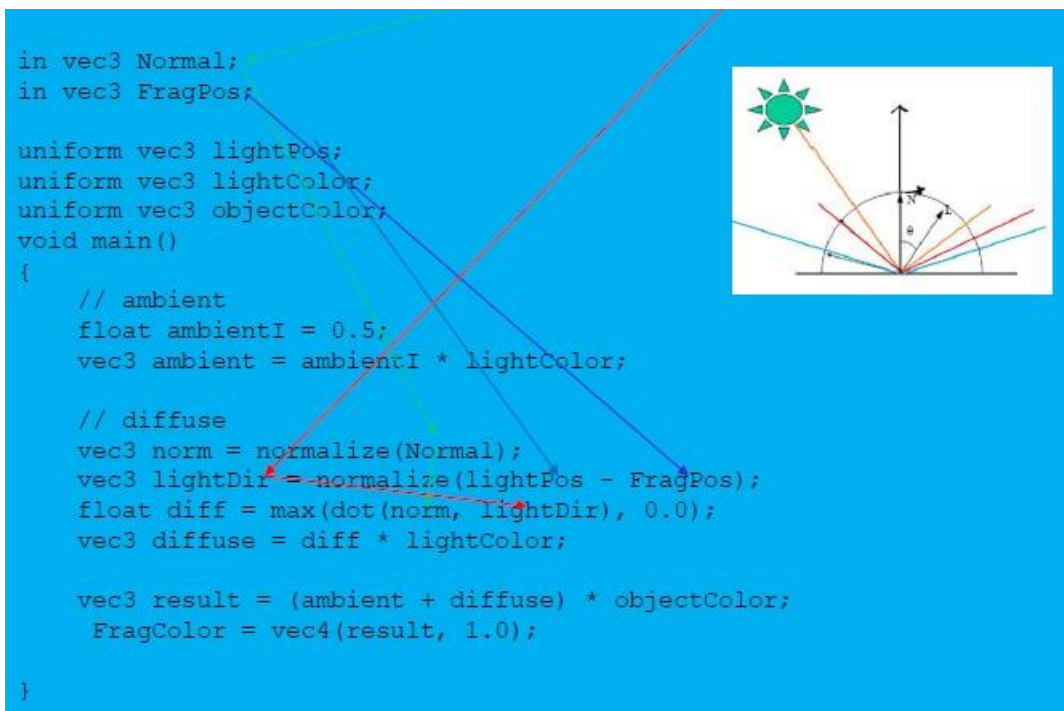
void main()
{
    gl_Position = projection * view * model * vec4(aPos, 1.0f);
    Normal = mat3(transpose(inverse(model))) * aNormal;
    FragPos = vec3(model * vec4(aPos, 1.0));
}

```

Pódese entrar na carpeta de *Teoría* do Tema 5 *Iluminación* e abrir o documento **The Normal Matrix** para ver unha aclaración de como se realiza o cálculo das novas normais en función da orientación do obxecto.

Por outro lado, o fragment shader quedaría da seguinte maneira. Temos en primeiro lugar as normais xa recalculadas que veñen do vertex shader e por outro os fragmentos da escena. Tamén temos a posición da luz, que xunto coa posición do fragmento vamos permitir calcular a liña de incidencia da luz na cara e desta maneira podemos calcular o ángulo entre esta liña de incidencia e a propia normal do obxecto. Temos que normalizar as normais para que estas non dependan do tamaño do obxecto, só da súa orientación. Se temos dúbidas de se normalizar ou se non, normalizamos. Esta é unha boa práctica de programación. Tamén normalizamos o vector de dirección da luz. Posteriormente realizamos un produto escalar entre a normal do fragmento e a dirección da luz (operación dot) e collemos o máximo entre o resultado desta operación e cero para que en caso de que este valor sexa negativo, que a iluminación do fragmento sexa nula.

A compoñente difusa vai ser o resultado da anterior operación pola cor da luz, mentres que o resultado vai ser a suma desta luz mais da ambiente pola cor do propio obxecto. Por último introducimos este resultado no FragColor engadindo a variable alpha.



Para practicar con ditas luces, recoméndase a visita aos diferentes simuladores que se atopan no arquivo comprimido **tutorial.zip**.

Luz especular

Este tipo de iluminación está asociada ó brillo orixinado por un foco de luz nun obxecto. Depende da intensidade da luz, da posición do foco e do **punto de vista do observador**. Ten un coeficiente de especularidade que aumenta ou diminúe o efecto do reflexo (Se é 1 teríamos un efecto mate e se é ∞ un efecto espello) e tamén intervéen unha variable de atenuación que vai facer posible aumentar ou disminuir o efecto deste tipo de iluminación. A posición do foco

utilizárase da mesma maneira que no caso da luz difusa, é dicir, vamos servir para calcular a dirección da luz e así calcular o ángulo entre esta dirección e a normal do fragmento.

Implementación da luz especular

Un posible fragment shader que implementa este tipo de iluminación é o seguinte. O único que ata agora non tivemos en conta nos distintos tipos de luz é a posición do observador, polo tanto precisamos un novo vector (viewPos) que almacene esta información. SpecularStrength é unha variable que usaremos para almacenar a anteriormente mencionada variable de atenuación. En ViewDir imos almacenar o cálculo do vector entre o observador e o fragmento, para saber a dirección. En reflectDir almacenamos a dirección da luz reflectida, calculada mediante a función reflect da librería GLSL á que lle pasamos a dirección da luz pero en negativo (sentido contrario) e a normal do fragmento. Posteriormente, calculamos a compoñente especular seguindo a súa fórmula. De novo volvemos escoller o máximo entre 0 e o produto escalar da dirección de visión e da dirección de reflexo para que se é un valor negativo non se ilumine. Ademais elevamos este valor ó coeficiente de especularidade, neste caso 128. A compoñente especular final vai ser o valor agora calculado multiplicado pola variable de atenuación e multiplicado pola cor da luz.

Por último calculamos a luz resultado como a suma das tres iluminacións anteriores multiplicadas pola cor do obxecto. Introducimos este resultado no FragColor engadindo a variable alpha.

Luz Specular. Shader-frag

```
#version 330 core
out vec4 FragColor;

in vec3 Normal;
in vec3 FragPos;

uniform vec3 viewPos;
uniform vec3 lightPos;
uniform vec3 lightColor;
uniform vec3 objectColor;
void main()
{
    // ambient
    float ambientI = 0.5;
    vec3 ambient = ambientI * lightColor;

    // diffuse
    vec3 norm = normalize(Normal);
    vec3 lightDir = normalize(lightPos - FragPos);
    float diff = max(dot(norm, lightDir), 0.0);
    vec3 diffuse = diff * lightColor;

    //Speculas
    float specularStrength = 1.0;
    vec3 viewDir = normalize(viewPos - FragPos);
    vec3 reflectDir = reflect(-lightDir, norm);
    float spec = pow(max(dot(viewDir, reflectDir), 0.0), 128);
    vec3 specular = specularStrength * spec * lightColor;

    vec3 result = (ambient + diffuse + specular) * objectColor;
    FragColor = vec4(result, 1.0);
}
```

//la posición del usuario/cámara (0,0,10) en nuestro caso
 unsigned int viewPosLoc = glGetUniformLocation(shaderProgram, "viewPos");
 glUniform3f(viewPosLoc, 0.0f, 0.0f, 10.0f);

Implementación da Luz en cono

Podemos darlle forma de cono á iluminación difusa e especular se condicionamos a súa aparición cunha sentenxa deste tipo: “if(acos(dot(-lightDir, -lightPos))< radians(15.0))”. O que estamos observando é se o ángulo entre a dirección da luz e a súa posición é menor que un ángulo concreto, podendo elixir non iluminar os obxectos fóra dese campo.

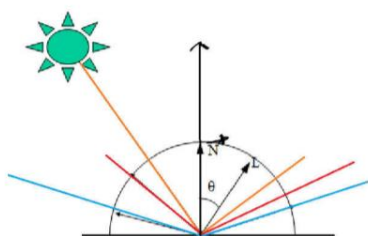
2. Anexo: fórmulas de iluminación

Luz ambiental

- Donde $I = I_a \cdot K_a$
 - k_a coeficiente de reflexión ambiental $[0,1]$ dependiente de cada objeto.
 - I_a intensidad ambiental en todo punto del espacio.

Luz difusa ou de Lambert

$$I = I_L \cdot K_d \cos(\theta) = I_L \cdot K_d \cdot (\vec{N} \cdot \vec{L})$$



- I_L : Intensidad de la fuente
- θ : Ángulo de incidencia $[0^\circ, 90^\circ]$
- K_d : Coeficiente de reflexión para la luz difusa $[0,1]$
- $\vec{N}$: Normal a la superficie (unitario)
- $\vec{L}$: Vector de iluminación (unitario)

Luz especular ou de phong

- Modelo de iluminación Especular:

$$I = I_L \cdot K_s \cos^n(\alpha) = I_L \cdot K_s \cdot (\vec{R} \cdot \vec{V})^n$$

α : Ángulo entre $\vec{R}$ y $\vec{V}$. Angulo entre vista y reflexión.

K_s : Coeficiente de reflexión especular dependiendo del material del objeto $[0,1]$

n : Coeficiente de especularidad o concentración del brill

$(1 \leq n < \infty)$, 1: mate, ∞ : espejo)

Modelo de iluminación simple con cor

É un modelo cos tres anteriores tipos de iluminación e separado por cores.

$$I_R = I_{aR} \cdot K_a \cdot C_{dR} + I_{LR} \cdot f_{at} \cdot \left(K_d \cdot C_{dR} \cdot (\vec{N} \cdot \vec{L}) + K_s \cdot C_{eR} \cdot (\vec{R} \cdot \vec{V})^n \right)$$

$$I_G = I_{aG} \cdot K_a \cdot C_{dG} + I_{LG} \cdot f_{at} \cdot \left(K_d \cdot C_{dG} \cdot (\vec{N} \cdot \vec{L}) + K_s \cdot C_{eG} \cdot (\vec{R} \cdot \vec{V})^n \right)$$

$$I_B = I_{aB} \cdot K_a \cdot C_{dB} + I_{LB} \cdot f_{at} \cdot \left(K_d \cdot C_{dB} \cdot (\vec{N} \cdot \vec{L}) + K_s \cdot C_{eB} \cdot (\vec{R} \cdot \vec{V})^n \right)$$



Escola
Técnica
Superior
de Enxeñaría



Grado en Ingeniería Informática

2º curso – 2º cuatrimestre

Computación Gráfica

08-04-2021

Clase 08

Temas trabajados:

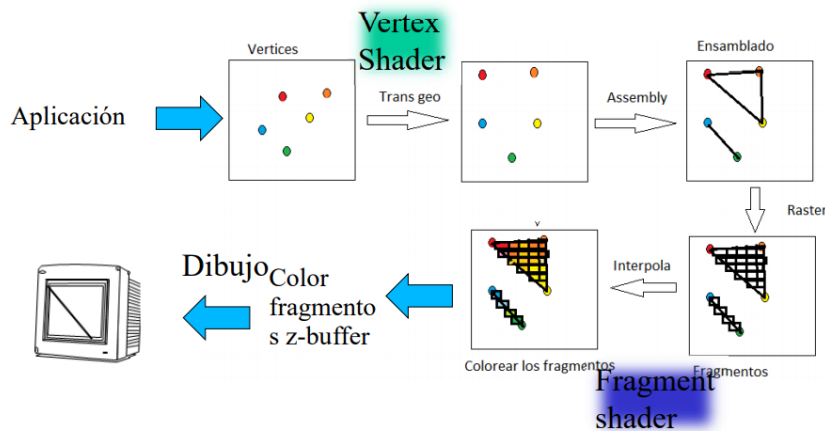
Tema 6: Texturización

Índice

1. Texturización.....	3
Pipe 3.3.....	3
Introducción	3
Texturizado	3
Mapeado, tipo texturas.	4
Texturas procedurales	4
Bump mapping	5
Light mapping.	5
Displacement mapping.....	5
Mapeado	6
Mapeado directo de texturas u,v.	6
Coordenadas U,V	6
2. Mapeado de Texturas (OpenGL).....	7
Habilitar el mapeado de texturas	7
Leer y activas la textura (hacerla operativa)	7
Almacenar en memoria.	8
Leerla de un fichero	9
Definir textura	9
Pegar Textura	10
En OpenGL 1.2 se puede usar en listas:.....	11
Ejemplo con dos texturas:	12
En OpenGL 3.3:	12
Parámetros de textura	14
Problemas de tamaño: glTexParameterf.....	15
Problemas de escala: glTexParameterf.....	15
Ejemplo Bob Esponja.....	16
Mapeado multicapa 3.3	19
Canales Alpha	20
Test Alpha	21
Alpha+Textures.....	21
Prácticas	22
Preguntas.....	22

1. Texturización

Pipe 3.3



Ya casi la hemos visto por completo, tenemos nuestra aplicación, nuestros vértices, el vertex Shader o transformaciones geométricas que colocan nuestros vértices en la escena. Luego se ensamblará, se hará un rasterizado. En este último proceso se verá que los elementos que generan las caras. Con el Fragment shader se utilizará una textura. Se cambia la textura resultando colores distintos, se puede mezclar con la iluminación....

Introducción

- La utilización de texturas:
 - o Aumenta el realismo de la imagen **sin aumentar el número de polígonos.**
 - o Proporciona una clave para la **percepción de la profundidad.**
 - o Permite simular multitud de efectos: sombras, reflejos, entornos.
 - o Permiten añadir FX.
 - o **PROBLEMA: Ocupan espacio...**
- **Mapa de texturas:** colección o array n-dimensional de valores que se representan sobre la geometría. No tienen que ser una imagen.

Texturizado

Es una palabra amplia porque hacer referencia a varios tipos de trabajos sobre la superficie de los objetos. En resumen, el texturizado permite darle propiedades a la superficie.

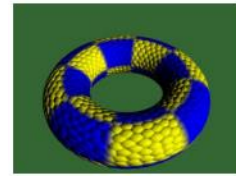
El texturizado consiste en superponer sobre los polígonos imágenes que aumenten su realismo. Existen varias técnicas de texturizado:

- **Texture Mapping:** Crea una textura a partir de un patrón (imagen) sobre la superficie. Es solamente colocar una imagen sobre una figura. (ejemplo del flotador)



1. Texturización

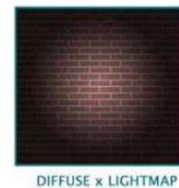
- **Bump Map:** Perturba la superficie mediante una modificación de las normales u otros (puede no considerado mapeado en su sentido estricto). Tenemos una imagen, pero la imagen no tiene información sobre el color, sino sobre la normal. (ejemplo del segundo flotador).(No vamos a utilizarlo)



- **Environmental Map:** Mapea el entorno sobre la superficie. (e.g. reflections, light mapping). Reflejos: ejemplo de la imagen del espejo.



- **Light mapping.** Mapea un mapa de iluminación sobre el objeto. (Pared de ladrillos)



Mapeado, tipo texturas.

¿Qué tipos de texturas tenemos?

- **Texturas 1D:** de color, son imágenes sin alto, sólo con ancho.
- **Texturas 2D:** con imágenes, Bitmaps ordinarias.
- **Texturas 3D:** texturas 2D apiladas. Sólo las aceptan algunos sistemas pero actualmente las pueden procesar cualquier pc. Ocupan mucha memoria.
- **Texturas procedurales.**

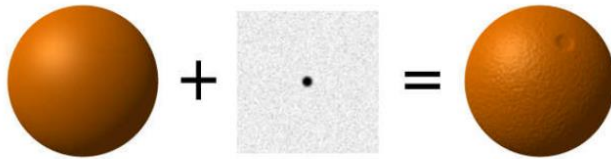
Texturas procedurales

- La “imagen” es calculada a partir de **una función matemática.**
- Se precisa un algoritmo una función R^n , generalmente en R^2
- Ocupa poco espacio, se puede realizar en tiempos de ejecución
- Se puede re-escalar fácilmente.
- Útil para superficies grandes o pequeñas
- No consume memoria sino CPU.
- Tipos de mapas (ejemplo)
 - Geométricos.
 - Patrón repetitivo.
 - Ruido fractal. (*nubes, árboles...Se usa mucho en los juegos multijugador para cargar distintos paisajes, islas...*)
 - Funciones de turbulencia
- Generación de mapas de juegos y fenómenos naturales.

Bump mapping

Modifica las normales de las caras de un objeto sin cambiar su geometría.

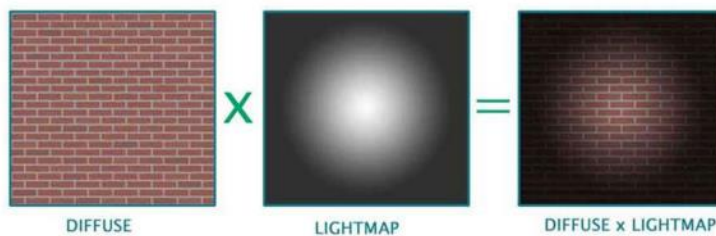
- Se utiliza un mapa de alturas, una matriz de datos $2n \times 2n$ generalmente una imagen.
- Se calcula la normal en función de ese mapa de alturas.
- Se combina las normales con la geometría del objeto.
- Se calcula el modelo de iluminación para esos nuevos valores de normales
- Podemos acceder a cada uno de los pixeles de la imagen
- Más simple que usando polígonos, mucho más óptimos.



Light mapping.

El brillo de las superficies en una escena se calcula previamente y se almacena en mapas de textura para su uso posterior. Los mapas de luz se aplican con mayor frecuencia a objetos estáticos para proporcionar efectos de iluminación como la iluminación global a un costo muy bajo.

Los mapas de luz son en blanco y negro y sirven para dar profundidad e iluminación a una escena.



Displacement mapping

Son muy similares al bump mapping solo que en este caso generan geometrías. No se van a ver en clase.

- Modifica la geometría de un objeto mediante un mapa de texturas.
- En general en la dirección de las normales de la misma.
- Genera una gran cantidad de geometría adicional
- Ventajas frente al bump mapping.
 - Puede genera sombras.
 - Pueden genera oclusiones.
 - Puede usar para sombreados.

- Desventajas es lento.
- Geometry shader: shader especial que nosotros no hemos visto, se aplica después del vertex shader y permite generar geometría.
- Usos: terrenos, Google maps los utiliza.
- No se usan mucho

Mapeado

Es coger una imagen y pegarla sobre el objeto.

Hay dos técnicas de mapeado:

- **Directo:** la textura se mapea sobre el objeto
- **Dos fases:** de esta forma se determina que parte de la textura corresponde a cada uno de los píxeles.
 - Se mapea la imagen sobre un objeto 3D
 - Esférico
 - Cilíndrico
 - Otros
 - Se proyecta sobre el objeto.
- **PARA PRÁCTICA:** las personas que hayan usado la esfera proporcionada por glut tendrán que usar el mapeado en dos fases, mientras que si se usa la esfera proporcionada por el profesor se puede utilizar un mapeado directo, lo que resulta más sencillo.

Mapeado directo de texturas u,v.

La textura se pega directamente en el objeto. Para calcular el valor de un píxel determinado es necesario utilizar **la relación** entre la textura (u, v) y los vértices superficie.

- Objetivo es obtener las coordenadas de textura unos valores que nos permita indicar que coordenada de la textura corresponde a cada vértice
- En general este proceso es muy complejo y suele utilizar un programa de diseño CAD para obtener las coordenadas de texturas.

Para este mapeado lo que necesitamos saber es un punto que tiene unas coordenadas 'x', 'y', 'z' y los puntos de la textura ('u' y 'v'). En OpenGL las coordenadas u y v van del 0 al 1. Lo que tendremos que establecer es una relación entre los puntos 'x', 'y', 'z' que tendrá una textura determinada. Este tamaño de textura es independiente de si la textura es cuadrada o rectangular, lo que va a hacer es coger y normalizar las texturas.

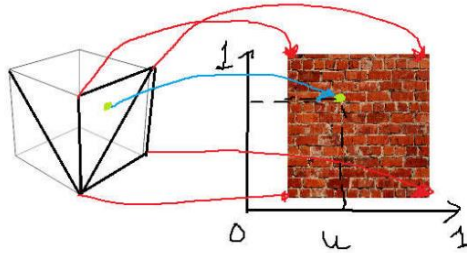
Obtener el mapa u y v de las texturas es indicar que coordenadas de las texturas corresponden a cada vértice.

Coordenadas U,V

- Una textura 2D es representada mediante un patrón de datos 2D, $T(u,v)$ donde $u(x)$ y $v(y)$ se denominan texture coordinates.

2. Mapeado de Texturas (OpenGL)

- En un mapa de texturas se asocia un único punto de $T(u, v)$ a un punto (x, y, z) de la geometría del objeto. Y entonces es mapeado dicho punto sobre la geometría y a continuación sobre la pantalla, o usarlo como información del fragment.shader. Se usa interpolación para saber el resto de los puntos que no se asocian directamente.



2. Mapeado de Texturas (OpenGL)

Práctica:

1. Habilitamos las texturas.
2. Damos un nombre a la textura: un índice, un entero. Función que nos dará los índices de las texturas.
3. Crear o leer la textura de algún sitio, fichero, imagen...
4. Descripción de la imagen: ancho, alto, donde empieza, si es o no RGB...
5. Dentro del objetos tenemos que decir que parámetros son para (u, v) y cómo se aplican
6. Activar la textura, es decir, hacerla operativa para cada objeto.

Habilitar el mapeado de texturas

En el `main()` o en `InitOpengl()`:

```
glEnable(GL_TEXTURE_2D);
```

Si quieres que la superficie aparezca sin textura con el color original en sólido será necesario deshabilitarlo.

```
glDisable(GL_TEXTURE_2D);
```

Se activar o desactivar, pero se recomienda.

```
glBindTexture(GL_TEXTURE_2D, 0);
```

Normalmente cuando queréis utilizar una textura, como veremos, se utilizan como las lista o los vaos, una vez que se habilita cualquier operación que se haga sobre una textura va a ser aplicada sobre la que está operativa. Existe una forma de no deshabilitar las texturas y no utilizar ninguna. Es como si se dejasen las luces encendidas, pero no desenchufadas, es hacer un bind a una textura de índice 0.

Leer y activas la textura (hacerla operativa)

Funciones:

- `void glGenTextures( GLsizei n, GLuint * textures );`
 - `n` es el número de índices a crear

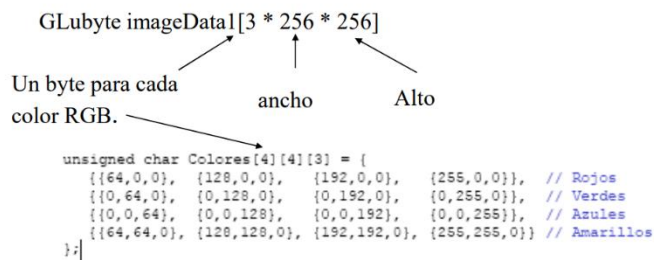
2. Mapeado de Texturas (OpenGL)

- texturas es un puntero al primer elemento del array en donde almacenarán los índices creados
- `glGenTextures(1, &texture[0]);`
- En general, el profesor genera texturas de uno en uno
- `void glBindTexture( GLenum target, GLuint texture );` para modificarla
 - target: `GL_TEXTURE_1D` o `GL_TEXTURE_2D`
 - textura es un índice de la textura que queremos enlazar (no en uso)
 - `glBindTexture(GL_TEXTURE_2D, texture[0]);`
- **Esto sirve tanto para hacerla operativa como usarla después al vuelo como las listas.**
- `glBindTexture(GL_TEXTURE_2D, 0);` // no queremos usar ninguna, útil para mezclar entre objetos texturizados y no texturizados.
- `glDeleteTextures (GLsizei num, const GLuint *nombres)`

Almacenar en memoria.

Darle valores a la imagen no es complicado, en algunos casos se hace y sobre todo se utiliza para modificar puesto que las imágenes son en principio R, G, B entonces cada uno es un Byte por lo que el tamaño de una imagen de 256*256 será:

`GLubyte imageData[3 * 256 * 256]`, entonces bastará con darle valores a cada uno de los componentes R,G,B para hacer la imagen.



En imagen 2D pocas veces se usa este método, a no ser que sea para modificar un punto en concreto.

Para crear una textura:

- 1) Escribir un programa que la genere, procedurales
- 2) Leerla de un fichero, imágenes

Leerla de un fichero

La forma natural de ver una imagen es leerla de un fichero. Esto se realiza con unas librerías auxiliares:

```
#define STB_IMAGE_IMPLEMENTATION
#include <stb_image.h>
```

La función principal que nos proporciona es

```
unsigned char *data = stbi_load(nome, &width, &height, &nrChannels, 0);
```

Función que lee una textura con un determinado nombre: **nome** es nombre de la textura, **width** y **height** son el ancho y alto de la textura respectivamente y **nrChannels** es el número de canales que nos devuelve RGBA (puede devolver 1 si usa un color, 2 si usa dos colores, 3 si usa RGB y 4 RGBA). Devuelve **data** que es un puntero a los datos.

De esta forma, desde el nombre de una textura podemos tener una serie de características de ella.

Ejemplo:

```
int myCargaTexturas(char *nome){
    int textura;
    // crear un índice de textura y lo cargue en la variable textura
    glGenTextures(1, &textura);
    //hacer un bind para decir que la textura operativa es la generada antes
    glBindTexture(GL_TEXTURE_2D, textura);
    //definir variables en donde se va a cargar el ancho, alto y número de canales
    int width, height, nrChannels;
    //leer textura
    unsigned char *data = stbi_load(nome, &width, &height, &nrChannels, 0);
    ...
}
```

Definir textura

Anteriormente leímos los datos de la textura más no le informamos al código lo que son dichos datos. Si nos fijamos `#include <stb_image.h>` es una librería estándar, no de OpenGL.

Por lo pronto, lo único que OpenGL sabe de la textura es lo relacionado con la variable **textura**. Ahora, queremos convertir esos datos a OpenGL, de una forma que meta los datos en la tarjeta gráfica y hacer corresponder a ese índice con esos valores.

Para esto se usa la función:

```
void glTexImage2D(GLenum objetivo, GLint nivel, GLint componentes , GLsizei ancho , GLsizei alto, GLint borde, GLenum formato, GLenum tipo, const GLvoid *pixels);
```

Ella genera una textura 2D a partir de esta imagen.

2. Mapeado de Texturas (OpenGL)

Donde:

- **objetivo:** define que textura hay que definir y debe ser GL_TEXTURE_2D (1D).
- **nivel:** Indica el nivel de detalle de las texturas y es normalmente 0.
- **componentes:** Indica el número de componentes de color usados en cada píxel.
- **ancho:** Especifica el ancho de la textura. Debe ser potencia de 2.
- **alto:** Especifica la altura de la textura. Debe ser potencia de 2.
- **borde** (como Elena con sus vacas): Controla el número de pixels de borde que el OpenGL utilizará
- **formato:** Indica el tipo de valores de color esperados.
- **tipo:** Tipo de la variable que almacena los valores de color por píxel.
- **pixels:** Puntero a la variable que almacena los colores por píxel.

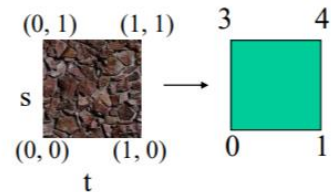
Ejemplo:

```
glTexImage2D(GL_TEXTURE_2D, 0, 3, width2, height2, 0, GL_RGB, GL_UNSIGNED_BYTE, imageData2);
```

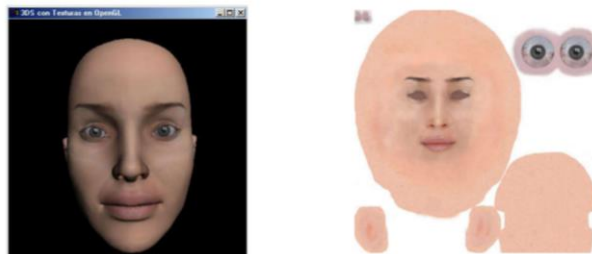
Pegar Textura

En OpenGL 1.2:

```
glBegin(GL_QUADS);  
    glTexCoord2f(0.0, 0.0); //valor de textura que se le asigna al siguiente vértice  
    glVertex3fv(vertices[0]);  
    glTexCoord2f(0.0, 1.0);  
    glVertex3fv(vertices[3]);  
    glTexCoord2f(1.0, 1.0);  
    glVertex3fv(vertices[4]);  
    glTexCoord2f(1.0, 0.0);  
    glVertex3fv(vertices[1]);  
glEnd();
```



El problema de asignar así los valores es que si se tiene una geometría muy compleja hay que ir asignando uno a uno.



Ejemplo (código tratado anteriormente, pero esta vez completo):

2. Mapeado de Texturas (OpenGL)

```
// Es necesario para que el stb_image funcione
#define STB_IMAGE_IMPLEMENTATION
#include <stb_image.h>
int myCargaTexturas(char *nome){
    int textura;
    glGenTextures(1, &textura);
    glBindTexture(GL_TEXTURE_2D, textura);
    // load image, create texture and generate mipmaps
    int width, height, nrChannels;
    unsigned char *data = stbi_load(nome, &width, &height, &nrChannels, 0);
    //Parametros
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_REPEAT);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_REPEAT);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR);

    glTexImage2D(GL_TEXTURE_2D, 0, GL_RGB, width, height, 0, GL_RGB, GL_UNSIGNED_BYTE,
    data); stbi_image_free(data);
    return (textura);
}
```

En OpenGL 1.2 se puede usar en listas:

```
void DrawGLScene(GLvoid) {
    glBindTexture(GL_TEXTURE_2D, texture[0]);
    glBegin(GL_QUADS);
        glTexCoord2f(1.0f, 1.0f);
        glVertex3f(-1.0f, -1.0f, -1.0f);
        glTexCoord2f(0.0f, 1.0f);
        glVertex3f( 1.0f, -1.0f, -1.0f);
        glTexCoord2f(0.0f, 0.0f);
        glVertex3f( 1.0f, -1.0f, 1.0f);
        TexCoord2f(1.0f, 0.0f);
        glVertex3f(-1.0f, -1.0f, 1.0f);
    glEnd();
    glBindTexture(GL_TEXTURE_2D, 0);
    glutSwapBuffers();
    glFlush();
}
```



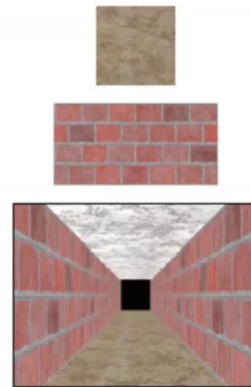
glCallList(1);

Para dibujar la textura:

```
Objeto objeto;  
void DrawGLScene(GLvoid) {  
    .....  
    glBindTexture(GL_TEXTURE_2D, objeto.texture);  
    glCallList(objeto.lista);  
    glBindTexture(GL_TEXTURE_2D, 0);  
}
```

Ejemplo con dos texturas:

```
Objeto suelo, pared;  
void Init() {  
    //generar dos texturas  
    glGenTextures (1, &suelo.texturas);  
    glGenTextures (1, &pared.texturas);  
    // textura del suelo  
    glBindTexture (GL_TEXTURE_2D, suelo.texturas);  
    glTexParameteri (...);  
    glTexImage2D (GL_TEXTURE_2D, ...);  
    // textura de la pared  
    glBindTexture (GL_TEXTURE_2D, pared.texturas);  
    glTexParameteri (...);  
    glTexImage2D (GL_TEXTURE_2D, ...);  
    ...  
}  
  
void Paint() {  
    // pinto el suelo  
    //textura que queremos aplicar  
    glBindTexture (GL_TEXTURE_2D, suelo.texturas);  
    //objeto al que queremos aplicarle la textura  
    Calllist (suelo.geometria);  
    // pinto la pared  
    glBindTexture GL_TEXTURE_2D, pared.texturas);  
    Calllist (pared.geometria);  
}
```



En OpenGL 3.3:

La carga es igual, pero tenemos que:

- 1) Especificar las coordenadas de textura.

2. Mapeado de Texturas (OpenGL)

```
float vertices[] = {
//Frontal vértices      Normales          texturas
-0.5f, -0.5f, 0.5f,      0.0f, 0.0f,1.0f,      0.0, 0.0,
0.5f, -0.5f, 0.5f,      0.0f, 0.0f,1.0f,      1.0, 0.0,
0.5f, 0.5f, 0.5f,       0.0f, 0.0f,1.0f,      1.0, 1.0,
0.5f, 0.5f, 0.5f,       0.0f, 0.0f,1.0f,      1.0, 1.0,
-0.5f, 0.5f, 0.5f,      0.0f, 0.0f,1.0f,      0.1, 1.0,
-0.5f, -0.5f, 0.5f,     0.0f, 0.0f,1.0f,      0.0, 0.0,
.....
// vertices
glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 8 * sizeof(float), (void*)0);
glEnableVertexAttribArray(0);
// Normales
glVertexAttribPointer(1, 3, GL_FLOAT, GL_FALSE, 8 * sizeof(float), (void*)(3 * sizeof(float)));
glEnableVertexAttribArray(1);
// texture
glVertexAttribPointer(2, 2, GL_FLOAT, GL_FALSE, 8 * sizeof(float), (void*)(6 * sizeof(float)));
glEnableVertexAttribArray(2);
```

```
glBindTexture(GL_TEXTURE_2D, texture);
sphere2.draw();
```

```
layout (location = 0) in vec3 aPos ;           //posicion
layout (location = 1) in vec3 aColor ;         //color
layout (location = 2) in vec2 aTexCoord ;     //textura ->pasarlas al fragment shader

out vec2 TexCoord ;
out vec3 ourColor ;
uniform mat4 transform ;
void main() {
    gl_Position = transform * vec4(aPos, 1.0);
    ourColor = aColor ;
    TexCoord = vec2(aTexCoord.x, aTexCoord.y);
}
```

```
#version 330 core
in vec3 ourColor ;
in vec2 TexCoord ;
uniform sampler2D texture1;
void main() {
```

```
gl_FragColor = texture(texture1, TexCoord);  
}  
}
```

Parámetros de textura

...

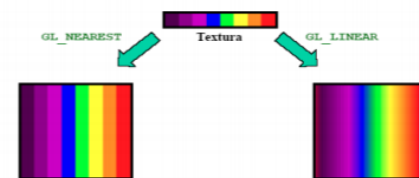
```
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_REPEAT);  
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_REPEAT);  
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR);  
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR);
```

...

- Sobre el Objeto: Problemas de tamaño Que hacer si la textura no abarca todo el objeto.



- Sobre la pantalla: Problemas de escala: Que hacer al rasterizar la textura es más grande o pequeña que su tamaño original. Ocupa un más de un píxel o menos sobre la pantalla.



- Otros efectos Mezclas: Que hacer si el objeto final tenía un color.



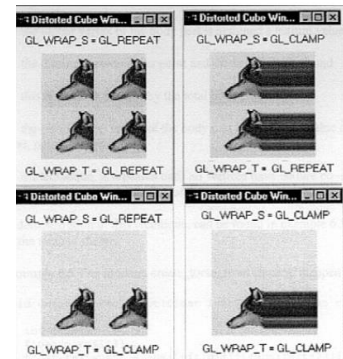
2. Mapeado de Texturas (OpenGL)

Problemas de tamaño: glTexParameterf

Especifica que ocurre si la textura no cubre a los límites del objeto Warp o warping process. Así especifica si es repetida repeat o estirada clamp. Por tanto, la función debe ser llamada 4 veces.

```
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_REPEAT);
```

```
glTexParameterf(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_CLAMP);
```

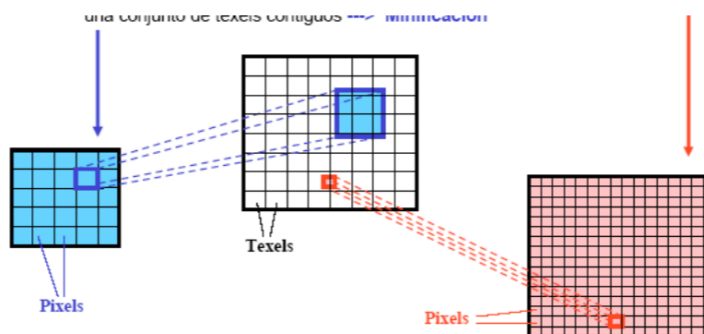


Problemas de escala: glTexParameterf

¿Qué pasa si cada píxel de la imagen no se corresponde en tamaño a lo que se ve en pantalla?

Al mapear la imagen de textura sobre una superficie, los texels no se van a corresponder exactamente con los pixels

- Si la superficie es mayor que la textura, cada pixel se corresponderá con un trozo pequeño de texel -> Magnificación
- Si la superficie es menor que la textura, cada pixel se corresponderá con una conjunto de texels contiguos -> Minificación Texels



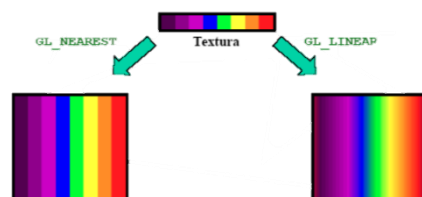
En OpenGL existen 4 tipos de simplificación:

GL_TEXTURE_MAG_FILTER

GL_TEXTURE_MIN_FILTER

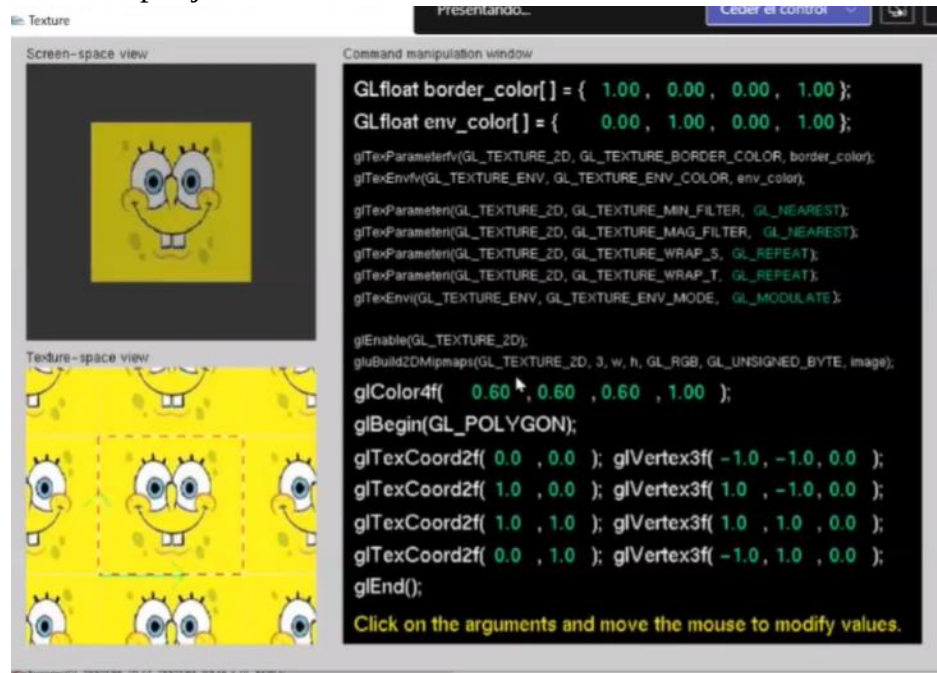
GL_LINEAR

GL_NEAREST



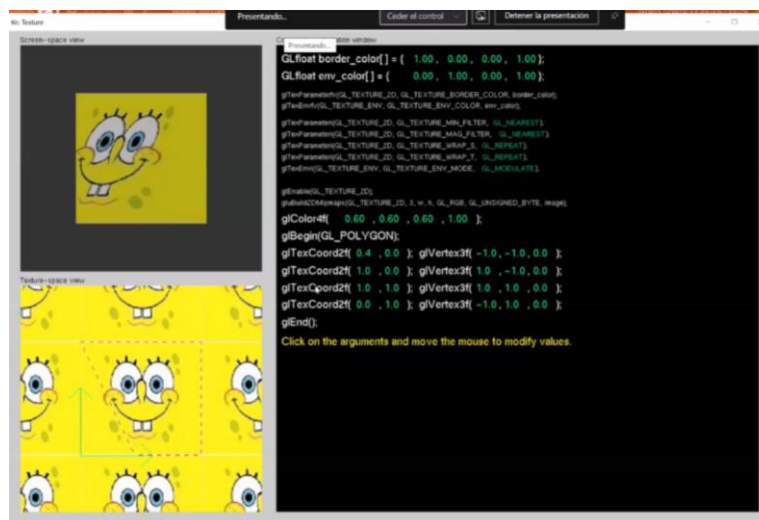
2. Mapeado de Texturas (OpenGL)

Ejemplo Bob Esponja

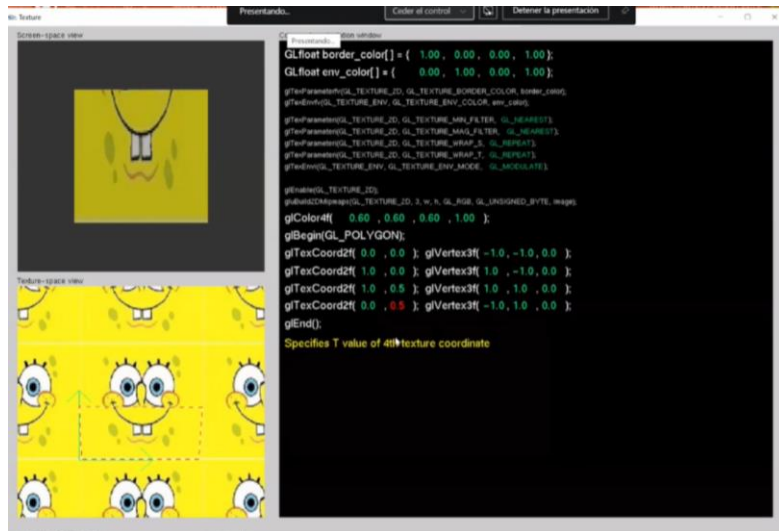


Dentro de la línea discontinua está la textura original.

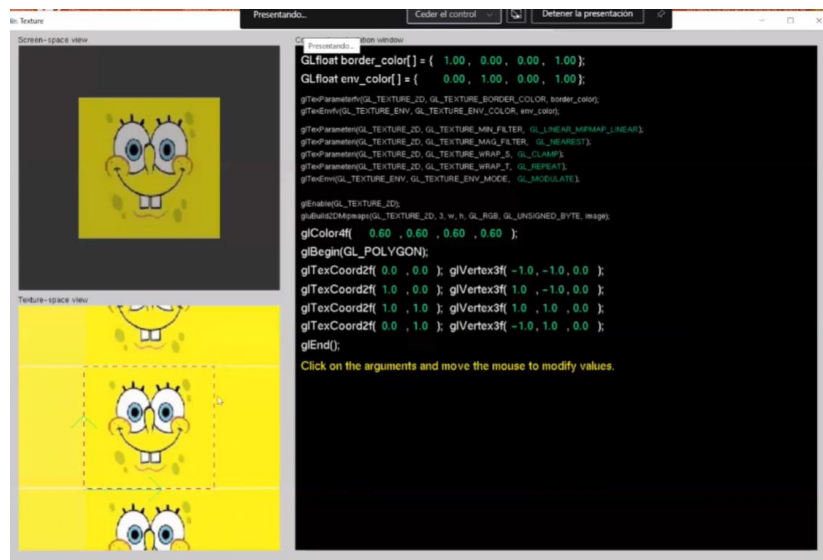
Cambiando las coordenadas para cambiar el ajuste de la textura:



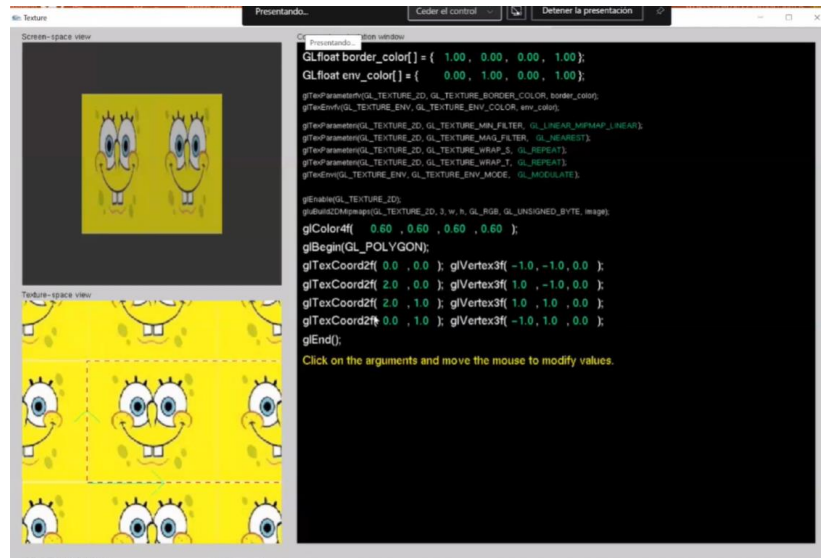
2. Mapeado de Texturas (OpenGL)



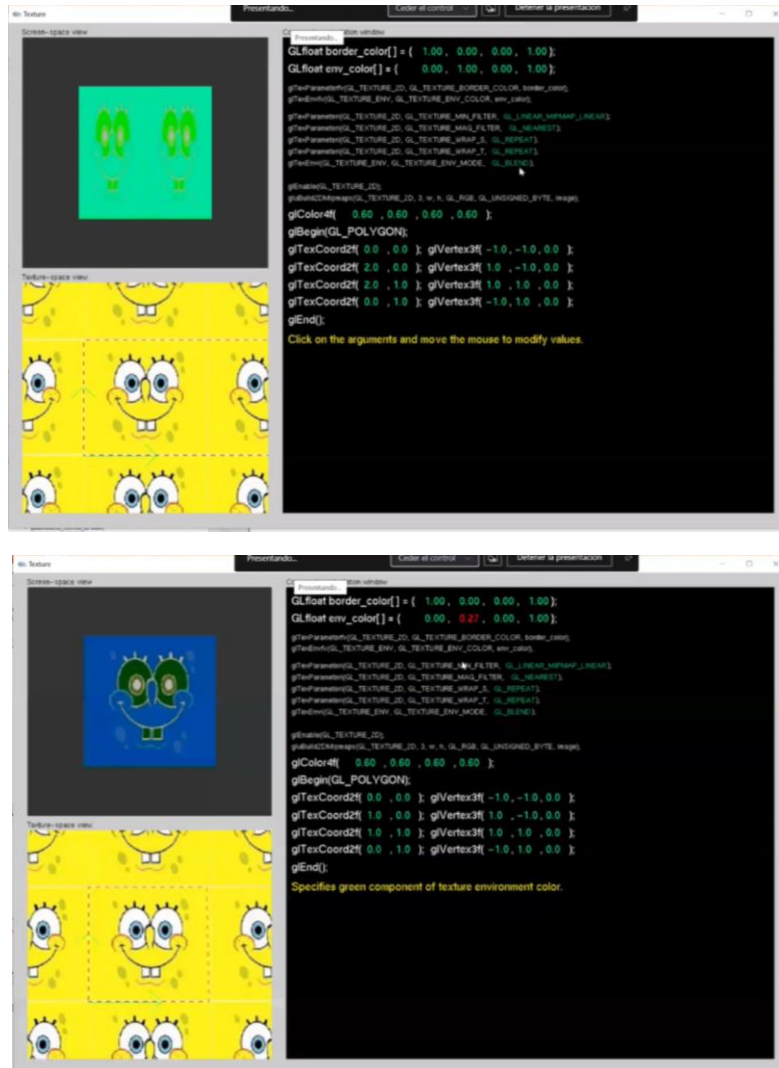
Ahora observaremos los cambios al variar los parámetros de la textura:



2. Mapeado de Texturas (OpenGL)

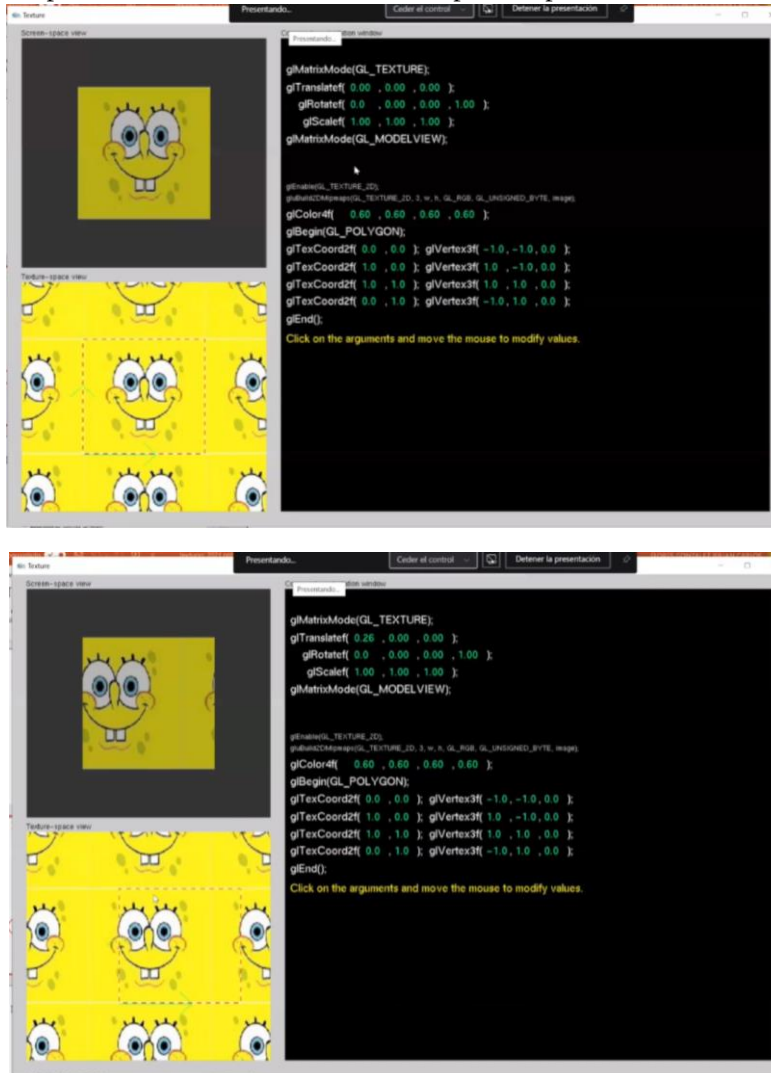


El GL_MODULATE indica cómo se mezcla el color con la textura. Hay varias opciones de mezcla.



2. Mapeado de Texturas (OpenGL)

Los parámetros de texturas también se pueden poner como matrices.



Mapeado multicapa 3.3

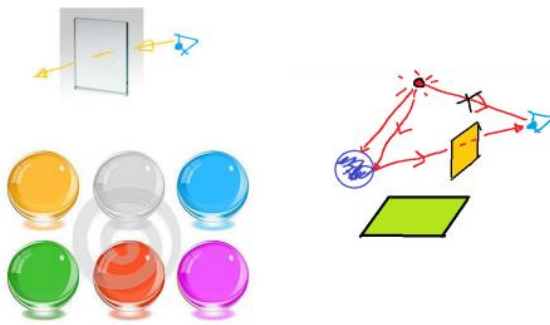
En OpenGL 3.3 es posible y fácil utilizar más de una textura a la vez sobre un objeto.

Para esto se utilizan las denominadas Unidades de texturas que permiten ir activando o desactivando las texturas manualmente.

```
glActiveTexture(GL_TEXTURE0);
glBindTexture(GL_TEXTURE_2D, texture);
glActiveTexture(GL_TEXTURE1);
glBindTexture(GL_TEXTURE_2D, texture2);
```

Canales Alpha

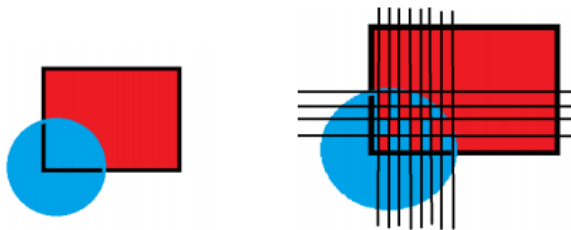
OpenGL puede simular los objetos transparentes.



Existe una aplicación por defecto para generar un Alpha channel.

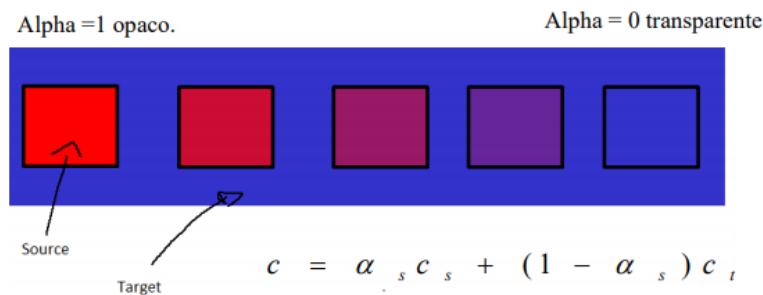
Blending-Mezcla

Es la forma más sencilla de hacer una transparencia, es mediante la técnica de la puerta o ventana transparente. Lo que hace es coger el color pixel que tiene detrás y mezclarlo.



2. Mapeado de Texturas (OpenGL)

Existe lo que se llama color fuente y color objetivo, entonces en función de los parámetros de cada uno de ellos se le aplica unos valores o otros, en este caso el Alpha 1 que es totalmente opaco (es el que utilizamos nosotros) y el alpha 0 es transparente.



El color fuente suele ser el que está delante y el color destino el que está detrás, entonces la ecuación es una interpolación lineal que termina de la siguiente forma: $c = \alpha_s c_s + (1 - \alpha_s) c_t$.

Test Alpha

- En RGBA modelo acepta el valor alpha. Para ello las texturas o colores deben tener un alpha por ejemplo .png .gifa, Se habilita con `glEnable(GL_ALPHA_TEST);`
- `glDisable(..);`
- El test compara el valor del fragmento con un valor de referencia, por defecto este valor el 0 y la comparación se hace entre en 0 - 1. Por defecto la función es “always” pero acepta otros valores pasados por la función.
- `glAlphaFunc(GLenum func, GLclampf ref)`
- FuncG L_NEVER, GL_LESS, GL_EQUAL, GL_LEQUAL, GL_GREATER, GL_NOTEQUAL, GL_GEQUAL, and GL_ALWAYS son aceptadas
- El valor inicial es GL_ALWAYS.
- Para cualquier otro elemento. De color RGBA (1,0.1,0.2, 0.8).
- `glEnable (GL_BLEND);`
- `glBlendFunc (GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);`

El alpha test es el que se encarga de usar los canales alpha, y si nosotros tenemos códigos de color R,G,B,A con valores por ejemplo en este caso 0.8 será transparente.

Alpha+Textures

Nosotros solo trabajaremos con texturas que tengan ya incorporado canal apha PNG o GIFA, para lo cual tendremos en cuenta el número de canales que tenemos. Deberemos por tanto incluir en la función de la lectura de la textura el número de canales.

2. Mapeado de Texturas (OpenGL)

```
unsigned char *data = stbi_load(path, &width, &height, &nrComponents, 0);
if (data) {
    if (nrComponents == 1)
        format = GL_RED;
    else if (nrComponents == 3)
        format = GL_RGB;
    else if (nrComponents == 4)
        format = GL_RGBA;
    glTexImage2D(GL_TEXTURE_2D, 0, format, width, height, 0, format, GL_UNSIGNED_BYTE, data);
}
```

RECORDAR
glEnable(GL_BLEND); glBlendFunc(GL_SRC_ALPHA,
GL_ONE_MINUS_SRC_ALPHA);

Para cualquier otro elemento.
De color RGBA (1,0.1,0.2, 0.8).

Prácticas

Vamos a hacer dos cosas un tanto curiosas, una es hacer el sistema solar texturizado e iluminado y en la grúa vamos a hacer texturizado y a meter una ‘piscina’.

Preguntas

No es obligatorio hacer las prácticas optativas, pero espera que si pregunta algo sepamos responder.



Escola
Técnica
Superior
de Enxeñaría



Grado en Ingeniería Informática

2º curso – 2º cuatrimestre

Computación Gráfica

14-04-2021

Clase 09

Temas trabajados:

Tema 7

Índice

1. Buffer de profundidad	3
2. Atenuación de la luz en función de la distancia	5
3. Niebla	6
4. Mipmapping	7
5. Skybox	7
6. Colisiones	7
Choque esfera-esfera	7
Choque caja-caja	8
Optimizaciones de las colisiones	8
7. Información sobre el examen	9

1. Buffer de profundidad

El z-buffer o buffer de profundidad, almacena la información de cada fragmento con el objetivo de comparar los valores de profundidad de cada uno con el anterior para saber cuál está más alejado y descartarlo en el buffer de color: por defecto en OpenGL, si el valor de profundidad del nuevo fragmento que llega es menor que el anterior, entonces se dibuja el nuevo fragmento, sino no.

Generalmente, el buffer posee el mismo ancho y alto que el buffer de color, con el que también comparte coordenadas (aunque solo se suele emplear la coordenada de color para **calcular el color en función de la profundidad**), poseyendo además un valor de profundidad, generalmente de 24 bits.

Este buffer actúa a continuación del *fragment shader*. Sin embargo, en vez de realizar las operaciones del *fragment shader* sobre cada fragmento cuando no todos se van a dibujar, en la mayoría de GPUs actualmente se realiza lo que se conoce como *profundidad temprana*, que consiste en realizar una comprobación de profundidades de los fragmentos previa al *fragment shader*, de manera que aquellos fragmentos que no se van a ver/dibujar ya no entran en dicho shader, evitando realizar los cálculos del color y favoreciendo el rendimiento.

Las funciones que activan y desactivan respectivamente el buffer de profundidad son las siguientes:

```
glEnable( GL_DEPTH_TEST);  
glClear (GL_DEPTH_BUFFER_BIT);
```

Puede resultar de utilidad deshabilitar temporalmente el z-buffer, ya que permite colocar encima de objetos ya dibujados aquellos que se dejaron para el final, por ejemplo: para dibujar un circuito de carreras, se puede dibujar primero el circuito, desactivar el buffer de profundidad y a continuación dibujar los coches, de manera que estos se dibujarán sobre del circuito al sobrescribirse en el buffer de color.

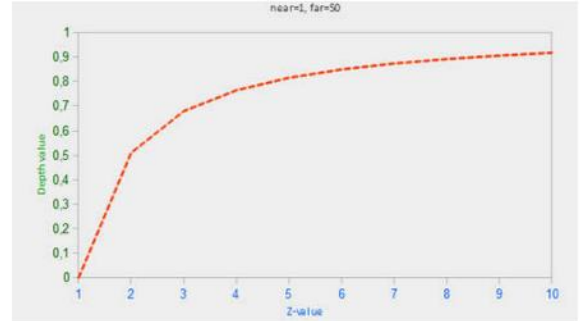
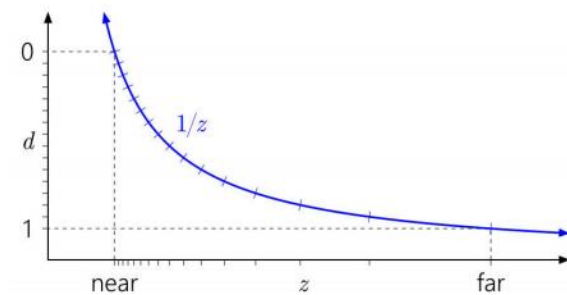
La comparación realizada para descartar los fragmentos en función de sus valores de profundidad se puede realizar respecto a diferentes comparadores: GL_NEVER, GL_LESS (comparador por defecto), GL_EQUAL, GL_LEQUAL, GL_GREATER, GL_NOTEQUAL, GL_GEQUAL, and GL_ALWAYS. Se puede establecer cualquiera de los anteriores comparadores para controlar dicha acción mediante la función `glDepthFunc`, por ejemplo: `glDepthFunc(GL_ALWAYS)` hace que los fragmentos sean siempre escritos, lo que equivaldría a implementar el algoritmo del pintor.

Los valores de profundidad varían entre 0 (cuando un fragmento está muy cerca) y 1 (cuando un fragmento está muy lejos). Una forma de calcularlo es mediante la fracción $1/z$, de manera que en función de la distancia tomará un valor entre 1 y 0. Sin embargo, actualmente el cálculo de los valores de profundidad se hace mediante la siguiente ecuación:

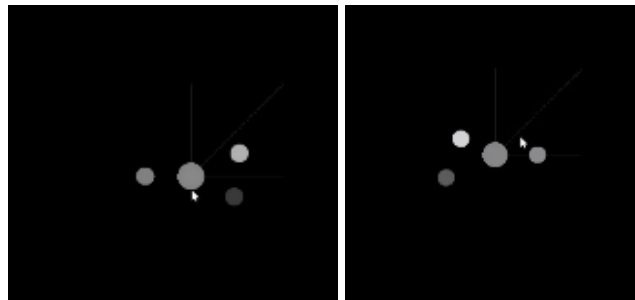
$$F_{depth} = \frac{\frac{1}{z} - \frac{1}{near}}{\frac{1}{far} - \frac{1}{near}}$$

1. Buffer de profundidad

Aplicación esta ecuación, se tiene que a partir de una distancia de 10 el valor de profundidad es muy próximo a 1, habiendo una mayor resolución a la izquierda de la gráfica (antes del valor 10), lo que garantiza que los objetos más cercanos tengan una resolución mayor, como ocurre en la vida real.

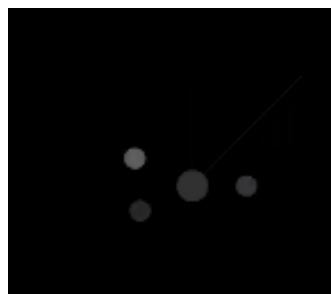


En OpenGL 3.3 existen muchas más posibilidades de trabajar con el buffer de profundidad que en 1.2: obtener el valor del z-buffer mediante la variable `gl_FragCoord.z`, se puede cambiar el color de un fragmento y su posición, mover un fragmento (no es muy habitual), **hacer que el color de un fragmento sea proporcional a su profundidad...**



En las imágenes anteriores se puede ver como se modifica el color de las esferas en función de la distancia a la cámara, siendo las más claras aquellas que están más próximas, y las más oscuras, las más alejadas. El color varía en gran medida debido a no seguir la ecuación lineal.

En caso de emplear la ecuación lineal los colores diferirían en menor medida, como se observa en la siguiente imagen:



De entre los problemas que se pueden encontrar relacionados con el buffer de profundidad destaca el conocido como **Z-fighting**, que consiste en un conflicto entre colores de diferentes objetos colocados muy próximos entre sí sobre un mismo fragmento, observándose patrones extraños de color como resultado. Para solventarlo existen 3 posibles soluciones:

- Aumentar la precisión del z-buffer (no es siempre posible).
- Modificar el *near* (lo cual suele ser posible) y el *far* (lo cual a veces no es posible).
- Separar un poco los objetos entre sí (es la más utilizada). Por ejemplo: en la práctica de la grúa, para evitar que se dé este problema entre el suelo y la base de la grúa, se puede dibujar la base un poco elevada con respecto al suelo.

2. Atenuación de la luz en función de la distancia

Para tener en cuenta cómo afecta la distancia de la luz a un objeto la ecuación de la intensidad de la luz se modifica de la siguiente manera:

$$I = \frac{1}{a + b \cdot d + c \cdot d^2} \cdot k_a \cdot L_a + k_d \cdot L_d \cdot l \cdot n + k_s \cdot L_s \cdot (r \cdot v)^\alpha$$

Con ella se consigue que la intensidad de iluminación de las luces decrezca a medida que aumenta la distancia, salvo para las luces situadas en el infinito.

Los subíndices *a*, *d* y *s* hacen referencia a las componentes de la luz: de ambiente, difusa y especular, respectivamente.

Tanto en 1.2 como en 3.3 se puede hacer que la intensidad dependa de una serie de parámetros, generalmente de una ecuación, ya sea lineal o cuadrática, como la observada en la fórmula anterior. En dicha ecuación, se tienen 3 constantes, *a*, *b* y *c* (*d* es distancia al observador), que permitirán configurar la intensidad lumínica. Estas constantes se corresponden con los siguientes nombres en OpenGL:

a: GL_CONSTANT_ATTENUATION

b: GL_LINEAR_ATTENUATION

c: GL_QUADRATIC_ATTENUATION.

Para modificar sus valores por defecto (*a*=1, *b*=0 y *c*=0), se puede emplear la función void `glLightf(GL_LIGHTx, GL_CONSTANT_ATTENUATION, value)`, donde el segundo parámetro será una de las constantes anteriormente introducidas (en este caso, *a*) y el tercero, el valor que se le desea proporcionar.

El uso de la ecuación de atenuación influye negativamente en eficiencia del cálculo.

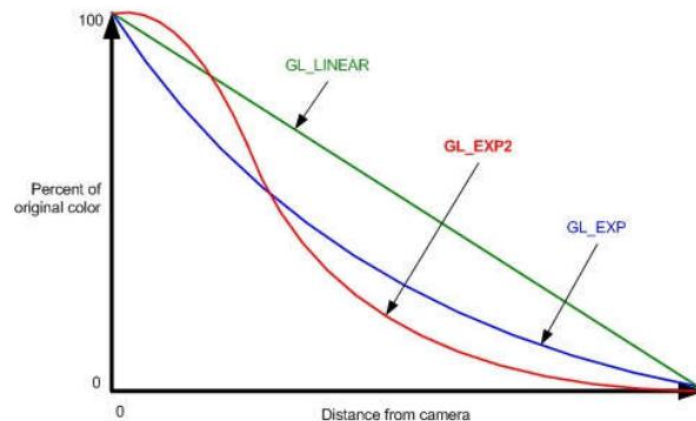
3. Niebla

Consiste en un efecto especial que difumina los objetos de la escena en función de la distancia (generalmente consiste en difuminar el color, oscureciendo los objetos más alejados, que llegan incluso a desaparecer, mientras que los más cercanos permanecen más claros).

Para activarlo en OpenGL 1.2 simplemente se debe llamar a la función `glEnable(GL_FOG)`.

Se pueden modificar los parámetros de densidad (`GL_FOG_DENSITY`, normalmente es 1), comienzo de la niebla (`GL_FOG_START`), final (`GL_FOG_END`), el indexado (`GL_FOG_INDEX`) y el color (`GL_FOG_COLOR`, por defecto es blanca) mediante la función `void glFogfv(GLenum pname, const GLfloat *params)`.

Otra característica que se puede modificar es la forma que tiene de presentarse la niebla: lineal (`GL_LINEAR`, gráfica verde), exponencial (`GL_EXP`, gráfica azul) y exponencial 2 (`GL_EXP2`, gráfica roja). La función que la permite modificar es `glFogi (GL_FOG_MODE, GL_LINEAR)`.



La exponencial 2 se diferencia de la exponencial debido a que al principio atenúa poco mientras que en el medio atenúa en gran medida. Las ecuaciones correspondientes a cada uno de estos modos son las siguientes:

$$f = fin - \frac{z}{fin} - comienzo$$

$$f = e^{-(densidad \cdot z)}$$

$$f = e^{-(densidad \cdot z)^2}$$

4. Mipmapping

El concepto de *mipmapping* hace referencia a las texturas. Permite disminuir el trabajo de la GPU al, por ejemplo, tener una imagen con una resolución 10000x10000 que se pretende representar en un solo píxel. Esta interpolación sometería a la GPU a una gran cantidad de trabajo. Para evitarlo, se puede hacer que automáticamente la GPU haga copias más pequeñas de la imagen original y en función de la distancia a la que se encuentre la cámara del objeto, escoja la imagen más adecuada en relación al píxel de la imagen y al píxel de la pantalla (nivel de detalle).

Hay que modificar la función que lee las texturas para poder trabajar con *mipmaps*. A continuación, se muestra un ejemplo de uso:

```
unsigned char *data = stbi_load(path, &width, &height, &nComponents, 0);
if (data) {
    //con mipmap
    gluBuild2DMipmaps(GL_TEXTURE_2D, GL_RGB, width, height, GL_RGB, GL_UNSIGNED_BYTE, data);
    glGenerateMipmap(GL_TEXTURE_2D);
}
```

La función concreta que genera *mipmaps* es `gluBuild2DMipmaps`, donde se indica si la textura es 2D, la profundidad de color, el ancho, el alto, el tipo de dato y el dato.

```
gluBuild2DMipmaps(GL_TEXTURE_2D, GL_RGB, ancho, alto, GL_RGBA, GL_UNSIGNED_BYTE, ptr);
```

5. Skybox

Permite generar una cubierta para el escenario. Para ello, se suele crear una caja (o una esfera), con las normales hacia dentro, sobre la que se pega una imagen que cubre toda la escena. Por ejemplo, en el Sistema Solar podría crearse un *Skybox* para representar un fondo estrellado.

6. Colisiones

Las colisiones consisten en la interacción de diferentes objetos entre sí, al interactuar la cara de uno con la de otro. Para detectarlas, cada objeto se introduce en lo que se conoce como *Bounding Box*, que consiste en el contenedor mínimo (generalmente un cubo o una esfera, aunque también se suelen usar cilindros para objetos como son los cuerpos de los personajes) que engloba al objeto, permitiendo simplificar el cálculo de las colisiones, ya que los objetos no siempre tienen formas simples con pocas caras, por lo que analizar si cada cara de un objeto complejo está interactuando con cada cara de otro objeto complejo sería computacionalmente inviable.

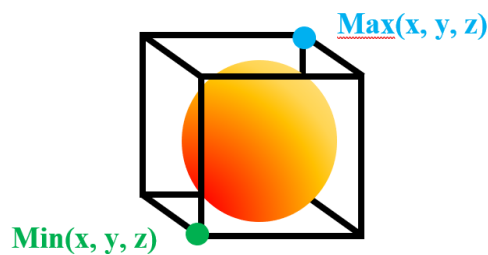
Choque esfera-esfera

En el caso de que la *Bounding Box* sea una esfera, esta se caracterizaría por su radio, así como la posición de su centro. De esta forma, para detectar colisiones entre objetos cuyas

Bounding Box son esferas en ambos casos lo que hay que tener en cuenta es si la distancia desde un centro al otro es menor o igual que la suma de los radios de ambas esferas, en cuyo caso se produciría una colisión.

Choque caja-caja

En el caso de que la *Bounding Box* sea una caja (un cubo), esta se caracteriza por tener un punto mínimo y otro máximo, que se corresponderían con las esquinas opuestas de la misma, como se observa en la figura siguiente, las cuales se ajustan en función del tamaño del objeto.



A la hora de estudiar las colisiones entre *Bounding Box* cúbicas hay que tener en cuenta que cuando el objeto gire mediante una rotación la *Bounding Box* rotará con él, por lo que a la hora de interactuar una cara de la *Bounding Box* de un objeto con la de otro hay que tener en cuenta si estas están alineadas o no:

- Si las caras están alineadas (la *Bounding Box* no se ha rotado), que es el caso más sencillo de calcular y computacionalmente barato, se calcula que ha habido colisión en caso de que alguna esquina de una de las *Bounding Box* esté dentro de la otra *Bounding Box*, es decir si para dos *Bounding Box* A y B:

$$(A_Min.x < B_Max.x \text{ and } A_Max.x > B_Min.x) \text{ and}$$

$$(A_Min.y < B_Max.y \text{ and } A_Max.y > B_Min.y) \text{ and}$$

$$(A_Min.z < B_Max.z \text{ and } A_Max.z > B_Min.z)$$

- Si las caras no están alineadas (lo cual puede evitarse al programar que las rotaciones solo afecten al objeto y no a la *Bounding Box*), suele trazarse una nueva *Bounding Box* con unos nuevos puntos mínimo y máximo en base a las nuevas coordenadas del objeto.

Optimizaciones de las colisiones

Para optimizar el cálculo de colisiones se pueden emplear técnicas como el uso de **árboles**, de manera que no solo los objetos tienen su propia *Bounding Box*, sino que también cada una de sus partes, de manera que primero se comprueban las colisiones en las *Bounding Box* mayores y posteriormente en las más pequeñas.

Por otro lado, a la hora de programar, por ejemplo, los límites de una pared que un usuario/jugador no puede traspasar se puede **particionar el espacio**, delimitando las diferentes

zonas de este. Otra opción sería el uso de un **mapa**, que es menos preciso, pero permite delimitar áreas (representadas por celdas) en las que se puede estar o no estar, de manera que en caso de intentar entrar en áreas no permitidas, se colisionaría.

7. Información sobre el examen

Para el examen es necesario conocer la práctica, ya que probablemente se proporcionará un objeto sobre el que se querrá aplicar una transformación y será necesario especificar el tipo de la misma o incluso hallar la matriz de transformación.

En cuanto a las ecuaciones de la luz ambiente, difusa y especular es necesario conocerlas.